

REPUBLIQUE DU BENIN

MINISTERE DE L'ENSEIGNEMENT SUPÉRIEUR ET

DE LA RECHERCHE SCIENTIFIQUE



ECOLE PIGIER-BENIN

DOMAINE : SCIENCES ET TECHNOLOGIES

MENTION : INFORMATIQUE

SPÉCIALITÉ : INTELLIGENCE ARTIFICIELLE & BIG DATA

THÈME

**SYSTÈME DE VISION PAR ORDINATEUR POUR LA
DéTECTION PRÉCOCE, L'IDENTIFICATION ET LA
LOCALISATION DE MALADIES ET RAVAGEURS DANS LES
CULTURES AGRICOLES : CAS D'ÉTUDE DU MAÏS**

Réalisé et présenté par :

ANANI COFFI CHARLY KENNETH

Sous la supervision :

M.C. ABALLO THÉOPHILE

Session de Juin 2025

Engagement

L'école PIGIER-BÉNIN n'entend donner aucune approbation ni désapprobation aux opinions émises dans ce mémoire. Ces opinions doivent être considérées comme propres à leurs auteurs.

Dédicaces

Dédié à mes très chers parents et généreuses tantes pour leur soutien et amour durant mon parcours académique. Profonde gratitude pour leur présence constante et guidance, merci infiniment.

Charly ANANI

Remerciements

Nous souhaitons exprimer notre profonde gratitude à toutes les personnes qui ont, de près ou de loin, contribué à la réalisation de ce travail. En signe de reconnaissance, nous tenons à leur adresser nos sincères remerciements. Nous remercions tout particulièrement :

✚ Monsieur **ABALLO Théophile**, pour avoir accepté de diriger ce mémoire avec bienveillance.

✚ L'école **PIGIER-BÉNIN**, ainsi que l'ensemble des membres de l'administration et du corps enseignant, pour la qualité de la formation dispensée et l'accompagnement tout au long de notre parcours académique.

Nous tenons également à exprimer notre reconnaissance :

✚ À nos camarades de promotion, pour leur esprit de collaboration, leur soutien moral et les échanges enrichissants.

✚ À notre famille, pour leur appui constant, leur soutien et leurs encouragements durant cette période exigeante.

✚ Les membres du jury, pour le temps consacré à l'évaluation de ce travail, leurs remarques pertinentes et leurs suggestions enrichissantes.

Enfin, à toutes les personnes ayant apporté un regard critique et constructif .

Sigles Acronymes Et Abréviations

AI : Artificial Intelligence

ML : Machine Learning

DL : Deep Learning

NN : Neural Networks

FC : Fully Connected

CNN : Convolutional Neural Network

CV : Computer Vision

YOLO : You Only Look Once

SSD : Single Shot MultiBox Detector

RT-DETR : Real-Time Detection Transformer

OBB : Oriented Bounding Box

COCO : Common Objects in Context

NMS : Non-Maximum Suppression

IoU : Intersection over Union

mAP : mean Average Precision

AP : Average Precision

mAP50 : mean Average Precision at IoU 0.5

mAP50-95 : mean Average Precision averaged over IoU thresholds from 0.5 to 0.95

RGB : Red Green Blue

XAI : Explainable AI

CPU : Central Processing Unit

GPU : Graphics Processing Unit

TFLite : TensorFlow Lite

iOS : iPhone Operating System

GPS : Global Positioning System

CUDA : Compute Unified Device Architecture

JPEG : Joint Photographic Experts Group

JPG : Joint Photographic Group (format d'image)

XML : eXtensible Markup Language

CSV : Comma-Separated Values

XAI : Explainable Artificial Intelligence

PCA : Principal Component Analysis

CAM : Class Activation Map

Liste Des Figures

FIGURE 1:ARCHITECTURE D’UN RESEAU DE NEURONES PROFOND (DEEP NEURAL NETWORK)	9
FIGURE 2:ARCHITECTURE TYPIQUE D’UN RESEAU DE NEURONES CONVOLUTIFS (CNN) COMPRENANT LES COUCHES DE CONVOLUTION, DE POOLING ET ENTIEREMENT CONNECTEE	13
FIGURE 3:REPRESENTATION HIERARCHIQUE DES CARACTERISTIQUES APPRISES PAR LES COUCHES CONVOLUTIONNELLES D’UN CNN LORS DE LA DETECTION D’UN OBJET (EXEMPLE : UN VELO).....	14
FIGURE 4: ILLUSTRATION DU PROCESSUS DE CONVOLUTION ENTRE UNE IMAGE ET UN FILTRE DANS UNE COUCHE CONVOLUTIVE	15
FIGURE 5:IMAGE EN COULEUR D'UNE FEUILLE DE MAÏS	16
FIGURE 6:LOGO OFFICIEL DE KAGGLE	27
FIGURE 7:LOGO OFFICIELLE DE PYTORCH	28
FIGURE 8:– LOGO OFFICIEL DE L’ENTREPRISE NVIDIA.....	29
FIGURE 9:LOGO OFFICIEL DE LA BIBLIOTHEQUE OPENCV	30
FIGURE 10:LOGO OFFICIEL DE LA DISTRIBUTION ANACONDA	30
FIGURE 11:LOGO OFFICIEL DE JUPYTER NOTEBOOKS	31
FIGURE 12: LOGO OFFICIEL DU LANGAGE PYTHON	31
FIGURE 13:LOGO OFFICIEL DE LA BIBLIOTHEQUE TENSORFLOW	32
FIGURE 14::LOGO OFFICIEL DE LA BIBLIOTHEQUE TENSORFLOW LITE	32
FIGURE 15:LOGO OFFICIEL DE STREAMLIT	33
FIGURE 16:LOGO OFFICIEL DE L’ORGANISATION ULTRALYTICS, CREATRICE DE YOLO11.....	34
FIGURE 17:ARCHITECTURE MODULAIRE DE YOLO11 : BACKBONE, NECK ET HEAD	36
FIGURE 18:COMPARAISON DES MESURES DE PERFORMANCE DES CINQ VARIANTES	

DE YOLO.....	37
FIGURE 19:SCHEMA EXPLICATIF DU BLOC C3_K2 OPTIMISE DANS L'ARCHITECTURE YOLO11.....	38
FIGURE 20:SCHEMA DU MODULE SPFF (SPATIAL PYRAMID FAST FUSION) DANS YOLO11.....	38
FIGURE 21:SCHEMA DU MODULE C2_PSA (CROSS STAGE PARTIAL + POSITION-SENSITIVE ATTENTION) DANS YOLO11	39
FIGURE 22:SCHEMA ILLUSTRANT L'OPERATION D'UPSAMPLING (SURECHANTILLONNAGE) PAR METHODE DU PLUS PROCHE VOISIN	39
FIGURE 23:SCHEMA DE L'OPERATION DE CONCATENATION DES CARTES DE CARACTERISTIQUES LE LONG DE L'AXE DES CANAUX	40
FIGURE 24:– SCHEMA DU BLOC DETECT : PREDICTION FINALE DES OBJETS A DIFFERENTES ECHELLES DANS YOLO11	40
FIGURE 25:ARCHITECTURE LOGIQUE GLOBALE DE LA SOLUTION DE DETECTION BASEE SUR YOLO11N.....	41
FIGURE 26:HISTOGRAMME DES ANNOTATIONS PAR CLASSE DATA.CSV	45
FIGURE 27:– HISTOGRAMME DES ANNOTATIONS EQUILIBREES PAR CLASSE DATA.CSV	46
FIGURE 28:SCHEMATISATION DES BOITES ENGLOBANTES ET DE LEURS COORDONNEES	47
FIGURE 29:DISTRIBUTION DE LA CLASSE FALL ARMYWORM LARVA DANS L'ENSEMBLE D'ENTRAINEMENT.....	48
FIGURE 30:DISTRIBUTION DE LA CLASSE HEALTHY MAIZE DANS L'ENSEMBLE D'ENTRAINEMENT	49
FIGURE 31:DISTRIBUTION DE LA CLASSE MAIZE STREAK DISEASE DANS L'ENSEMBLE D'ENTRAINEMENT.....	49
FIGURE 32:TREEMAP REPRESENTANT LA TAILLE MOYENNE DES BOITES ENGLOBANTES PAR CLASSE PHYTOSANITAIRE, ISSUE DES ANNOTATIONS D'ENTRAINEMENT ET GENEREE A L'AIDE DE LA BIBLIOTHEQUE SQUARIFY	51

FIGURE 33:PREMIERE SERIE DE VISUALISATIONS DES ANNOTATIONS : MALADIE DU MAÏS ET PLANTE SAINNE	52
FIGURE 34:DEUXIEME SERIE DE VISUALISATIONS DES ANNOTATIONS : RAVAGEURS FALL_ARMYWORM ET PLANTE SAINNE.....	53
FIGURE 35: VISUALISATION DES MODELES EXPORTEES AU FORMAT TENSORFLOW LITE : FLOAT32, FLOAT16 ET INT8	68
FIGURE 36:— ÉVOLUTION DES PERFORMANCES LORS DES 10 PREMIERES ET 10 DERNIERES EPOQUES DE L'ENTRAINEMENT YOLO.....	71
FIGURE 37:MATRICE DE CONFUSION DES PREDICTIONS PAR CLASSE	72
FIGURE 38:COURBES DE PRECISION EN FONCTION DU SEUIL DE CONFIANCE POUR CHAQUE CLASSE	73
FIGURE 39:COURBES DE RAPPEL EN FONCTION DU SEUIL DE CONFIANCE POUR CHAQUE CLASSE	74
FIGURE 40:COURBES DE PRECISION RAPPEL	76
FIGURE 41:COURBES D'EVOLUTION DES PERTES ET DES METRIQUES DE PERFORMANCE SUR LES JEUX D'ENTRAINEMENT ET DE VALIDATION AU COURS DE L'ENTRAINEMENT DE NOTRE MODELE.	78
FIGURE 42:RESULTATS D'EVALUATION DU MODELE SUR LE JEU DE TEST : METRIQUES GLOBALES ET PAR CLASSE (PRECISION, RAPPEL, MAP@0.5, MAP@0.5:0.95).....	83
FIGURE 43: MATRICE DE CONFUSION NORMALISEE SUR LE JEU DE TEST	84
FIGURE 44:— COURBES DE PRECISION EN FONCTION DU SEUIL DE CONFIANCE POUR CHAQUE CLASSE SUR LE JEU DE TEST	85
FIGURE 45:COURBES DE RAPPEL EN FONCTION DU SEUIL DE CONFIANCE POUR CHAQUE CLASSE SUR LE JEU DE TEST	87
FIGURE 46:COURBES DE PRECISION/RAPPEL SUR LE JEU DE TEST (PRECISION- RECALL CURVE).....	89
FIGURE 47:EFFET D'UNE MAUVAISE ORIENTATION EXIF SUR LES PREDICTIONS DU MODELE.....	91

FIGURE 48:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE FALL_ARMYWORM	92
FIGURE 49:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE MAIZE_STREAK....	92
FIGURE 50:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE HEALTHY	93
FIGURE 51:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE HEALTHY	93
FIGURE 52:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE MAIZE_STREAK....	94
FIGURE 53:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE FALL_ARMYWORM	94
FIGURE 54:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE FALL_ARMYWORM	95
FIGURE 55:EXEMPLE D’INFERENCE CORRECTE D’UN CAS VRAIMENT PRECOCE SUR LA CLASSE MAIZE_STREAK	95
FIGURE 56:EXEMPLE D’INFERENCE CORRECTE SUR LA CLASSE FALL_ARMYWORM	96
FIGURE 57:MODELES TENSORFLOW LITE EXPORTES POUR UNE UTILISATION SUR DIFFERENTS TYPES DE DISPOSITIFS AGRICOLES EMBARQUES.....	100
FIGURE 58: INTERFACE STREAMLIT AFFICHANT UNE SIMULATION DE DETECTION DE MALADIES OU DE RAVAGEURS EN TEMPS REEL AU SEIN D’UN FLUX VIDEO....	101
FIGURE 59:INTERFACE STREAMLIT AFFICHANT UNE SIMULATION DE DETECTION DE MALADIES OU DE RAVAGEURS AU SEIN D’UNE IMAGE DE PLANTATION DE MAIS ATTEINT DU VIRUS DE LA STRIURE	102

Liste Des Tableaux

TABLEAU 1: COMPLEMENTARITE ET DIFFERENCES ENTRE TRANSFER LEARNING ET FINE-TUNING	18
TABLEAU 2 : PRINCIPALES MENACES DU MAÏS ABORDEES DANS CETTE ETUDE	21
TABLEAU 3:VISUALISATION MULTI-NIVEAUX DES CARTES CAM (LARGE, MEDIUM, SMALL) ET DETECTION FINALE ASSOCIEES A L'IMAGE ID_1972CD66(MAIZE_STREAK).	97
TABLEAU 4:VISUALISATION MULTI-NIVEAUX DES CARTES CAM (LARGE, MEDIUM, SMALL) ET DETECTION FINALE ASSOCIEES A L'IMAGE ID_9695CFC8(MAIZE STREAK).	98
TABLEAU 5:VISUALISATION MULTI-NIVEAUX DES CARTES CAM (LARGE, MEDIUM, SMALL) ET DETECTION FINALE ASSOCIEES A L'IMAGE ID_20672EC7(FALL_ARMYWORM)	99

Résumé

Notre étude vise à concevoir un système intelligent dont le but principal est **de détecter précocement** les maladies et ravageurs du maïs en localisant précisément les zones touchées sur les plantes. Grâce à des techniques de **détection d'objet**, le système sera capable d'identifier non seulement la présence d'un **problème (maladie ou ravageur)**, mais aussi de montrer sur **l'image** ou en **temps réel** dans **un flux vidéo**, les **parties exactes de la plante affectées**. Le **système intelligent** proposé constitue une solution automatisée, rapide et accessible, capable de détecter précocement certaines maladies et ravageurs, ainsi que des affections émergentes. Le modèle développé repose sur **l'intelligence artificielle et la vision par ordinateur**. Il a été spécialement conçu pour fonctionner en temps réel, même sur des **dispositifs mobiles ou embarqués à faibles ressources** (tels que des drones, robots ou capteurs intelligents), y compris en l'absence de connexion Internet. À plus grande échelle, ce système peut être intégré dans des drones ou des robots agricoles autonomes, capables de surveiller l'état sanitaire des cultures sur de vastes surfaces sans intervention humaine directe. Ces dispositifs intelligents peuvent ainsi effectuer une **surveillance continue** et proactive des champs, et même déclencher des **interventions ciblées** (comme l'application localisée de traitements), évitant ainsi l'usage excessif de produits chimiques et contribuant à une agriculture plus durable et respectueuse de **l'environnement**. Pour aider les utilisateurs à comprendre les décisions prises par le système, des techniques **d'explicabilité** sont intégrées. Ces techniques fournissent des explications claires et visuelles sur la manière dont notre modèle effectue ses prédictions, garantissant ainsi la transparence et instaurant la confiance dans les solutions d'IA.

Le projet s'appuie sur l'ensemble de données **KaraAgro AI Maize**, qui contient **16 552 images annotées** couvrant des catégories, telles que la maladie de la striure du maïs (**maize streak disease**), la chenille légionnaire (**fall armyworm**), les plants de maïs sains (**healthy maize**),

Un choix stratégique s'est donc porté sur l'architecture **YOLO** (You Only Look Once), un modèle de détection d'objets reconnu pour sa rapidité et son efficacité. Initialement pré-entraîné sur le jeu de données **COCO**, il a été ensuite affiné (**fine-tuning**) à l'aide du jeu de données annotées sur les plants de maïs.

Mots Clés : Intelligence Artificielle, Vision par Ordinateur, Deep Learning, Diagnostic Précoce, Maladies, Ravageurs, YOLO, Détection d'Objets, Fine-Tuning.

Abstract

Our study aims to design an intelligent system whose main objective is the **early detection** of **maize diseases** and **pests** by precisely **localizing** the affected areas on the plants. Using object detection techniques, the system will be able to identify not only the presence of a problem (**disease** or **pest**), but also to highlight, on the image or in real time through a video stream, the exact parts of the plant that are affected. The proposed intelligent system provides an **automated**, **fast**, and **accessible** solution capable of detecting certain diseases and pests at an early stage, as well as **emerging threats**. The developed model relies on **artificial intelligence** and **computer vision**. It is specifically designed to operate in **real time**, even on **low-resource** mobile or embedded devices (such as drones, robots, or smart sensors), including in the absence of an Internet connection. At a larger scale, this system can be integrated into autonomous drones or agricultural robots capable of monitoring the health status of crops over large areas without direct human intervention. These **intelligent devices** can therefore perform **continuous** and **proactive** field surveillance, and even trigger **targeted interventions** (such as localized treatment applications), thus avoiding the excessive use of chemicals and contributing to more **sustainable** and **environmentally friendly** agriculture. To help users understand the system's decisions, **explainability** techniques are integrated. These techniques provide clear and visual explanations of how our model makes its predictions, thereby ensuring **transparency** and building **trust** in AI-based solutions.

The project is based on the **KaraAgro AI Maize** dataset, which contains **16,552 annotated images** covering categories such as **maize streak disease**, **fall armyworm**, and **healthy maize** plants.

A strategic decision was made to use the **YOLO (You Only Look Once)** architecture, an object detection model known for its **speed** and **efficiency**. Initially **pre-trained** on the **COCO** dataset, it was then **fine-tuned** using the annotated maize plant dataset.

Keywords: Artificial Intelligence, Computer Vision, Deep Learning, Early Diagnosis, Diseases, Pests, YOLO, Object Detection, Fine-Tuning.

Sommaire

ENGAGEMENT	I
DEDICACES.....	II
REMERCIEMENTS	III
SIGLES ACRONYMES ET ABREVIATIONS	IV
LISTE DES FIGURES	VI
LISTE DES TABLEAUX	X
RESUME.....	XI
ABSTRACT	XIII
SOMMAIRE	XIV
INTRODUCTION.....	2
CHAPITRE 1 : REVUE DE LITTERATURE	6
SECTION 1 : DEFINITIONS DES THEMES ET CONCEPTS	6
SECTION 2 : VUE GLOBALE SUR LES PLANTES DE MAIS.....	19
SECTION 3 : ÉTUDE ET CRITIQUE DE L'EXISTANT	22
CHAPITRE 2 : ARCHITECTURE MATERIELLE, LOGICIELLE ET METHODOLOGIE DE MISE EN ŒUVRE	26
SECTION	1 : ARCHITECTURE MATERIELLE
26	
SECTION 2 : ARCHITECTURE LOGICIELLE.....	28
SECTION 3 : ARCHITECTURE LOGIQUE DE LA SOLUTION	41
CHAPITRE 3 : RESULTATS ET DISCUSSIONS.....	70

SECTION ... 1 : RESULTATS SUR LE PIPELINE D'ENTRAINEMENT AVEC EVALUATION CONTINUE SUR LE JEU DE VALIDATION.....	70
SECTION 2 : ÉVALUATION DES PERFORMANCES SUR LE JEU DE TEST	82
SECTION 3 RESULTATS D'INFERENCE SUR DES IMAGES DE TEST	90
SECTION 4. INTERPRETABILITE DU MODELE : VISUALISATION DES FACTEURS D'EXPLICABILITE	96
SECTION 5 : RESULTATS DE L'EXPORTATION DU MODELE POUR PERIPHERIQUES EMBARQUES.....	99
SECTION 6 : DEPLOIEMENT DU MODELE VIA UNE APPLICATION INTERACTIVE ET AUTONOME	100
SECTION 7 : DISCUSSIONS ET PERSPECTIVES.....	102
CONCLUSION	104
REFERENCES WEBOGRAPHIQUES & BIBLIOGRAPHIQUES	XII
TABLE DES MATIÈRES	15

Introduction

Introduction

1. Problématique

Le **maïs** , culture essentielle pour la sécurité alimentaire et l'économie agricole, notamment en **Afrique**, est régulièrement menacé par des **maladies** et des **ravageurs dévastateurs**, tels que la maladie de la striure du maïs et la chenille légionnaire. Ces affections, difficiles à distinguer à l'œil nu, entraînent des pertes de rendement massives et poussent souvent les agriculteurs à une utilisation excessive de pesticides, faute de diagnostic précoce et précis. Les méthodes actuelles de détection reposent principalement sur l'observation visuelle par des experts, une approche longue, coûteuse et peu accessible, surtout dans les zones rurales isolées où l'accès aux services techniques est limité. Cette dépendance à des inspections manuelles retarde les interventions, aggrave les dégâts et compromet les récoltes. Face à ces défis, il est crucial de développer un **système intelligent**, capable d'identifier automatiquement et en temps réel les symptômes des maladies et la présence de ravageurs, directement depuis **des périphériques embarqués**, sans nécessiter de **connexion internet**. Ce système pourrait également être déployé à grande échelle en l'intégrant à des drones agricoles et des pulvérisateurs intelligents .

2. Contexte et motivations

Bien que le secteur **agricole** soit une priorité nationale de développement **économique** en **Afrique subsaharienne**, les ravageurs et les maladies des cultures constituent un défi majeur pour les principales cultures essentielles à la sécurité alimentaire, comme le **maïs**.

La **chenille légionnaire touche 44 pays d'Afrique**. Lors de nouvelles enquêtes auprès des ménages au **Ghana** et en **Zambie**, **98 %** des agriculteurs ont déclaré que le maïs était touché. La perte moyenne de maïs signalée par les agriculteurs au **Ghana** était de **26,6 %** et en **Zambie de 35 %**. L'extrapolation de ces pertes à l'échelle nationale donne une estimation de la perte de valeur de la récolte **annuelle de maïs à 177 millions de dollars américains** au

Ghana et à **159 millions de dollars américains en Zambie**. Selon les recherches, la maladie de la striure du maïs, causée par **le virus de la striure du maïs**, est considérée comme la **troisième maladie la plus grave affectant le maïs en Afrique subsaharienne**. L'importance de ces maladies a considérablement affecté les rendements de la culture vivrière la plus importante d'Afrique. L'état actuel de la collecte de données et du diagnostic des ravageurs et des maladies des cultures évolue de l'identification des maladies à l'aide de symptômes visibles à l'utilisation de solutions basées sur les données, appliquant des techniques **d'apprentissage automatique et de vision par ordinateur**. [1]

Dans ce contexte, la **Stratégie Nationale d'Intelligence Artificielle et de Mégadonnées du Bénin 2023-2027**, définie par le **Ministère du Numérique et de la Digitalisation**, prend tout son sens. Elle prévoit, parmi ses **axes d'intervention prioritaires**, **la détection des maladies affectant les productions agricoles**, reconnue comme une **initiative à fort impact sur les métiers agricoles**. Ce positionnement stratégique reflète une volonté claire de mobiliser les technologies avancées au service de la sécurité alimentaire et du développement durable, et d'affirmer le rôle du Bénin comme leader de l'innovation numérique en Afrique de l'Ouest. [2]

3. Objectifs de l'étude

L'objectif principal de cette étude est de concevoir un **système intelligent, autonome et accessible, capable de détecter, localiser et diagnostiquer en temps réel les maladies et ravageurs affectant les plants de maïs à partir d'images ou de flux vidéo**. S'inscrivant dans une dynamique d'agriculture de précision et durable, ce système vise à répondre aux limites des **méthodes traditionnelles d'inspection visuelle, souvent lentes, coûteuses et inaccessibles dans les zones rurales**. Grâce aux avancées récentes **en intelligence artificielle, en vision par ordinateur et en deep learning**, il s'appuie sur une architecture performante de type **YOLO (You Only Look Once)**, spécialement affiné à partir du jeu de données annotées **KaraAgro AI Maize**, couvrant plusieurs catégories de symptômes et états **phytosanitaires du maïs**.

L'un des objectifs spécifiques est d'assurer un fonctionnement en temps réel même sur **des dispositifs à faibles ressources et sans connexion internet**, permettant même aux agriculteurs des zones rural sans connexion d'obtenir instantanément une analyse visuelle

précise des zones affectées. Pour favoriser la **transparence** et la **confiance**, des techniques d'explicabilité seront également intégrées afin de fournir des **explications visuelles** et compréhensibles sur le raisonnement du modèle.

In fine, ce système a pour vocation de limiter les pertes agricoles, de réduire l'utilisation excessive et anarchique de pesticides, de favoriser des interventions ciblées et de contribuer à la modernisation de l'agriculture africaine, en **rendant les technologies d'intelligence artificielle accessibles même dans les zones reculées**, conformément aux orientations stratégiques en matière d'innovation numérique et de sécurité alimentaire.

Chapitre 1 : Revue de Littérature

Chapitre 1 : Revue de Littérature

Section 1 : Définitions des Thèmes et Concepts

1.1 Intelligence Artificielle

L'**intelligence artificielle (IA)** désigne un ensemble de théories, de méthodes et de technologies visant à permettre à des machines, notamment les systèmes informatiques, d'imiter certaines capacités cognitives humaines telles que l'apprentissage, le raisonnement, la perception visuelle, la compréhension du langage naturel et la prise de décision. Créée par **John McCarthy en 1956**, l'**IA** vise à construire des machines capables de raisonner, d'apprendre et d'agir de manière autonome.

Les applications et les appareils assistés par l'**IA** peuvent voir et identifier les objets. Ils sont capables de comprendre le langage humain et d'y répondre. Ils peuvent tirer des enseignements de nouvelles informations et de nouvelles expériences. Ils peuvent formuler des recommandations détaillées aux utilisateurs et aux experts. Ils peuvent agir indépendamment, remplaçant ainsi l'intelligence ou l'intervention humaine (un exemple classique étant la voiture autonome). [3]

1.1.1 Concepts fondamentaux et sous-domaines

L'IA est un terme générique qui englobe plusieurs sous-domaines clés, chacun contribuant au développement de systèmes intelligents :

A. **Machine Learning (ML): “Machine Learning is the field of study that gives computers the ability to learn without being explicitly programmed.” (Samuel, 1959)**

Ainsi Arthur Samuel (1959) définit le Machine Learning comme « le domaine d'étude qui donne aux ordinateurs la capacité d'apprendre sans être explicitement programmés ». Pour donner à un ordinateur la capacité d'apprendre, on utilise des méthodes d'apprentissage qui sont fortement inspirées de la façon dont nous, les êtres humains, apprenons à faire des choses. Parmi ces méthodes, on compte :

✚ L'apprentissage supervisé (Supervised Learning) :

L'apprentissage supervisé consiste à entraîner un modèle à partir d'un jeu de données étiqueté, c'est-à-dire pour lequel les résultats attendus sont connus. Le modèle apprend à associer des entrées (**features**) à des sorties (**targets**).

Exemple : prédire si un e-mail est un spam ou non à partir de ses caractéristiques.

✚ L'apprentissage non supervisé (Unsupervised Learning) :

Dans l'apprentissage non supervisé, les données ne sont pas étiquetées. L'objectif est de trouver des structures cachées, des patterns ou des regroupements au sein des données.

Exemple : regrouper automatiquement des clients selon leurs comportements d'achat.

✚ L'apprentissage par renforcement (Reinforcement Learning) :

L'apprentissage par renforcement est basé sur un système d'essais-erreurs. Un agent apprend à interagir avec un environnement pour maximiser une récompense.

Exemple : un robot qui apprend à jouer à un jeu vidéo en recevant ou non des points selon ses actions. [4]

Dans cette étude, nous travaillons avec un **ensemble de données d'images annotées**, où chaque image peut contenir plusieurs annotations correspondant à différentes classes. Ces annotations fournissent un jeu de données étiqueté, avec des informations précises sur la localisation et la catégorie de chaque instance présente. Ainsi, notre approche s'appuie sur l'apprentissage supervisé, qui consiste à entraîner un modèle à partir de ces données étiquetées. Le modèle apprend à associer les caractéristiques visuelles aux différentes classes présentes dans chaque image, lui permettant de détecter et classifier plusieurs instances dans une même image. **L'apprentissage supervisé** est particulièrement adapté ici, car il exploite pleinement ces annotations détaillées pour guider le modèle dans sa phase **d'entraînement**.

B. Deep Learning (DL): Un type spécialisé de ML qui utilise des réseaux neuronaux artificiels avec plusieurs couches (architectures profondes) pour modéliser et comprendre des modèles complexes dans de grands ensembles de données. Le DL est à l'origine d'avancées significatives, notamment dans les réseaux neuronaux convolutifs (CNN) pour les tâches visuelles. Des organismes de recherche de premier plan tels que DeepMind et OpenAI utilisent largement la DL.

- **Réseaux de Neurones (Neural Networks) :**

Les réseaux de neurones sont des modèles de machine learning conçus pour imiter le fonctionnement du cerveau humain. Ils sont composés de couches de nœuds interconnectés, appelés neurones artificiels, qui coopèrent pour analyser des données, reconnaître des motifs, évaluer des options et prendre des décisions. En reproduisant la manière dont les neurones biologiques traitent l'information, les réseaux de neurones permettent d'identifier des phénomènes complexes et d'arriver à des conclusions de manière autonome. [4] Ils se composent généralement de trois types de couches :

- **Couche d'entrée (Input Layer) :**

Elle reçoit les données brutes, comme les **pixels** d'une **image**, les valeurs numériques ou les mots d'un texte.

- **Couches cachées (Hidden Layers) :**

Ces couches intermédiaires réalisent des opérations mathématiques complexes pour extraire des caractéristiques pertinentes (**features**). Plus il y a de couches cachées, plus le réseau est capable de modéliser des relations non linéaires complexes dans les données.

- **Couche de sortie (Output Layer) :** Elle fournit le résultat final du modèle, tel qu'une prédiction, une probabilité, ou une classification (ex. : "chat" ou "chien").

Un réseau comportant plusieurs couches cachées est appelé un **réseau profond (deep neural network)**, d'où le terme **Deep Learning**. Ces réseaux sont à la base des avancées majeures en **vision par ordinateur**, **traitement du langage naturel**, et d'autres domaines de l'IA. [6]

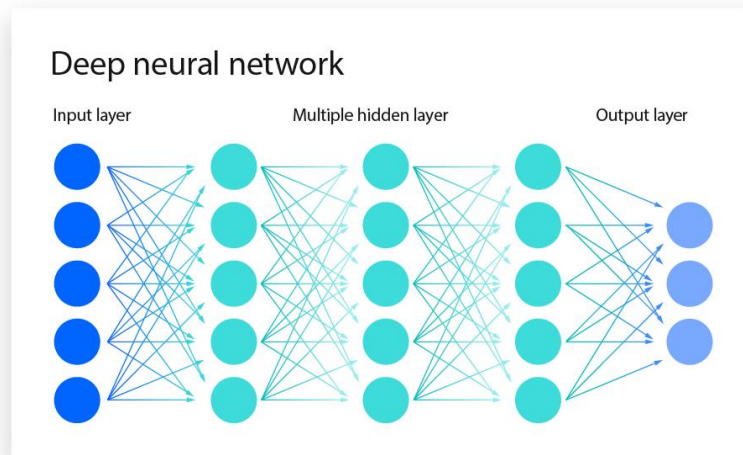


Figure 1: Architecture d'un réseau de neurones profond (Deep Neural Network) [6]

Les termes « **apprentissage profond** » et « **réseaux de neurones** » sont souvent utilisés de manière interchangeable, ce qui peut prêter à confusion. Par conséquent, il convient de noter que le terme « **profond** » dans « **apprentissage profond** » désigne simplement la profondeur des couches d'un réseau de neurones. Un réseau de neurones composé de plus de trois couches, incluant les entrées et les sorties, peut être considéré comme un algorithme d'apprentissage profond. Un réseau de neurones ne comportant que deux ou trois couches n'est qu'un réseau de neurones basique. [6]

C. **Vision par ordinateur (CV):** Ce domaine permet aux machines d'interpréter et de comprendre les informations visuelles des images ou des vidéos. Les tâches comprennent la reconnaissance d'images, la détection d'objets et la segmentation d'images. [7]

1.1.2 Considérations éthiques

Le développement et le déploiement rapides de l'**IA** soulèvent d'importantes questions éthiques. Les principaux sujets de préoccupation comprennent les biais algorithmiques potentiels menant à des résultats injustes, la garantie de la confidentialité et de la sécurité des données, le maintien de la transparence dans la prise de décision en matière d'**IA** (**XAI**) et l'établissement d'une responsabilité pour les actions de l'**IA**. Ces considérations éthiques sont

prises en compte tout au long de cette étude, notamment en ce qui concerne la gestion responsable des biais algorithmiques, la transparence des décisions, la sécurité des données et le respect des droits d'utilisation des données, afin d'assurer un usage équitable, légal et durable de l'intelligence artificielle dans le contexte agricole.

1.2 L'agriculture de précision

L'agriculture de précision est une approche moderne de la gestion agricole qui repose sur l'utilisation des technologies avancées pour optimiser les rendements agricoles, réduire les coûts et minimiser l'impact environnemental. Elle consiste à utiliser des outils technologiques (capteurs, GPS, drones, intelligence artificielle, etc.) pour observer, mesurer et gérer les cultures et les ressources agricoles de manière ciblée et efficace. **Son objectif principal est d'appliquer la bonne action, au bon endroit, au bon moment, dans la et bonne quantité .**

Au sein de cette étude **l'agriculture de précision** est mise en œuvre à travers un système intelligent basé sur la vision par ordinateur, capable de détecter précocement et localiser en temps réel les maladies et ravageurs du maïs, afin d'aider à une intervention rapide, ciblée et efficace, même en l'absence de connexion internet ou sur des dispositifs à faibles ressources. Le système est également conçu pour être compatible avec diverses plateformes, notamment les smartphones, les caméras mobiles et les drones agricoles, afin de permettre une surveillance automatisée à grande échelle, une détection précoce et une application ciblée des traitements, contribuant ainsi à une gestion durable des cultures.

1.3 Détection d'objets avec les Réseaux de Neurones Convolutifs (CNN)

La **détection d'objets** est une tâche fondamentale de la **vision par ordinateur (CV)** qui consiste à identifier la **présence**, **l'emplacement** et le **type** d'un ou de plusieurs **objets** dans une image ou une vidéo.

Contrairement à la classification des images, qui attribue une **étiquette unique** à l'ensemble de l'image (par exemple, "chenille"), la **détection d'objets** délimite précisément **chaque instance d'objet** à l'aide d'une **boîte englobante** et lui attribue une étiquette de classe (par exemple, "chenille" aux coordonnées x, y, largeur, hauteur). Cette capacité permet aux

machines de comprendre les scènes visuelles avec une plus grande **granularité**, imitant de plus près la **perception visuelle humaine** et permettant des interactions plus complexes avec l'environnement.

La détection d'objets combine généralement deux tâches essentielles : la **classification des objets** (déterminer "quel" objet est présent) et la **localisation des objets** (déterminer "où" l'objet est situé, généralement via les coordonnées de la boîte englobante). Les systèmes modernes de détection d'objets s'appuient fortement sur l'apprentissage profond (**DL**), en particulier les réseaux neuronaux convolutifs (**CNN**). Ces réseaux sont formés sur de grands ensembles de données annotés, tels que le populaire ensemble de données **COCO** ou Open Images V7, afin d'apprendre les caractéristiques visuelles et les motifs associés aux différentes classes d'objets.

Pendant le fonctionnement (appelé **inférence**), le modèle formé traite une image ou une trame vidéo d'entrée. Il produit une liste d'objets potentiels, chacun représenté par une boîte englobante, une étiquette de classe prédite (par exemple, "larves", "oeufs") et un score de confiance indiquant la certitude du modèle quant à la détection. Des techniques telles que la suppression non maximale (**NMS**) sont souvent utilisées pour affiner ces résultats en supprimant les boîtes redondantes qui se chevauchent pour le même objet. Les performances de ces modèles sont généralement évaluées à l'aide de mesures telles que l'intersection sur l'union (**IoU**) et la précision moyenne (**mAP**). [8]

Types de modèles de détection d'objets

Les modèles de détection d'objets se répartissent généralement en deux grandes catégories, qui diffèrent principalement par leur approche et les compromis vitesse/précision :

Détecteurs d'objets à deux niveaux : Ces modèles proposent d'abord des régions d'intérêt où des objets pourraient se trouver, puis classent les objets dans ces régions. La famille R-CNN (Fast R-CNN, Faster R-CNN) en est un exemple. Ils atteignent souvent une grande précision mais ont tendance à être plus lents.

Détecteurs d'objets à un étage : Ces modèles prédisent directement les boîtes de délimitation

et les probabilités de classe à partir de l'image d'entrée en un seul passage, sans étape de proposition de région séparée. Parmi les exemples, on peut citer le modèle **Ultralytics YOLO (You Only Look Once)**, **SSD (Single Shot MultiBox Detector)** et **RetinaNet**. Elles sont généralement plus rapides, ce qui les rend adaptées à l'inférence en temps réel, parfois au prix d'une précision légèrement inférieure à celle des méthodes en deux étapes, bien que des modèles comme **YOLO11** comblent efficacement cet écart.

Des approches encore plus récentes, comme les **détecteurs sans ancrage (anchor-free)**, simplifient davantage le processus en supprimant l'utilisation de boîtes d'ancrage prédéfinies. Un exemple de ce type est RT-DETR (Real-Time Detection Transformer). Cependant, bien qu'il soit conçu pour une détection rapide et efficace, RT-DETR n'est pas encore tout à fait adapté pour certains cas d'usage exigeants en temps réel sur des dispositifs embarqués, en raison de sa complexité computationnelle. [8]

1.4 Les Réseaux de neurones convolutifs :

Les Réseaux de neurones convolutifs **CNN (Convolutional Neural Networks)** sont des modèles très efficaces pour les tâches de vision par ordinateur, car ils extraient automatiquement des caractéristiques visuelles à différents niveaux (formes, textures, objets).

Cette efficacité repose sur leur architecture particulière : un réseau neuronal convolutif (**CNN**) est un type spécialisé de réseau neuronal (**NN**) particulièrement efficace pour traiter les données en forme de grille, telles que les images et les vidéos. Contrairement aux réseaux neuronaux traditionnels qui traitent les entrées comme des vecteurs plats, les **CNN** sont conçus pour apprendre automatiquement et de manière adaptative des hiérarchies spatiales de caractéristiques directement à partir des données d'entrée. Cela se fait principalement par l'application de l'opération de convolution, ce qui en fait une pierre angulaire de la vision par ordinateur moderne et entraîne des avancées significatives dans le domaine de l'intelligence artificielle (**IA**). Leur capacité à capturer les dépendances locales et les relations spatiales les rend particulièrement adaptées aux tâches où la disposition des pixels est importante.

Les réseaux neuronaux convolutifs (**CNN**) se distinguent des autres réseaux neuronaux par leurs performances supérieures avec des entrées de signaux d'image, de parole ou audio. Ils présentent trois principaux types de couches :

Couche de convolution , Couche de pooling , Couche entièrement connectée (FC) [9]

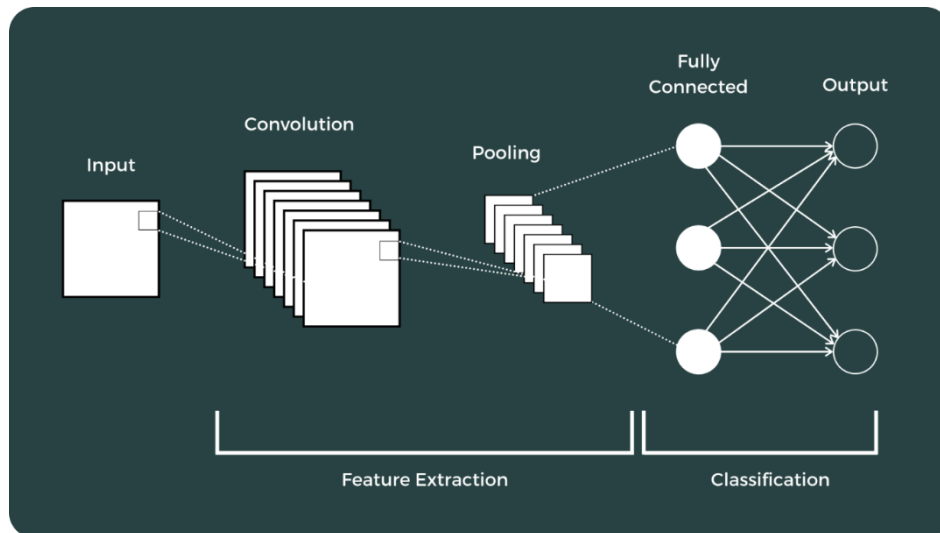


Figure 2: Architecture typique d'un réseau de neurones convolutifs (CNN) comprenant les couches de convolution, de pooling et entièrement connectée

Couche de convolution

La couche convolutive est le bloc fonctionnel principal d'un CNN, c'est là que se produit la majorité des calculs. Elle nécessite quelques composants, à savoir des **données d'entrée, un filtre et une carte de caractéristiques (ou « feature map »)**. Supposons que l'entrée soit une image couleur, composée d'une matrice de pixels en 3D. Cette entrée aura alors trois dimensions (hauteur, largeur et profondeur) qui correspondent à la combinaison RVB dans une image. Nous disposons également d'un détecteur de caractéristiques, également connu sous le nom **de noyau ou de filtre**, qui se déplacera sur les champs réceptifs de l'image pour vérifier si la caractéristique est présente. **Ce processus est appelé convolution.**

Une autre couche de convolution peut suivre la couche de convolution initiale. Dans ce cas, la structure du **CNN** peut devenir hiérarchique, car les couches suivantes peuvent voir les pixels dans les champs réceptifs des couches précédentes. Supposons par exemple que nous cherchions à déterminer si une image contient un vélo. Nous pouvons considérer le vélo comme un ensemble de parties : il est composé d'un cadre, d'un guidon, de roues, de pédales, etc. Chaque partie du vélo constitue un schéma de niveau inférieur dans le réseau neuronal, et la combinaison de ses parties représente un schéma de niveau supérieur, créant ainsi une hiérarchie de caractéristiques au sein du **CNN**. Enfin, la couche convolutionnelle convertit l'image en valeurs numériques, ce qui permet au réseau neuronal d'interpréter et d'extraire

des modèles pertinents. [9]



Figure 3: Représentation hiérarchique des caractéristiques apprises par les couches convolutionnelles d'un CNN lors de la détection d'un objet (exemple : un vélo)

🚦 Couche de pooling

Les couches de **pooling**, également appelée couches de sous-échantillonnage, permettent de réduire la dimensionnalité, c'est-à-dire de réduire le nombre de paramètres d'entrée. Comme pour la couche de convolution, l'opération de **pooling** balaie un filtre sur toute l'entrée, à la différence que ce filtre n'a aucun poids. Au lieu de cela, le **noyau** applique une **fonction d'agrégation** aux valeurs du champ **réceptif**, remplissant ainsi la matrice de sortie. Il existe deux principaux types de **pooling** :

Max pooling : à mesure que le filtre se déplace sur l'entrée, il sélectionne le pixel ayant la valeur maximale pour l'envoyer à la matrice de sortie. Soit dit en passant, cette approche a tendance à être plus souvent utilisée que l'**average pooling**.

Average pooling : à mesure que le filtre se déplace sur l'entrée, il calcule la valeur moyenne dans le champ réceptif pour l'envoyer à la matrice de sortie. Bien que de nombreuses informations soient perdues dans la couche de pooling, celle-ci présente malgré tout un certain nombre d'avantages pour le **CNN**. Elle permet de réduire la complexité, d'améliorer l'efficacité et de limiter le risque de surajustement.

🚦 Couche entièrement connectée (FC)

La couche **entièrement connectée** porte bien son nom. Comme mentionné précédemment, les valeurs de pixel de l'image d'entrée ne sont pas directement connectées à la

couche de sortie dans les couches partiellement connectées. Cependant, dans la couche entièrement connectée, chaque nœud de la couche de sortie se connecte directement à un nœud de la couche précédente. Cette couche effectue la classification en fonction des caractéristiques extraites à partir des couches précédentes et de leurs différents filtres. Alors que les couches de convolution et de **pooling** ont tendance à utiliser les fonctions **ReLU**, les couches FC exploitent généralement une fonction d'activation **softmax** pour classer les entrées de manière appropriée, produisant une probabilité de 0 à 1.

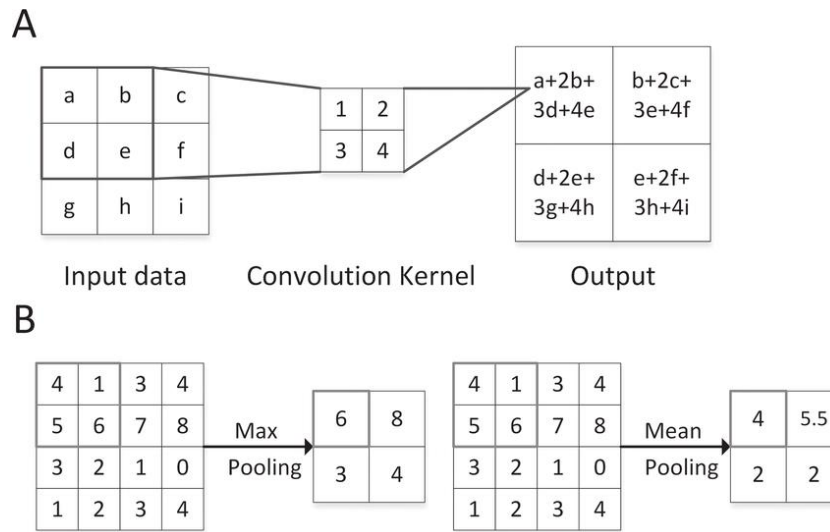


Figure 4: Illustration du processus de convolution entre une image et un filtre dans une couche convolutive

1.5 Définition théorique d'un filtre (ou noyau) et de la convolution en vision par ordinateur

Un **filtre** (ou **noyau**) en **vision par ordinateur** est une petite matrice de poids appliquée sur une image par opération mathématique appelée **convolution** [10]. Ce processus extrait des caractéristiques locales de l'image en combinant les pixels voisins selon les poids du filtre. Formellement, la convolution d'une image I par un filtre K se calcule ainsi sur chaque position (x, y) [10] :

$$S(x, y) = \sum_i \sum_j I(x + i, y + j) \times K(i, j)$$

$$S(x, y) = I(x+i_1, y+j_1) * K(i_1, j_1) + I(x+i_2, y+j_2) * K(i_2, j_2) + \dots$$

où :

- $I(x + i, y + j)$: valeur du pixel de l'image autour de la position (x, y)
- $K(i, j)$: poids du filtre
- $S(x, y)$: résultat de la convolution à la position (x, y)
(aussi appelée "**feature map**" ou **carte de caractéristiques**)

Cette opération est au cœur de la détection des bords, motifs ou textures dans les réseaux CNN comme **YOLO**.

1.6 Image numérique

Une image est une **grille** (matrice) composée de petits carrés appelés **pixels**. Chaque pixel contient une ou plusieurs valeurs qui représentent sa couleur ou son intensité.



Figure 5: Image en couleur d'une feuille de maïs

1.7 Pixel

Un pixel (contraction de "**picture element**") est la **plus petite unité** d'une image numérique. [11] Il contient une valeur dans une image en niveaux de gris : un seul nombre (**entre 0 = noir et 255 = blanc**) et dans une image couleur (**RGB**) : trois nombres correspondant aux **canaux Rouge (R), Vert (G), Bleu (B)**. Les canaux représentent les composantes de couleur d'une image :

✚ **Canal Rouge (R) (255, 0, 0) : intensité de la couleur rouge.**

✚ **Canal Vert (G) (0, 255, 0) : intensité de la couleur verte.**

✚ **Canal Bleu (B) (0, 0, 255) : intensité de la couleur bleue.**

Une image couleur est donc une superposition de trois matrices (R, G, B), formant une image 3D.

1.8 Apprentissage par transfert (Transfert Learning) avec les réseaux de neurones convolutifs

L'agriculture de précision s'appuie de plus en plus sur l'intelligence artificielle pour automatiser la détection des maladies et ravageurs qui affectent les cultures .

Dans ce projet, nous utilisons un modèle de vision par ordinateur (**YOLO**) pour détecter visuellement les symptômes sur les feuilles, fruits ou tiges. Cependant, un modèle YOLO pré-entraîné ne connaît pas les maladies végétales par défaut. Il est donc nécessaire d'adapter ce modèle à notre problématique spécifique. C'est ici que l'apprentissage par transfert (**transfer learning**) et le **fine-tuning** interviennent.

1.8.1 Transfer Learning – Réutiliser l'intelligence déjà acquise.

L'**apprentissage par transfert** consiste à utiliser un modèle pré-entraîné sur un grand dataset générique (par exemple **COCO (Common Objects in Context)** ou ImageNet) comme point de départ pour résoudre une nouvelle tâche, souvent plus spécifique.

Le modèle **YOLO** que nous utilisons a été pré-entraîné sur le dataset **COCO**, contenant plus de 80 classes générales (personnes, voitures, animaux, etc.). Même si ces classes n'ont rien à voir avec les plantes, ce modèle a appris à extraire des **caractéristiques visuelles** bas niveau (bords, textures, couleurs), repérer des formes et structures visuelles complexes. Plutôt que d'entraîner un nouveau modèle de zéro (ce qui prend beaucoup de ressources et de données), nous réutilisons ces couches convolutives déjà entraînées comme base. Cela nous permet de bénéficier d'une base de connaissance visuelle robuste, applicable à nos images de plantes.

1.8.2 Fine-Tuning – Adapter le modèle à notre tâche spécifique :

Le **fine-tuning** consiste à réentraîner partiellement ou totalement un modèle pré-entraîné sur notre propre **dataset** spécifique, afin qu'il soit capable de détecter des objets/classes qui ne faisaient pas partie de son apprentissage initial.

Nous avons constitué un **dataset** d'images de plantes (maïs), annotées avec des zones

malades (tâches , motifs), des **ravageurs** (larves), des parties **saines** comme classe de contrôle. Nous avons ensuite : modifié la **tête** de **YOLO** pour remplacer les anciennes classes par nos propres classes (**healthy, fall-armyworm, maize-streak-diseases.**).

Élément	Transfer Learning	Fine-Tuning
Objectif	Réutiliser les connaissances acquise	Adapter le modèle à une tâche spécifique
Couches concernées	Couches convolutives de base	Couches finales (ou tout le modèle)
Avantage principal	Gain de temps et d'efficacité de calcul	Haute précision sur un contexte ciblé
Exemple dans le projet	Utilisation des poids de YOLO11 pré-entraîné	On réentraîne YOLO11 sur des images de maïs malade

Tableau 1: Complémentarité et différences entre Transfer Learning et Fine-Tuning

Dans le cadre de notre projet de détection des maladies et ravageurs dans les plantes, l'usage combiné de l'apprentissage par transfert et du fine-tuning est essentiel pour adapter un modèle d'intelligence artificielle à notre domaine d'application agricole.

Cette approche permet de réduire les ressources nécessaires à l'entraînement, améliorer la performance du modèle sur des données spécialisées, offrir un outil pratique pour une agriculture plus intelligente, rapide et durable.

Section 2 : Vue globale sur les plantes de Maïs

2.1 Définition

Le maïs (**Zea mays**) est l'une des cultures vivrières les plus importantes à l'échelle **mondiale**, en particulier en **Afrique subsaharienne**. Source essentielle de glucides, il est aussi une base économique pour de nombreuses familles rurales. Cependant, sa production est fortement menacée par diverses maladies et ravageurs, qui peuvent entraîner des pertes de rendement considérables si aucune mesure de gestion appropriée n'est prise.

2.2 Maladie des Ravageurs des Plantes de Maïs : causes, symptômes, mesures préventives

La **chenille légionnaire touche 44 pays d'Afrique**. Lors de nouvelles enquêtes auprès des ménages au **Ghana** et en **Zambie**, **98 %** des agriculteurs ont déclaré que le maïs était touché. La perte moyenne de maïs signalée par les agriculteurs au **Ghana** était de **26,6 %** et en **Zambie** de **35 %**. L'extrapolation de ces pertes à l'échelle nationale donne une estimation de la perte de valeur de la récolte **annuelle de maïs à 177 millions de dollars américains** au Ghana et à **159 millions de dollars américains en Zambie**. Selon les recherches, la maladie de la striure du maïs, causée par **le virus de la striure du maïs**, est considérée comme la **troisième maladie la plus grave affectant le maïs en Afrique subsaharienne**.

Le terme "**fall armyworm**" vient de l'anglais et désigne un insecte nuisible extrêmement destructeur pour les cultures, notamment le maïs. Son nom reflète son comportement : les chenilles, appelées "**armyworms**", se déplacent en grand nombre comme une armée, ravageant les champs en consommant les feuilles des plantes. Le mot "**fall**" fait référence à leur forte activité en automne dans certaines régions des États-Unis. Leur nom scientifique est **Spodoptera frugiperda** ; il s'agit de chenilles qui se transforment en papillons nocturnes à l'âge adulte. Originaires des Amériques, ces ravageurs se sont récemment propagés en Afrique, en Asie, puis en Australie. Ce parasite s'attaque à **plus de 80 types de cultures**, mais il cible principalement le maïs, en perforant les feuilles et les épis, ce qui en fait un fléau redouté et difficile à éradiquer une fois établi. **[1]**

Infos /Maladies	Maize Streak Disease (MSD)	Fall Armyworm (Spodoptera frugiperda)
Cause	Virus MSV transmis par les cicadelles (Cicadulina)	Insecte lépidoptère invasif, stade larvaire fortement ravageur
Formes visibles/ stades	Stries jaunes	Œufs groupés Excréments (frass) Dégâts (trous foliaires) Larves dans feuilles roulées
Symptômes	Stries jaunes linéaires Nanisme Déformation des feuilles Épis peu développés	Trous irréguliers Présence d' excréments Larves visibles Œufs sous feuilles
Période critique	Semis à 6 feuilles	3 feuilles à formation des épis
Mesures préventives	Contrôle des cicadelles Semis précoces	Surveillance régulière Pesticides chimiques
Rendement affecté	Jusqu'à 100%	Jusqu'à 35% (voire + sans intervention)
Zones touchées	Afrique de l'Est, Australe	44 pays d'Afrique (Ouest, Centrale, Australe)
Image		

		
--	---	---

Tableau 2 : Principales menaces du maïs abordées dans cette étude

[1]

Section 3 : Étude et critique de l'existant

3.1 Publications scientifiques

3.1.1. CTR_YOLOv5n : Détection des maladies du maïs avec YOLOv5n et mécanisme d'attention

Méthodologie : Cette étude propose un modèle amélioré basé sur **YOLOv5n**, nommé **CTR_YOLOv5n**, intégrant un mécanisme d'attention coordonnée (Coordinate Attention) et une tête de détection **Swin Transformer**. L'objectif est d'accélérer la convergence du modèle, d'améliorer sa généralisation et d'augmenter la précision et la vitesse de détection.

Maladies ciblées : Trois maladies courantes des feuilles de maïs sont étudiées : **la tache foliaire (Leaf Spot)**, **la tache grise (Gray Leaf Spot)** et **la rouille (Common Rust)**. Le jeu de données comprend également des images de feuilles de maïs saines (**Healthy Corn Leaves**).

Résultats : Le modèle CTR_YOLOv5n a montré une amélioration significative en termes de précision et de vitesse de détection par rapport aux modèles YOLOv5n standard. Ce modèle, nommé CTR_YOLOv5n, a été conçu pour une détection en temps réel sur des appareils mobiles, mettant l'accent sur la légèreté et la rapidité.

Limites : Bien que performant, ce modèle ne couvre pas spécifiquement la maladie des stries du maïs ni les dégâts causés par la chenille légionnaire d'automne. De plus, l'intégration de mécanismes d'attention et de transformateurs peut augmenter la complexité computationnelle, ce qui pourrait limiter son déploiement sur des appareils mobiles à faibles ressources. En outre, le modèle ne prévoit **ni mécanisme d'explicabilité intégré** (comme les cartes d'attention visuelle pour aider à interpréter les décisions), **ni processus d'optimisation structurelle** (quantification, export tflite ,calibration) permettant de **faciliter son adaptation efficace aux environnements mobiles ou embarqués**.

Ma, L., Yu, Q., Yu, H., & Zhang, J. (2023). *Maize Leaf Disease Identification Based on YOLOv5n Algorithm Incorporating Attention Mechanism*. *Agronomy*, 13(2), Article 521.

Disponible sur [12] : <https://www.mdpi.com/2073-4395/13/2/521>

3.1.2 AMaizeD : Pipeline de détection automatique des maladies du maïs

Méthodologie : Le projet **AMaizeD** développe un pipeline complet utilisant des images multispectrales capturées par drones pour détecter des maladies du maïs. Un ensemble de données personnalisé a été collecté, couvrant diverses variétés de maïs, pratiques culturales et conditions environnementales.

Maladies ciblées : Le pipeline est conçu pour détecter plusieurs maladies, notamment l'oïdium, l'anthracnose et la brûlure des feuilles.

Résultats : Le cadre proposé a démontré une performance de pointe sur l'ensemble de données personnalisé, contribuant au domaine de la détection automatisée des maladies en agriculture. Plusieurs architectures de réseaux neuronaux ont été comparées dans l'étude, et les résultats de test ont montré des taux de précision très élevés : **GoogleNet** : 99,8 % de précision avec une perte de 0,0637, **Vision Transformer** : 99,23 % de précision avec une perte de 0,023, **EfficientNet** : 97,50 % avec une très faible perte de 0,0138, : 97,66 % avec une perte de 0,0961, **AlexNet** : 93,5 % avec une perte de 0,175 .

Limites : Le pipeline **AMaizeD** repose sur l'utilisation d'images multispectrales capturées par drones, ce qui entraîne un coût élevé lié à l'acquisition, à la maintenance des drones et à l'exploitation de capteurs spécialisés. Cette dépendance à un matériel technologique avancé limite fortement l'accessibilité du système, en particulier dans les zones rurales, où les agriculteurs ne disposent généralement pas de tels moyens.

Par ailleurs, la méthodologie proposée par **AMaizeD** se concentre uniquement sur la **classification de maladies foliaires spécifiques** (telles que l'oïdium, l'anthracnose et la brûlure des feuilles), **sans recourir à la détection d'objets**. Contrairement à notre approche, elle ne permet ni **l'identification précise**, ni la **localisation spatiale** des symptômes ou des ravageurs sur les feuilles de maïs.

Enfin, contrairement à notre approche qui cible une détection en temps réel, légère et fonctionnelle sur des appareils mobiles à faibles ressources et sans connexion Internet, **AMaizeD** n'est pas adaptée à un déploiement autonome sur le terrain, directement exploitable par les agriculteurs.

arXiv – AMAizeD: An End-to-End Pipeline for Automatic Maize Disease Detection, Anish Mall, Sanchit Kabra, Ankur Lhila, Pawan Ajmera. [13]

<https://arxiv.org/abs/2308.03766>

3.1.3 Modèle basé sur l'attention et les caractéristiques multi-échelles

Méthodologie : Cette recherche introduit un modèle combinant un mécanisme d'attention et des caractéristiques multi-échelles pour améliorer la détection des maladies et ravageurs du maïs. Le modèle utilise des modules d'attention pour se concentrer sur les zones pertinentes des images et des fusions de caractéristiques à différentes échelles pour capturer les détails variés des maladies.

Maladies ciblées : Le modèle est conçu pour détecter plusieurs maladies et ravageurs du maïs, bien que les maladies spécifiques ne soient pas détaillées dans le résumé.

Résultats : Le modèle a montré une meilleure performance que les approches traditionnelles, avec une précision, un rappel, une exactitude et un score F1 élevés.

Limites : Le modèle nécessite des ressources computationnelles importantes, ce qui limite son déploiement sur des appareils mobiles à faibles ressources. De plus, cette étude est basée sur la classification des images et non sur la détection d'objets, qui permettrait non seulement d'identifier la présence de maladies, mais aussi de localiser précisément les zones affectées. Par conséquent, ce modèle n'est pas adapté à une détection en temps réel sur le terrain, contrairement à notre approche qui fonctionne efficacement sur des appareils mobiles à faibles ressources et sans connexion Internet. De plus, cette étude ne traite pas des ravageurs spécifiques comme la chenille légionnaire, ni la maladie virale du maïs

ScienceDirect – A Multi-Scale Feature Fusion Neural Network for Multi-Class Disease Classification in Maize Leaf Images, Liangliang Liu, Shixin Qiao, Jing Chang, Weiwei

Ding, Cifu Xu, Jiamin Gu, Tong Sun, Hongbo Qiao. [14]

<https://www.sciencedirect.com/science/article/pii/S2405844024042956>

Chapitre 2 : Architecture matérielle, logicielle et méthodologie de mise en œuvre

Chapitre 2 : Architecture matérielle, logicielle et méthodologie de mise en œuvre

Section 1 : Architecture matérielle

1.1. Unités de traitement

Le traitement et l'entraînement des modèles d'apprentissage profond reposent sur la puissance de calcul des unités de traitement. Deux types d'unités sont principalement impliqués : le processeur central (**CPU**) et le processeur graphique (**GPU**).

CPU (Central Processing Unit)

Le **CPU** est l'unité principale de traitement d'un ordinateur. Il est conçu pour exécuter des instructions de manière séquentielle et est particulièrement efficace pour les tâches générales comme la gestion du système d'exploitation, l'exécution de scripts, ou le traitement de fichiers. Toutefois, il montre ses limites lorsqu'il s'agit de traiter simultanément un grand volume de données, comme c'est le cas pour l'entraînement de modèles de deep learning.

GPU (Graphics Processing Unit)

Le GPU est une unité de traitement conçue initialement pour gérer le rendu graphique. Il est cependant devenu un outil central dans le domaine du deep learning en raison de sa capacité à exécuter un grand nombre d'opérations en parallèle. Cela le rend particulièrement adapté aux calculs matriciels et tensoriels nécessaires à l'entraînement des réseaux de neurones convolutionnels (**CNN**), comme YOLO. [15]

1.2. Plateforme matérielle utilisée

Pour l'implémentation et les tests de notre modèle de détection de maladies et de ravageurs de plantes à l'aide de l'architecture **YOLO**, nous avons choisi d'exploiter un

environnement de calcul cloud. Cette solution offre une infrastructure scalable et performante, permettant d'exécuter efficacement des tâches de deep learning sans les contraintes matérielles liées à un poste local.

Nous avons utilisé **Kaggle Notebooks**, un environnement de développement collaboratif basé sur le cloud, qui met à disposition des ressources matérielles avancées. Plus précisément, l'entraînement du modèle a été réalisé sur un processeur graphique (**GPU**) **NVIDIA T4** une carte graphique conçue pour les applications d'intelligence artificielle qui offre un bon compromis entre performance et accessibilité pour les tâches de deep learning. Ce choix nous a permis de bénéficier :

- D'un environnement préconfiguré avec les bibliothèques nécessaires (PyTorch, OpenCV, etc.),
- D'une exécution rapide des scripts, notamment lors de l'entraînement sur des images,
- D'un gain de temps considérable pour les expérimentations.



Figure 6: Logo officiel de Kaggle

Site officiel : <https://www.kaggle.com/>

En plus de l'environnement cloud sur **Kaggle**, certains traitements de base, tels que le nettoyage des données ou la vérification des formats, ont été réalisés localement sur un ordinateur personnel **Dell Inspiron 7501** équipé d'un processeur **Intel® Core™ i5-10300H** à 8 cœurs, avec 16 Go de mémoire RAM et un espace de stockage de 250,1 Go. Ce système fonctionne sous **Ubuntu 24.04.2 LTS** avec le noyau **Linux 6.11.0-26-generic** et utilise l'interface graphique **GNOME 46**. Ces traitements ont été effectués dans un environnement virtuel géré avec **Anaconda** et exécutés dans des notebooks **Jupyter**, ce qui a facilité la manipulation interactive des données et l'intégration de bibliothèques Python spécialisées. Cela a permis de préparer les données avant les phases d'entraînement intensif

sur **GPU**.

Section 2 : Architecture logicielle

2.1. Bibliothèques et frameworks utilisés

➤ **PyTorch**

PyTorch est un **framework open source d'apprentissage** profond développé par Meta AI, largement reconnu pour sa flexibilité, sa simplicité d'utilisation et ses capacités d'intégration avec Python. Il est particulièrement adapté à la recherche en intelligence artificielle et au prototypage rapide de modèles complexes. Dans notre projet de détection de maladies et de ravageurs des plantes à l'aide de l'architecture **YOLO11**, **PyTorch** a joué un rôle central. Il a permis de charger efficacement les modèles **YOLO** pré-entraînés, d'effectuer un entraînement personnalisé avec notre propre jeu de données, et de gérer les calculs intensifs via le **GPU** grâce à son support natif de **CUDA**. [16]



Figure 7: Logo Officielle de Pytorch

Site officiel : <https://pytorch.org/>

➤ **CUDA (Compute Unified Device Architecture)**

CUDA est une plateforme de calcul parallèle et une interface de programmation (API) développée par **NVIDIA**. Elle permet d'utiliser la puissance de calcul des cartes graphiques (**GPU**) **NVIDIA** pour exécuter des tâches qui demandent beaucoup de ressources, comme l'entraînement des modèles de deep learning. **CUDA** permet à des frameworks comme **PyTorch** ou **TensorFlow** d'utiliser le **GPU** au lieu du **CPU**, ce qui accélère énormément les calculs. Par exemple, au lieu d'entraîner un modèle en 10 heures sur un **CPU**, on peut le faire en 1 heure ou moins sur un **GPU** grâce à **CUDA**.

NVIDIA est une entreprise technologique américaine spécialisée dans la conception de processeurs graphiques (**GPU**) et de technologies associées. Elle est reconnue mondialement pour ses cartes graphiques hautes performances utilisées dans les jeux vidéo, la visualisation graphique, mais aussi dans les domaines du calcul scientifique et de l'intelligence artificielle. **NVIDIA** a développé **CUDA**, une plateforme de calcul parallèle qui permet d'exploiter la puissance de ses **GPU** pour accélérer des applications telles que l'entraînement des modèles de **deep learning**.

Site officiel : <https://www.nvidia.com/en-us/>

[17]



Figure 8:– Logo officiel de l'entreprise NVIDIA

➤ **OpenCV (Open-Source Computer Vision Library)**

OpenCV est une bibliothèque open source spécialisée dans le traitement d'images et la vision par ordinateur. Dans notre projet, **OpenCV** a été principalement utilisée pour les étapes de prétraitement des images, notamment pour la lecture, la conversion de formats, le redimensionnement à une taille compatible avec **YOLO11**, ou encore l'annotation visuelle des prédictions. Sa compatibilité avec **NumPy** et sa vitesse d'exécution en font un choix idéal pour préparer efficacement les données avant l'entraînement d'un modèle de deep learning.

[18]



Figure 9:Logo officiel de la bibliothèque OpenCV

Site officiel : <https://opencv.org/>

➤ **Anaconda**

Anaconda est une distribution open source de **Python** et R, largement utilisée en science des données, apprentissage automatique et intelligence artificielle. Dans notre projet, Anaconda a été utilisé pour créer et gérer un environnement virtuel dédié à la phase de préparation des données. Cet environnement contenait toutes les bibliothèques nécessaires (**OpenCV**, **Albumentations**, **scikit-learn**, etc.) sans interférer avec le système global. Cela a assuré une compatibilité parfaite entre les dépendances logicielles et facilité le déploiement de scripts reproductibles sur différentes machines.



Figure 10:Logo officiel de la distribution Anaconda

[19] **Site officiel :** <https://www.anaconda.com>

➤ **Jupyter Notebooks**

Jupyter est une application open-source permettant de créer des documents interactifs combinant du code exécutable, des visualisations dynamiques, des explications textuelles et des formules mathématiques. Dans notre cas, les notebooks Jupyter ont servi de support interactif pour explorer, visualiser et transformer les données d'entrée avant l'entraînement du modèle. Cela a permis une manipulation itérative et documentée des images, annotations, tout en gardant une trace claire de chaque étape de préparation.



Figure 11: Logo officiel de Jupyter Notebooks

[20] Site officiel : <https://jupyter.org>

➤ Python

Python est un langage de programmation interprété, orienté objet et de haut niveau, caractérisé par une sémantique dynamique et une syntaxe claire et intuitive. Dans notre projet, Python a servi de fondation pour le développement de l'ensemble des scripts de traitement de données, d'entraînement des modèles et d'intégration des différentes bibliothèques utilisées. Sa richesse en bibliothèques spécialisées, telles que PyTorch, OpenCV et scikit-learn, a permis une mise en œuvre efficace des différentes étapes du pipeline de détection des maladies et ravageurs des plantes.



Figure 12: Logo officiel du langage Python

[21] Site officiel : <https://www.python.org>

➤ TensorFlow

TensorFlow est une bibliothèque open-source de machine learning développée par **Google**, largement utilisée pour la création, l'entraînement et le déploiement de modèles d'intelligence artificielle. Elle prend en charge les réseaux de neurones profonds, les flux de données, ainsi que l'accélération **GPU**, et elle est compatible avec divers langages comme **Python, JavaScript ou C++**. TensorFlow sert de base pour de nombreux outils et extensions, notamment TensorFlow Lite, qui est spécifiquement conçu pour l'inférence sur appareils mobiles ou embarqués.



Figure 13: Logo officiel de la bibliothèque TensorFlow

[22] Site officiel : <https://www.tensorflow.org>

➤ TensorFlow Lite (TFLite)

TensorFlow Lite est une bibliothèque légère développée par **Google** pour permettre l'exécution de modèles de machine learning sur des appareils mobiles et embarqués (Android, iOS, microcontrôleurs). Conçue pour la performance et la faible consommation de ressources, **TFLite** prend en charge l'inférence de modèles de deep learning directement sur l'appareil, sans besoin de connexion à un serveur. **[23]**

Dans notre projet, **TensorFlow Lite** a été utilisé pour convertir et optimiser notre modèle entraîné avec **PyTorch**. Cette conversion a permis d'**exporter** le modèle en formats **float32**, **float16** et même **int8**, facilitant ainsi son déploiement sur des **appareils mobiles** et **embarqués** à **faibles ressources**. Cela permet un appel efficace du modèle avec une consommation réduite de mémoire et une rapidité d'inférence adaptée à la détection locale des maladies et ravageurs des plantes.



Figure 14:: Logo officiel de la bibliothèque TensorFlow Lite

Site officiel : <https://www.tensorflow.org/lite>

➤ Streamlit

Streamlit est un outil **open-source en Python** qui permet de créer facilement des applications web interactives.

Dans notre projet, Streamlit a permis de simuler l'efficacité du modèle en conditions d'utilisation réelles, en proposant une interface accessible pour tester les performances de détection . Il a servi à construire l'interface permettant de visualiser les résultats de détection des maladies et des ravageurs.



Figure 15: Logo officiel de Streamlit

[24] Site officiel : <https://streamlit.io>

➤ **Ultralytics**

Ultralytics est une bibliothèque Python open-source spécialisée dans la vision par ordinateur, offrant des modèles avancés basés sur la famille **YOLO** (You Only Look Once). Conçue pour des performances optimales, elle permet de répondre à une variété de tâches telles que la détection d'objets, la classification d'images et la segmentation sémantique. Dans le cadre de notre projet, nous avons exploité **YOLO11** via Ultralytics pour détecter avec précision les maladies et parasites affectant les plantes.

Cette solution nous a permis de bénéficier d'une infrastructure complète allant du prétraitement des images à l'inférence finale, tout en intégrant facilement des modules pour la détection. L'architecture modulaire et les performances optimisées de la bibliothèque ont largement accéléré nos cycles de développement, tout en assurant un haut niveau de fiabilité dans les résultats. L'intégration fluide avec les environnements Jupyter a en outre facilité notre travail d'expérimentation et d'analyse.

Site officiel : <https://ultralytics.com>

➤ **Ultralytics YOLO11**

YOLO11 est la dernière itération de la série des détecteurs d'objets en temps réel d'Ultralytics redéfinissant ce qui est possible avec une précision, une vitesse et une efficacité de pointe. S'appuyant sur les avancées impressionnantes des versions précédentes de YOLO , YOLO11 apporte des améliorations significatives à l'architecture et aux méthodes d'apprentissage, ce qui en fait un choix polyvalent pour un large éventail de tâches de vision par ordinateur. [25]



Figure 16: Logo officiel de l'organisation Ultralytics, créatrice de YOLO11

Caractéristiques principales : [25]

- **Extraction améliorée des caractéristiques :** YOLO11 utilise une architecture améliorée de l'épine dorsale et du cou, qui renforce les capacités d'extraction des caractéristiques pour une détection plus précise des objets et l'exécution de tâches complexes.
- **Optimisé pour l'efficacité et la rapidité :** YOLO11 présente des conceptions architecturales raffinées et des pipelines de formation optimisés, offrant des vitesses de traitement plus rapides et maintenant un équilibre optimal entre la précision et la performance.
- **Une plus grande précision avec moins de paramètres :** Grâce aux progrès réalisés dans la conception du modèle, **YOLO11m** atteint une précision moyenne plus élevée (**mAP**) sur l'ensemble de données **COCO** tout en **utilisant 22 %** de paramètres en moins que YOLOv8m, ce qui le rend efficace sur le plan des calculs sans compromettre la précision.
- **Adaptabilité à tous les environnements :** YOLO11 peut être déployé de manière transparente dans différents environnements, y compris les appareils périphériques, les plateformes cloud et les systèmes prenant en charge les GPU NVIDIA , ce qui garantit une flexibilité maximale.

- **Large éventail de tâches prises en charge** : Qu'il s'agisse de détection d'objets, de segmentation d'instances, de classification d'images, d'estimation de la pose ou de détection d'objets orientés (OBB), YOLO11 est conçu pour répondre à un ensemble varié de défis en matière de vision par ordinateur.

Architecture de YOLO11 :

L'architecture YOLO 11 est divisée en trois grandes parties :

- ✓ **Backbone** : C'est la partie du réseau qui extrait les caractéristiques essentielles de l'image grâce à une série de couches convolutives. Il transforme l'image brute en cartes de caractéristiques riches, servant de base à la détection d'objets.
- ✓ **Neck** : Ce module intermédiaire fusionne et affine les caractéristiques extraites par le backbone. Il permet de combiner l'information provenant de différentes échelles pour produire des représentations multi-échelles, essentielles pour détecter des objets de tailles variées dans l'image.
- ✓ **Head** : La **head** utilise ces représentations pour effectuer les prédictions finales.

Elle identifie la classe de chaque objet détecté, localise précisément sa position via la régression des coordonnées des boîtes englobantes, et attribue un score de confiance à chaque détection. Les **heads YOLO11** sont optimisées pour la rapidité et la précision, et peuvent être anchor-free selon la configuration du modèle.

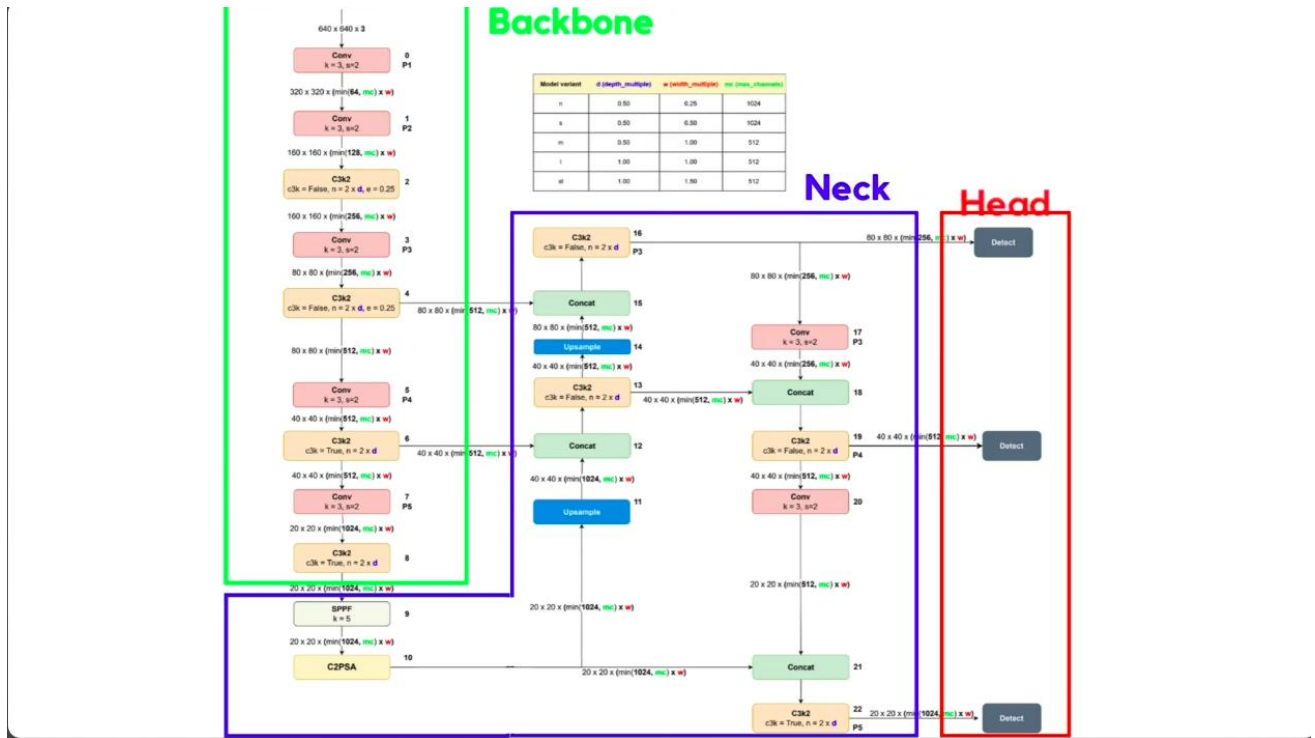


Figure 17: Architecture modulaire de YOLO11 : Backbone, Neck et Head

Variantes de YOLO11 et Structure Architecturale :

YOLO11 existe en cinq variantes adaptées à différents niveaux de puissance de calcul et de précision :

- YOLO11n – Nano, conçu pour des tâches petites et légères.
- YOLO11s – Small, une version légèrement améliorée du Nano avec un peu plus de précision.
- YOLO11m – Medium, destiné à une utilisation polyvalente.
- YOLO11l – Large, offre une meilleure précision, mais avec un coût de calcul plus élevé.
- YOLO11x – Extra-large, pour une précision et des performances maximales. [25]



Performance

Détection (COCO)

Segmentation (COCO)

Classification (ImageNet)

Pose (COCO)

OBB (DOTAv1)

Voir [Détection Docs](#) pour des exemples d'utilisation de ces modèles formés sur [COCO](#), qui comprennent 80 classes préformées.

Modèle	taille (pixels)	mAPval 50-95	Vitesse CPU ONNX (ms)	Vitesse T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

Figure 18: Comparaison des mesures de performance des cinq variantes de YOLO

Explication des blocs architecturaux [26]

C3_K2

Le bloc **C3_K2** est une version optimisée du bloc C3 utilisé dans YOLOv5, conçue pour améliorer l'efficacité computationnelle tout en maintenant de bonnes performances. Il conserve la structure générale des blocs CSP (Cross Stage Partial), combinant des connexions résiduelles avec des convolutions profondes, mais adopte une architecture modulaire basée sur des blocs **C3K**, qui utilisent des noyaux 3×3 organisés de manière plus compacte et efficace. Plutôt que de simplement réduire la taille des noyaux, le bloc C3_K2 optimise le flux d'information à travers une série de blocs C3K, ce qui diminue le nombre de paramètres et les opérations coûteuses tout en préservant la capacité d'extraction de caractéristiques riches. Cette conception allégée permet un apprentissage plus rapide, limite la redondance des gradients et s'avère particulièrement adaptée aux dispositifs à ressources limitées ou aux applications en temps réel.



Figure 19: Schéma explicatif du bloc C3_K2 optimisé dans l'architecture YOLO11

SPPF (Spatial Pyramid Pooling Fast) :

Le module **SPPF (Spatial Pyramid Pooling Fast)**, conservé dans YOLO11, permet d'extraire efficacement des caractéristiques à partir de différentes régions de l'image et à plusieurs échelles, améliorant ainsi la détection d'objets de tailles variées, notamment les petits objets souvent difficiles à identifier dans les versions précédentes. Il utilise plusieurs opérations de **max pooling** avec des noyaux de tailles différentes pour agréger des informations contextuelles multi-échelles, ce qui enrichit la représentation des objets à différentes résolutions. Cette approche renforce la capacité du modèle à détecter des objets sur l'ensemble de l'image tout en **maintenant une vitesse de traitement en temps réel**, ce qui est crucial pour les systèmes embarqués et les applications interactives.

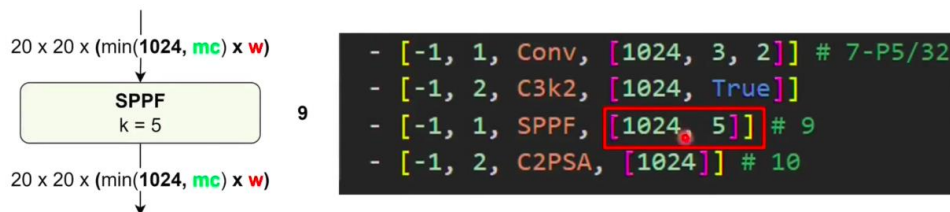


Figure 20: Schéma du module SPF (Spatial Pyramid Fast Fusion) dans YOLO11

C2_PSA

Le bloc **C2PSA (Cross Stage Partial with Spatial Attention)** constitue l'une des innovations clés de YOLO11, introduisant des mécanismes d'attention visant à améliorer la détection d'objets petits ou partiellement occultés en mettant l'accent sur la pertinence spatiale des cartes de caractéristiques. Il intègre une attention sensible à la position, où les

tenseurs d'entrée sont traités par une couche d'attention, concaténés avec l'entrée d'origine, puis passés à travers un réseau de neurones feed-forward et deux blocs convolutifs (avec et sans activation), avant une nouvelle concaténation. Par ailleurs, le bloc C2PSA utilise deux modules d'attention spatiale partielle (PSA) appliqués sur des branches séparées de la carte de caractéristiques, qui sont ensuite fusionnées selon une architecture proche du bloc C2F. Cette conception permet à YOLO11 de mieux se focaliser sur les régions pertinentes d'une image tout en maintenant un bon équilibre entre précision et efficacité computationnelle, surpassant ainsi les performances de YOLOv8 dans les scénarios exigeant une détection fine des détails.

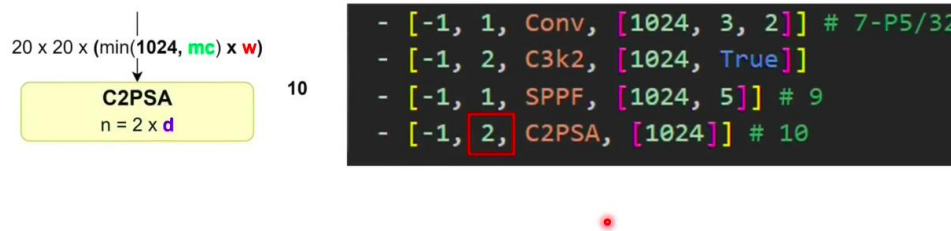


Figure 21: Schéma du module C2_PSA (Cross Stage Partial + Position-Sensitive Attention) dans YOLO11

Upsample

Upsample (**suréchantillonnage**) : il s'agit d'une opération qui augmente la taille spatiale (hauteur et largeur) des cartes de caractéristiques. Dans YOLO, l'opération « nn.Upsample » avec le mode « nearest » permet d'aligner les résolutions entre différentes cartes avant leur fusion, notamment lors de la concaténation d'informations issues de plusieurs profondeurs du réseau

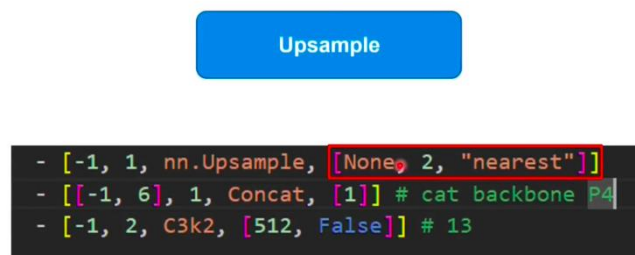


Figure 22: Schéma illustrant l'opération d'upsampling (suréchantillonnage) par méthode du plus proche voisin

Concat

Concat (Concaténation) : cette opération fusionne plusieurs cartes de caractéristiques en les concaténant le long d'une dimension spécifiée (généralement l'axe des canaux), tout en conservant la même hauteur et largeur. Cela permet de regrouper des informations issues de différentes couches du réseau, enrichissant la représentation globale du modèle

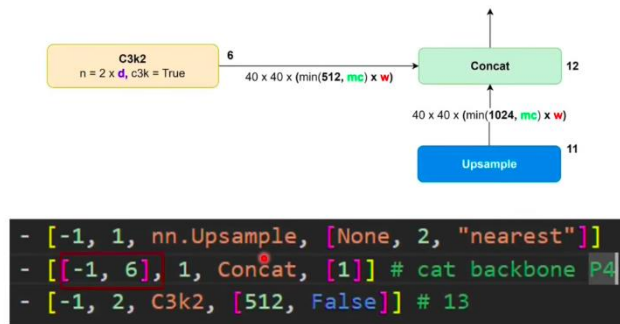


Figure 23: Schéma de l'opération de concaténation des cartes de caractéristiques le long de l'axe des canaux

Detect

Detect : C'est le bloc de sortie final du modèle, chargé de prédire les objets à différentes échelles (petits, moyens, grands). Il génère pour chaque cellule de la carte de caractéristiques des boîtes englobantes avec leurs coordonnées, une confiance d'objet, et les classes probables, permettant ainsi de localiser et classer les objets dans l'image.

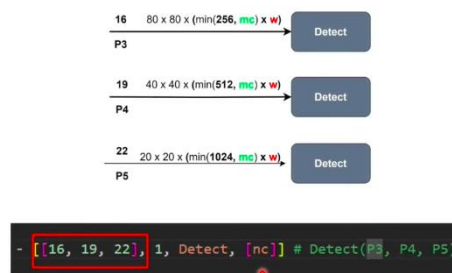


Figure 24:– Schéma du bloc Detect : prédiction finale des objets à différentes échelles dans YOLO11

Section 3 : Architecture logique de la solution

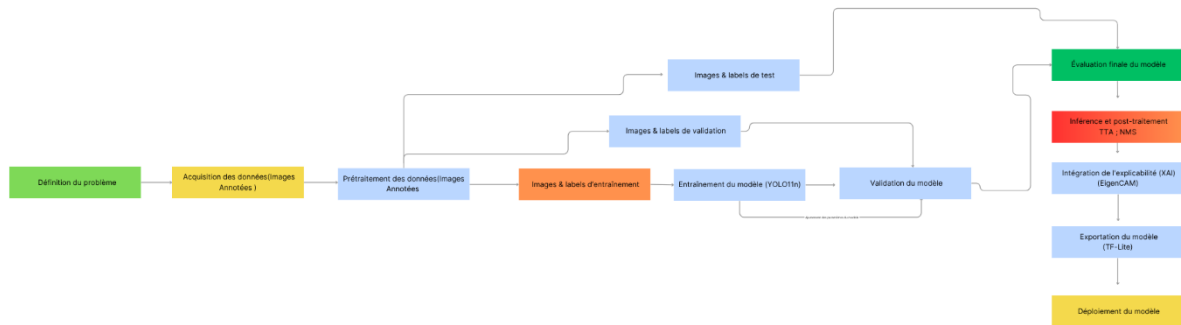


Figure 25: Architecture logique globale de la solution de détection basée sur YOLO11n

3.1 Définition du problème :

Cette phase initiale consiste à bien comprendre le problème à résoudre afin de sélectionner les techniques de Machine Learning les plus adaptées. Pour cela, il faut déterminer si le problème est de type supervisé (classification, régression) ou non supervisé (clustering), puis choisir les algorithmes appropriés.

Dans notre cas, le problème est de type **supervisé**, car il s'agit de détecter, identifier et localiser précisément, à partir d'images annotées de plants de maïs, différentes maladies et ravageurs. Cette approche supervisée nous permet d'exploiter un jeu de données annoté (**KaraAgro AI Maize**) pour entraîner un modèle performant capable de reconnaître et localiser les symptômes sur les images.

3.2 Acquisition de données (Data acquisition) :

Notre étude repose sur un jeu de données spécialement conçu pour le diagnostic des maladies du maïs en conditions réelles africaines. Ce dataset, développé par la **KaraAgro AI**

Foundation en collaboration avec des experts agronomes ghanéens, a été constitué selon une méthodologie rigoureuse garantissant sa qualité et sa représentativité.

La collecte s'est déroulée pendant quatre mois (mai à août **2021**) dans l'ensemble des zones de production du Ghana. Des agents formés ont utilisé une application mobile dédiée pour capturer des images des plants de maïs, couvrant ainsi différentes variétés, stades de croissance et conditions environnementales. Au total, plus de **18 565 images** ont été recueillies, documentant trois états phytosanitaires : plants sains , infectés par le virus de la striure et attaqués par la chenille légionnaire .

L'utilisation de ce dataset présente plusieurs avantages majeurs pour notre recherche. Sa couverture géographique exhaustive permet de tenir compte des spécificités agroécologiques des différentes régions. La validation experte des annotations assure la qualité des données d'apprentissage. Enfin, la richesse des métadonnées associées ouvre des perspectives intéressantes pour des analyses spatiales et temporelles des pathologies du maïs. Toutefois, on note un fort déséquilibre au niveau des annotations, c'est-à-dire dans le nombre d'instances par classe (zones annotées sur les images), alors que le nombre d'images par classe reste relativement équilibré. Cette disparité pourrait biaiser l'apprentissage du modèle en favorisant la détection des classes les plus représentées au niveau des annotations.

3.3 Prétraitement & séparation des données

3.3.1 Analyse et nettoyage des annotations

Pour commencer le prétraitement, nous avons passé en revue toutes les images du dataset, principalement au format **.jpeg et .jpg**, ainsi que leurs annotations dans un dossier **labels/**. Nous avons fait un inventaire précis des différentes classes présentes : par exemple, **les dégâts causés par les larves de la légionnaire d'automne représentent près de 41 % des instances, tandis que les plants sains représentent environ 26 %, et les traces d'excréments de la larve environ 16 %**. Au total, 18 625 fichiers d'annotation **XML** ont été examinés, et nous avons retiré toutes les annotations inutiles ou non pertinentes, notamment celles marquées comme none-healthy. Ce nettoyage a permis d'assurer que nos données sont de bonne qualité et sans doublons, aussi bien pour les images que pour les fichiers **XML**. Grâce à cela, notre jeu de données est propre, fiable et prêt à être utilisé pour entraîner notre modèle.

3.3.2 Renommage et organisation des fichiers

Pour mieux organiser nos données et éviter toute confusion liée aux noms de fichiers parfois longs ou compliqués, nous avons procédé à une étape de renommage automatique. Chaque image a été copiée dans un nouveau dossier avec un nom unique, facile à identifier, comme par exemple **ID_c9e4c8b7.jpg**. Les fichiers d'annotation correspondants ont aussi été renommés de la même manière pour garder une cohérence parfaite entre les images et leurs annotations. **Grâce à ce travail, nous avons pu traiter un total de 18 625 images et annotations, sans perdre aucun fichier.** Pour garder une trace claire, nous avons aussi créé un tableau récapitulatif qui relie les anciens noms aux nouveaux, ce qui facilitera le suivi dans les étapes suivantes.

3.3.3 Validation des annotations et contrôle de qualité

Dans cette troisième phase du prétraitement, nous avons pris le temps de passer en revue l'ensemble des annotations **XML** afin de nous assurer que leur structure était correcte et que les objets identifiés dans les images étaient valides. Cette étape a été réalisée après avoir écarté les annotations liées à la classe non-healthy, jugées non pertinentes pour notre analyse. **L'examen a permis de recenser sept classes bien distinctes dans le jeu de données : fall-armyworm-egg, fall-armyworm-frass, fall-armyworm-larva, fall-armyworm-larval-damage, healthy-images, healthy-maize et maize-streak-disease.** Très peu d'anomalies ont été relevées, que ce soit dans les fichiers ou les objets annotés, ce qui montre que les données sont dans un état globalement cohérent et maîtrisé à ce stade.

3.3.4 Suppression des objets invalides dans les annotations

Lors de cette étape, nous avons procédé à une vérification détaillée des coordonnées des objets annotés dans les fichiers XML afin de détecter et supprimer ceux qui étaient invalides (par exemple, des boîtes englobantes mal définies ou avec des valeurs négatives). Grâce à un script automatisé, 44 objets incorrects ont été retirés du dataset. Après cette correction, une nouvelle validation complète a été effectuée, confirmant que toutes les annotations restantes sont désormais parfaitement valides, sans aucun fichier corrompu ni objet invalide. Le jeu de données compte à présent 18 600 images annotées, avec un total de **101 412** objets validés, garantissant ainsi une meilleure qualité pour la suite du traitement.

3.3.5 Génération du fichier CSV des annotations à partir des XML & Analyse statistique de la distribution des classes d'objets

Pour faciliter l'analyse et le traitement des données, nous avons converti les annotations XML en un fichier CSV nommé **Data.csv**, contenant une ligne par objet annoté avec les informations suivantes : le nom du fichier image correspondant (**Image_ID**), le nom de la classe de l'objet (**class**), une confiance fixée par défaut à 1.0 (**confidence**), les coordonnées de la boîte englobante sous la forme **ymin, xmin, ymax, xmax**, l'identifiant numérique de la classe (**class_id**) et le chemin complet vers l'image (**ImagePath**). Le script recherche les fichiers **XML** dans le dossier labels, associe chaque XML à une image dans images/ en testant plusieurs extensions (.jpg, .jpeg, etc.), lit et nettoie les noms de classes (suppression des espaces et caractères invisibles), vérifie que les classes font partie des 7 classes définies, puis extrait et écrit ces informations dans le CSV. Des alertes sont affichées en cas d'images manquantes, classes inconnues, objets sans boîte englobante ou coordonnées invalides, ce qui permet de contrôler la qualité des données avant extraction. Après génération du fichier, nous avons analysé la distribution des annotations par classe, qui révèle que la classe la plus **fréquente est fall-armyworm-larval-damage avec 41 307 objets (40,73 %)**, suivie de **healthy-maize (25,93 %)** et **fall-armyworm-frass (15,84 %)**, tandis que les **classes fall-armyworm-egg et healthy-images sont minoritaires avec respectivement 1,10 % et 0,31 %**. En nombre d'images uniques par classe (sur un total de 18 624 images), **maize-streak-disease couvre 36,68 %, healthy-maize 31,09 % et fall-armyworm-larval-damage 31,02 %, tandis que healthy-images représente seulement 0,17 %**. Pour visualiser cette distribution, nous avons utilisé pandas, matplotlib et seaborn pour créer un graphique à barres verticales ordonné et coloré selon chaque classe, affichant le nombre exact d'annotations sur chaque barre, avec une légende colorée facilitant l'identification. Cette visualisation permet de détecter les déséquilibres entre les classes au niveau des instances d'objets annotés, une information cruciale pour assurer un entraînement équilibré et efficace des modèles d'apprentissage automatique, ce qui nous permettra plus tard de garantir une efficacité optimale tout en évitant les biais, assurant ainsi l'équité et le respect des principes éthiques de l'IA.

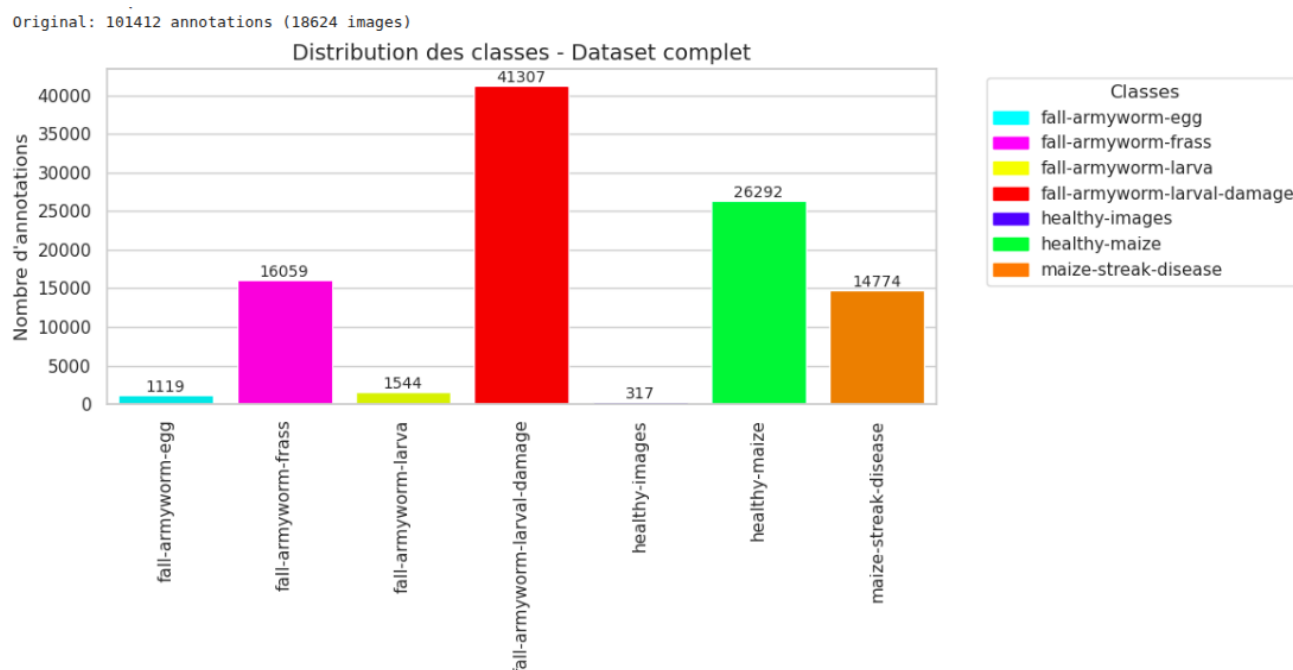


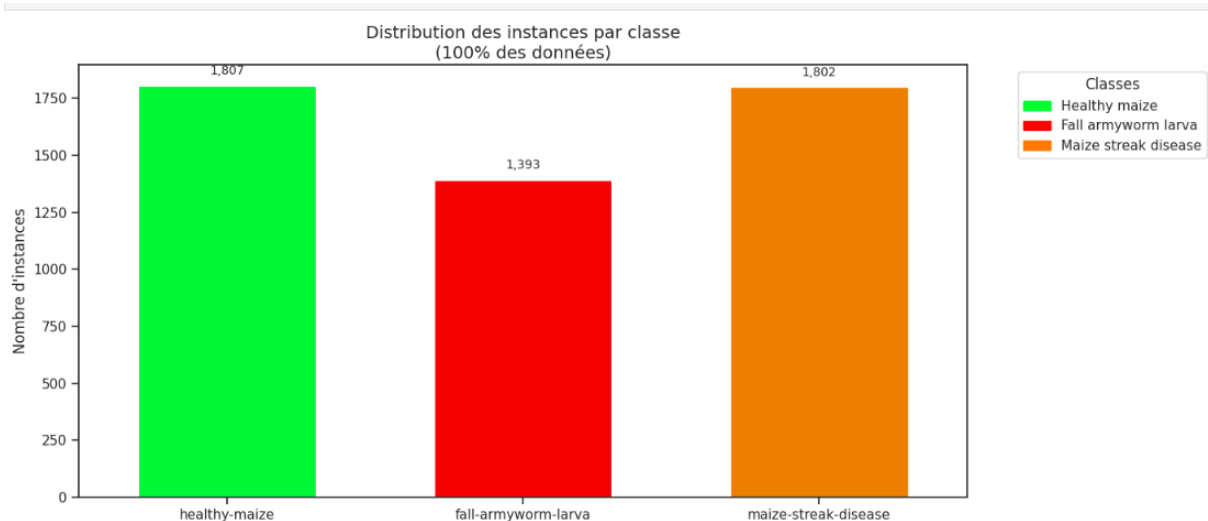
Figure 26: Histogramme des annotations par classe Data.csv

3.3.6 Regroupement et équilibrage des classes phytosanitaires

Dans la démarche de résolution du déséquilibre entre classes, les annotations du fichier Data.csv ont été filtrées afin de ne conserver que trois classes pertinentes pour l'analyse : **fall-armyworm-larva (larves)**, **healthy-maize (maïs sain)** et **maize-streak-disease (maladie de la striure du maïs)**. Les annotations appartenant à d'autres classes ont été exclues, et les images associées, ne contenant aucune annotation valide, ont été déplacées dans un dossier dédié images/excluded. Cette étape a permis de générer le fichier Filtered_Data.csv contenant uniquement les annotations valides, ainsi que **Excluded_Data.csv** pour les annotations écartées. L'analyse de la répartition des classes dans **Filtered_Data.csv** montre un déséquilibre important : healthy-maize représente 61,88 % des annotations (26 292), maize-streak-disease 34,80 % (14 774), et fall-armyworm-larva seulement 3,63 % (1 544).

Pour atténuer ce **déséquilibre**, un **sous-échantillonnage a été réalisé à partir du fichier Filtered_Data_Mapped.csv**, en sélectionnant un nombre limité d'images par classe afin d'obtenir une distribution plus équilibrée sans perte de cohérence contextuelle. Les images ont été sélectionnées de manière à ne pas dépasser 1,3 fois le nombre minimal d'annotations observé parmi les classes, garantissant ainsi un compromis entre diversité et équité. Un split stratifié de ces images (**90 % pour l'entraînement, 10 % pour le test**) a été

effectué selon la classe dominante de chaque image, générant les fichiers **mini_train.csv** et **mini_test.csv**. Ce processus a abouti à un jeu d'entraînement mieux équilibré avec environ 36 % d'annotations pour healthy-maize (1 807), 36 % pour maize-streak-disease (1 802), et 28 % pour fall-armyworm-larva (1 393), tandis que l'échantillon de test reflète une distribution similaire.



*Figure 27:– Histogramme des annotations équilibrées par classe
Data.csv*

3.3.7 Conversion des annotations et génération des labels au format YOLO

Après avoir filtré et équilibré les annotations, l'étape suivante consiste à les convertir dans un format adapté à l'entraînement des modèles **YOLO**. Cela signifie transformer les coordonnées des boîtes englobantes présentes dans le fichier CSV en labels YOLO, où les coordonnées (centre x, centre y, largeur, hauteur) sont normalisées par rapport à la taille de chaque image. Le processus se déroule en plusieurs étapes essentielles : d'abord, les images sont chargées avec une correction de l'orientation **EXIF** pour s'assurer que les coordonnées correspondent bien à la position réelle des objets ; ensuite, les annotations sont regroupées par image afin de traiter toutes les boîtes associées en une seule fois ; puis, les coordonnées sont normalisées en divisant les valeurs absolues par la largeur et la hauteur de

l'image ; enfin, un fichier texte est généré pour chaque image, respectant le format YOLO (**class_id x_center y_center width height**), avec des valeurs comprises entre 0 et 1. Ce travail a été entièrement automatisé grâce à un script Python qui lit les fichiers CSV d'annotations d'entraînement et de test, charge les images en tenant compte de leur orientation, calcule les coordonnées normalisées, puis enregistre les fichiers **txt** correspondants dans des dossiers spécifiques.

Dans le format YOLO, les coordonnées des boîtes englobantes sont normalisées par rapport à la taille de l'image, ce qui signifie que toutes les valeurs sont comprises entre 0 et 1. Chaque ligne de label correspond à : **class_id x_center y_center width height** où :

- **x_center, y_center** : coordonnées du centre de la boîte, normalisées par la largeur et la hauteur de l'image.
- **width, height** : largeur et hauteur de la boîte, également normalisées.

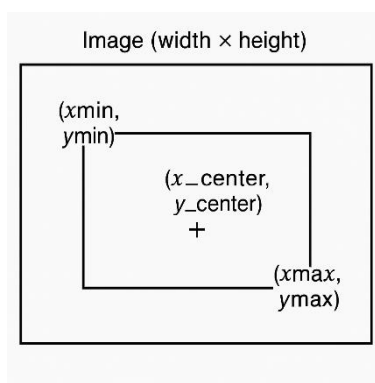


Figure 28: Schématisation des boîtes englobantes et de leurs coordonnées

3.3.6 Génération des cartes thermiques (heatmaps) des zones d'annotations

Afin de visualiser la répartition spatiale des objets annotés dans les images d'entraînement, des cartes thermiques (**heatmaps**) ont été générées pour chaque classe phytosanitaire à partir du fichier train.csv. Le processus repose sur un script Python qui lit les annotations, charge les images tout en corrigeant leur orientation via les métadonnées **EXIF**, puis normalise les coordonnées des boîtes englobantes en les adaptant à une grille de taille fixe (1024×1024). Pour chaque boîte, les pixels correspondants sont incrémentés afin de créer une carte de densité par classe, illustrant les zones où les objets apparaissent le plus

fréquemment. Cette visualisation, réalisée à l'aide de **Seaborn**, a permis d'identifier des concentrations spatiales spécifiques : les larves de chenille légionnaire (**fall-armyworm-larva**) sont souvent localisées dans des zones précises, tandis que les plants sains (**healthy-maize**) présentent une répartition plus homogène. Les heatmaps obtenues ont été sauvegardées dans le dossier **heatmaps_seaborn** sous forme d'images PNG, offrant un aperçu précieux de la densité et de la dispersion des annotations, utile pour détecter d'éventuels biais de cadrage, orienter les stratégies d'augmentation de données, ou encore ajuster les paramètres d'apprentissage des modèles **YOLO**.

La heatmap nous montre la concentration spatiale des objets (elle permet de visualiser si les objets annotés sont fréquemment situés au centre de l'image, dans les coins ou répartis de manière uniforme) ainsi que la qualité des annotations (une distribution anormalement concentrée peut révéler un biais dans le jeu de données, comme des annotations trop homogènes ou mal réparties). Elle aide également à la préparation à l'entraînement (en fournissant des informations utiles pour décider s'il faut appliquer des techniques d'augmentation de données comme le recadrage ou la rotation, ou encore ajuster les ancres des boîtes dans un modèle comme YOLO).

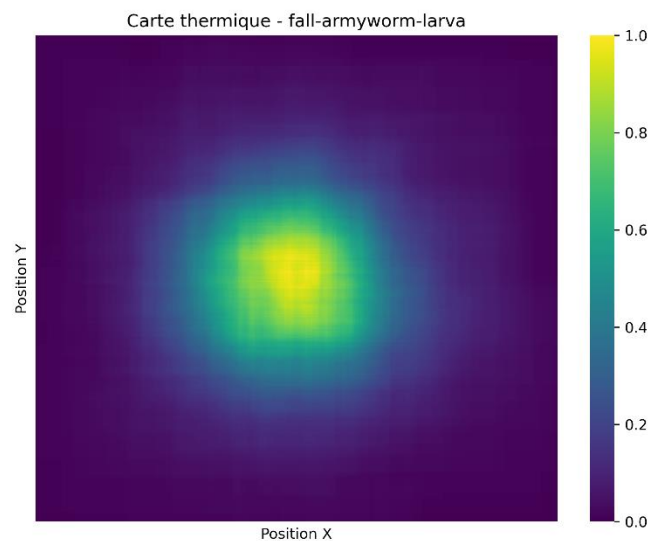


Figure 29: Distribution de la classe Fall Armyworm Larva dans l'ensemble d'entraînement

La **heatmap** de la classe « **fall-armyworm-larva** » montre une forte concentration des annotations au centre de l'image, avec une densité qui décroît rapidement vers les bords. Cela indique que, dans le jeu d'entraînement, les larves de chenille légionnaire sont principalement annotées au centre des images.

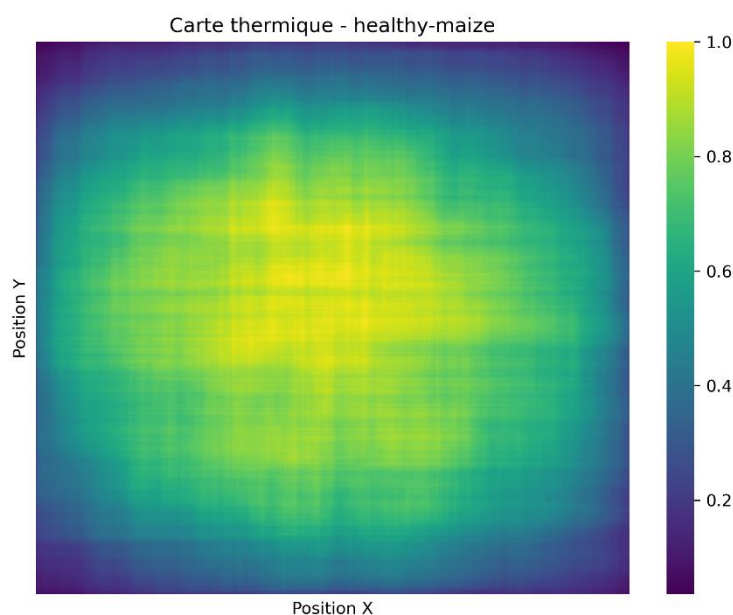


Figure 30: Distribution de la classe *Healthy Maize* dans l'ensemble d'entraînement

La **heatmap** de la classe « **healthy-maize** » présente une répartition beaucoup plus homogène des annotations sur l'ensemble de l'image, avec une légère densité accrue au centre mais sans zone dominante. Cela suggère que les plants de maïs sains sont annotés de façon plus uniforme, couvrant toute la surface des images.

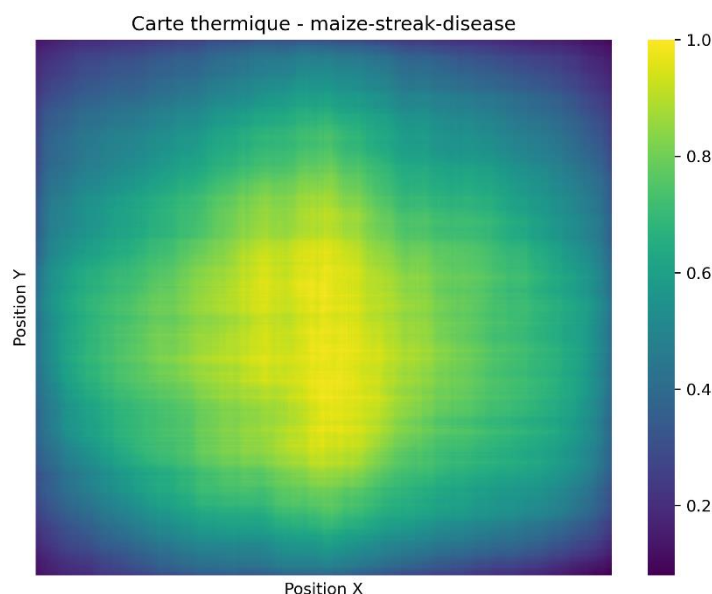


Figure 31: Distribution de la classe *Maize Streak Disease* dans l'ensemble d'entraînement

La **heatmap** de la classe « **maize-streak-disease** » montre une concentration

intermédiaire : les annotations sont plus fréquentes au centre, mais la densité s'étend aussi vers les bords, de façon moins marquée que pour les plants sains. Cela indique que les symptômes de la maladie sont souvent annotés près du centre, tout en étant présents ailleurs dans l'image.

3.3.7 Visualisation des tailles moyennes des boîtes englobantes par classe

Dans le but d'approfondir l'analyse qualitative des annotations du jeu de données d'entraînement, une étude a été menée sur les tailles moyennes des boîtes englobantes (**bounding boxes**) associées à chaque classe phytosanitaire. Cette étape vise à mieux comprendre la distribution des dimensions des objets annotés et à détecter d'éventuels déséquilibres entre les classes, pouvant influencer les performances du modèle. À l'aide d'un script Python, le fichier **train.csv** a été chargé ; il contient les colonnes **class**, **xmin**, **ymin**, **xmax** et **ymax** décrivant chaque annotation. L'aire de chaque boîte a été calculée selon la formule $(xmax - xmin) \times (ymax - ymin)$, puis les annotations ont été regroupées par classe afin de déterminer la surface moyenne occupée par les objets. Les résultats ont été représentés graphiquement sous forme de **treemap** (carte arborescente), une visualisation pertinente pour illustrer les proportions relatives : chaque rectangle correspond à une classe et sa taille reflète l'aire moyenne de ses boîtes englobantes.

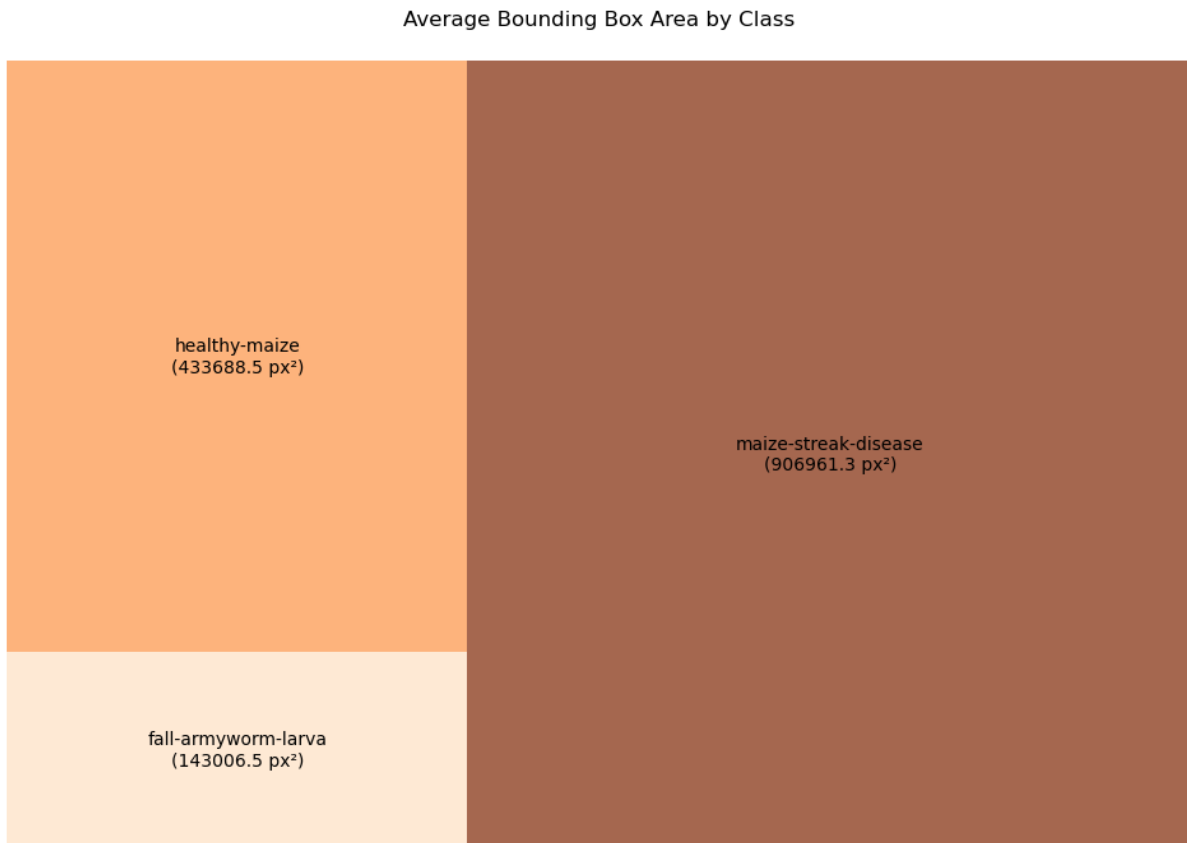


Figure 32: Treemap représentant la taille moyenne des boîtes englobantes par classe phytosanitaire, issue des annotations d'entraînement et générée à l'aide de la bibliothèque squarify

On observe que la classe "**maize-streak-disease**" occupe en moyenne la plus grande surface (environ **906 961 px²**), suivie par "**healthy-maize**" (environ **433 688 px²**), tandis que la classe "**fall-armyworm-larva**" présente la plus petite surface moyenne (environ **143 006 px²**). Cela indique que, dans les images, les symptômes de la maladie de la striure du maïs sont généralement détectés sur des zones beaucoup plus étendues que les plants sains ou les larves de la chenille légionnaire.

3.3.8 Vérification visuelle des annotations : images avec boîtes englobantes

Pour terminer la phase de prétraitement, une vérification visuelle a été réalisée en affichant quelques images du jeu d'entraînement avec leurs boîtes englobantes. L'objectif est de s'assurer que les annotations sont bien positionnées, que les classes sont correctement

attribuées, et que les dimensions des boîtes correspondent réellement aux objets ciblés. Un script Python a été utilisé pour charger chaque image, corriger son orientation grâce aux métadonnées EXIF, et superposer les boîtes englobantes selon les coordonnées présentes dans le fichier train.csv. Chaque classe est affichée dans une couleur spécifique pour faciliter la lecture.

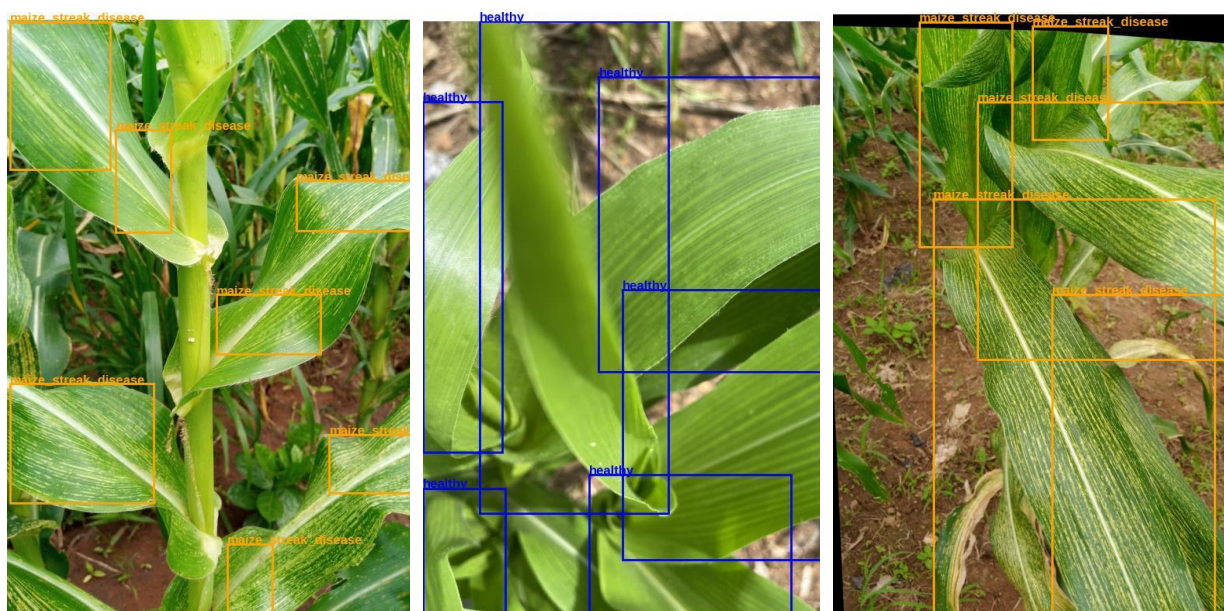


Figure 33: Première série de visualisations des annotations : maladie du maïs et plante saine

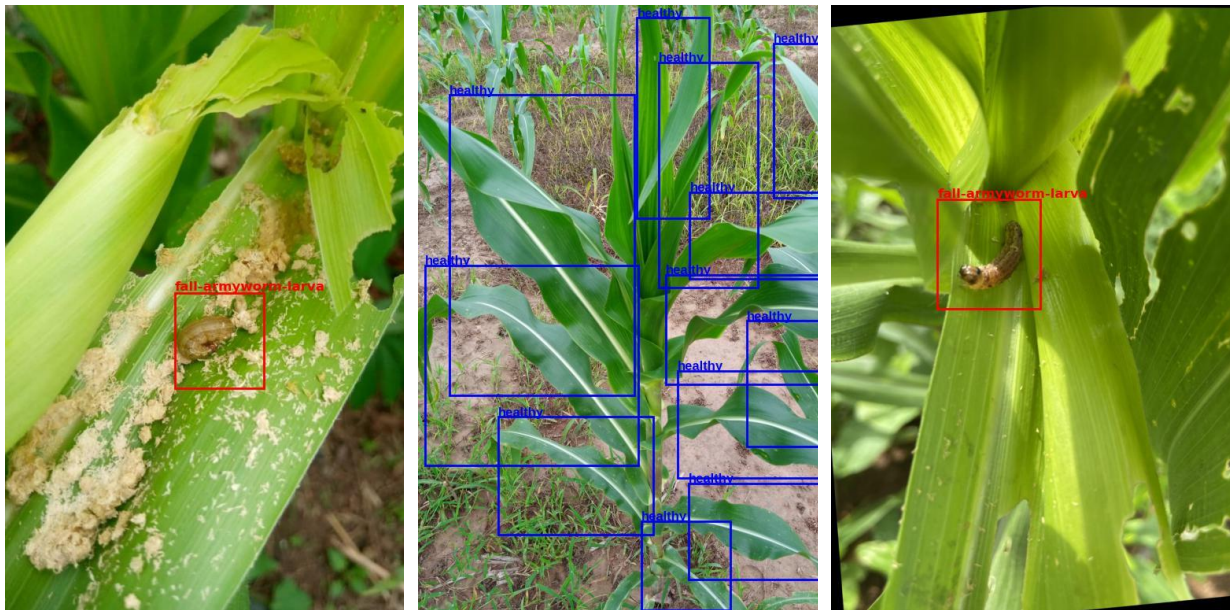


Figure 34:Deuxieme série de visualisations des annotations : ravageurs fall_armyworm et plante saine

3.4 Entraînement du modèle avec YOLO11 (Boucle d'entraînement incluant l'ajustement des paramètres du modèle):

Dans l'environnement **Kaggle**, nous avons préparé un fichier YAML (**data.yaml**) qui détaille les chemins vers les jeux d'entraînement, de validation et de test, ainsi que les trois classes ciblées : '**healthy-maize**', '**maize-streak-disease**' et '**fall-armyworm-larva**'. Pour entraîner notre modèle, nous avons opté pour un modèle **YOLO11n pré-entraîné**, en profitant de l'apprentissage par transfert. Cette technique consiste à partir d'un modèle déjà formé sur un large corpus d'images, ce qui permet de bénéficier des connaissances acquises sur des tâches similaires. Ensuite, nous avons réalisé un **fine-tuning** en ajustant ce modèle à notre jeu de données pendant **300 époques**, en utilisant des images redimensionnées à **640x640 pixels et des lots de 20 images**. L'entraînement s'est déroulé sur un **GPU T4 (15 Go de RAM)**, en appliquant plusieurs techniques d'augmentation des données telles que le flip horizontal, les variations de teinte, saturation et luminosité, la rotation, la mosaïque, l'effacement aléatoire pour renforcer la robustesse et la capacité de généralisation du modèle. Un **scheduler cosine** a permis d'adapter le **taux d'apprentissage de manière dynamique**,

tandis qu'une régularisation par weight-decay à 0.01 a aidé à prévenir le surapprentissage. Les pertes de classification et de localisation ont été pondérées respectivement à 1 et 7.5, afin de bien équilibrer la reconnaissance des classes et la précision des boîtes englobantes. Enfin, l'utilisation du calcul mixte en **FP16** a accéléré l'entraînement, avec un arrêt anticipé activé si aucune amélioration n'était détectée pendant **10 époques**.

Lors de l'entraînement, **YOLO11** intègre automatiquement plusieurs étapes de prétraitement dans son pipeline, notamment la conversion des images en format **RVB**, la mise à l'échelle des pixels dans l'intervalle **[0, 1]** (**normalisation Min-Max**), ainsi que la normalisation avec des valeurs prédéfinies **de moyenne et d'écart-type**. Ces étapes facilitent une meilleure convergence et améliorent la performance globale du modèle.

Implémentation du code d'entraînement basée sur les recommandations officielles de la documentation d'Ultralytics YOLO11:

```
from ultralytics import YOLO
import time

start_time = time.time()

model = YOLO("yolo11n.pt")

model.train(
    data=yaml_path,
    epochs=300,
    imgsz=640,
    batch=20,
    device=0,
    patience=10,
    auto_augment=False,
    augment=True,
    amp=True,
    verbose=True,
    save=True,
    exist_ok=True,
    cos_lr=True,
    plots=True,
    weight_decay=1e-2,
    cache='disk',
    fliplr=0.5, flipud=0.0, hsv_h=0.015,
    hsv_s=0.7, hsv_v=0.4, scale=0.5,
    translate=0.1, shear=0.0, perspective=0.0, degrees=10.0,
    mosaic=1.0,
    mixup=0.0,
    cutmix=0.0,
    copy_paste=0.1,
    copy_paste_mode='flip',
    erasing=0.4,
    close_mosaic=100,
    cls=1,
    box=7.5
)

end_time = time.time()
training_time = end_time - start_time
```

Mesures de performance

Les mesures de performance sont des outils essentiels pour évaluer la précision et l'efficacité des modèles de détection d'objets. Elles mettent en lumière l'efficacité avec laquelle un modèle peut identifier et localiser des objets dans des images. Pour évaluer les performances de notre modèle, nous utilisons comme métrique principale la Précision Moyenne (**mAP**), calculée à un seuil d' $\text{IoU} \geq 0,5$ (notée **mAP@0.5**), afin de mesurer la capacité du modèle à détecter et localiser correctement les objets dans les images.

Mean Average Precision (mAP) [27]

La précision moyenne (**mAP**) est une mesure largement utilisée pour évaluer les performances des modèles de détection d'objets, tels que ceux du célèbre logiciel **Ultralytics YOLO**. Elle fournit un score unique et complet qui résume la capacité d'un modèle à identifier et à localiser correctement des objets dans différentes classes et avec différents niveaux de confiance. Contrairement aux mesures plus simples, le **mAP** équilibre efficacement le compromis entre la recherche de tous les objets pertinents (**rappel**) et la garantie que les objets trouvés sont effectivement corrects (**précision**), ce qui le rend crucial pour l'évaluation des modèles utilisés dans des applications complexes telles que les systèmes autonomes et les diagnostics médicaux. La compréhension de **mAP** est essentielle pour toute personne impliquée dans le développement ou le déploiement de modèles de vision par ordinateur.

Pour comprendre le mAP, il est essentiel de saisir d'abord la précision et le rappel. Dans le contexte de la détection d'objets :

Précision : Mesure le nombre d'objets identifiés qui sont effectivement corrects. Une précision élevée signifie que le modèle fait peu de prédictions faussement positives.

Rappel : Mesure le nombre d'objets présents correctement identifiés par le modèle. Un rappel élevé signifie que le modèle trouve la plupart des objets pertinents (peu de faux négatifs)

Ces deux mesures ont souvent une relation inverse ; améliorer l'une peut parfois diminuer l'autre. mAP permet d'évaluer le modèle sur différents points de ce compromis, offrant ainsi une vue plus équilibrée de la performance.

Comment la mAP est-elle calculée ?

Le calcul de mAP comporte plusieurs étapes, ce qui permet une évaluation solide des performances d'un modèle pour différentes classes d'objets et différents seuils de détection :

- **Tri de confiance** : Pour chaque classe d'objets, les prédictions du modèle (boîtes englobantes détectées) sont triées en fonction de leur score de confiance, du plus élevé au plus bas.
- **Courbe de précision/rappel** : Une courbe de précision-rappel est générée pour chaque classe. Cette courbe représente la précision par rapport aux valeurs de rappel calculées à différents seuils de confiance. À mesure que le seuil diminue, le rappel augmente généralement (plus d'objets sont détectés), tandis que la précision peut diminuer (plus de faux positifs peuvent être inclus).
- **Précision moyenne (AP)** : La précision moyenne (AP) pour une classe unique est calculée comme la surface sous sa courbe de précision-rappel. Elle fournit un chiffre unique qui résume les performances du modèle pour cette classe spécifique à travers tous les niveaux de rappel. Il existe différentes méthodes pour calculer approximativement cette surface.
- **Précision moyenne (mAP)** : Enfin, la mAP est calculée en faisant la moyenne des valeurs AP sur toutes les classes d'objets de l'ensemble de données. On obtient ainsi un score de performance global pour le modèle sur l'ensemble du jeu de données.

Souvent, le mAP est rapporté à des seuils spécifiques d'intersection sur l'union (IoU). L'IoU mesure le chevauchement entre la boîte de délimitation prédite et la boîte de délimitation de la vérité de terrain (réelle). Les variantes courantes de mAP comprennent :

mAP@0.5 (ou mAP50) : Calculé en utilisant un seuil fixe de 0,5 pour l'IoU. Une détection est considérée comme correcte uniquement si son IoU avec la vérité de terrain est de 0,5 ou plus.

mAP@0.5:0.95 (ou mAP95) : Calculé en faisant la moyenne du mAP sur plusieurs seuils d'IoU, généralement de 0,5 à 0,95 avec un pas de **0,05**. Cette mesure, couramment utilisée dans l'évaluation de l'ensemble de données COCO, fournit une évaluation plus complète en

prenant en compte les performances en fonction de différentes exigences de précision de localisation.

3.5 Validation du modèle

Une fois le modèle entraîné, nous avons procédé à sa validation sur l'ensemble de validation défini dans le fichier **data.yaml**. Cette étape permet d'évaluer les performances du modèle sur des données qu'il n'a jamais vues, et ainsi d'avoir une estimation réaliste de sa capacité à généraliser.

La validation est automatiquement réalisée à la fin de chaque époque d'entraînement, mais nous avons également lancé une validation manuelle après l'entraînement pour sauvegarder les résultats et générer les métriques et graphiques nécessaires. **YOLO11n** évalue chaque classe à l'aide des métriques standards en détection d'objets telles que **la précision (P)**, **le rappel (R)**, **la mAP@0.5** et **la mAP@0.5:0.95**.

Code d'implémentation basée sur les recommandations officielles de la documentation d'Ultralytics YOLO11 pour la validation :

```
model_path = 'runs/detect/train/weights/best.pt'

model = YOLO(model_path)

metrics = model.val(
    data=yaml_path,
    save=True,
    save_txt=True,
    save_conf=True,
    save_hybrid=True,
    plots=True
)

print("\n Évaluation terminée avec les paramètres par
défaut :")
for k, v in metrics.results_dict.items():
    print(f"{k}: {v:.4f}")
```

3.6 Évaluation finale sur l'ensemble de test

Après avoir validé le modèle sur les données de validation, nous avons procédé à l'évaluation finale sur l'ensemble test. Cette étape permet d'obtenir une mesure fiable et

définitive des performances réelles du modèle, sur des données totalement inconnues lors de l'entraînement et de la validation.

Nous utilisons encore une fois la méthode `model.val()` avec l'argument `split="test"` pour indiquer à **YOLO11n** de tester le modèle sur l'ensemble spécifié. Les résultats sont enregistrés, les images annotées sauvegardées, et plusieurs visualisations sont automatiquement générées (**métriques, matrice de confusion, courbes de précision/rappel**).

Code d'implémentation basée sur les recommandations officielles de la documentation d'Ultralytics YOLO11 pour le test :

```
model_path = 'runs/detect/train/weights/best.pt'
model_path = 'runs/detect/train/weights/best.pt'

model = YOLO(model_path)

metrics = model.val(
    data=yaml_path,
    split="test",
    save=True,
    save_txt=True,
    save_conf=True,
    save_hybrid=True,
    plots=True
)

print("\n Évaluation terminée avec les paramètres par
défaut :")
for k, v in metrics.results_dict.items():
    print(f"{k}: {v:.4f}")
```

3.7 Inférence

L'inférence est le processus par lequel un modèle préalablement entraîné est utilisé pour faire des prédictions sur de nouvelles données.

Dans le contexte de la détection d'objets appliquée à la reconnaissance des maladies du maïs, cela consiste à soumettre une image au modèle pour obtenir, en sortie, les

emplacements des objets détectés (sous forme de boîtes englobantes), leurs classes prédictives

(**fall-armyworm-larva**, **maize-streak-disease**, **healthy-maize**) ainsi que leurs scores de confiance. Cette opération est effectuée à l'aide de la commande : `model(img_path, iou=0.5, augment=True, imgsz=640)`. Cette commande encapsule plusieurs étapes de post-traitement automatique, dont le redimensionnement de l'image à une taille standard de **640×640 pixels**, l'application d'augmentations légères pour renforcer la robustesse du modèle, et la suppression non maximale (**NMS**) qui élimine les doublons de détection en conservant uniquement les boîtes les plus pertinentes.

Un point souvent négligé mais pourtant essentiel durant l'inférence est la gestion de l'orientation des images. En effet, certaines images, notamment celles issues de smartphones ou d'appareils photo numériques, possèdent des métadonnées **EXIF (Exchangeable Image File Format)** indiquant une orientation spécifique (par exemple, une rotation de **90°** , **180°**). Si cette orientation n'est pas correctement interprétée avant la détection, les objets peuvent apparaître correctement dans l'image, mais les boîtes de détection seront mal alignées avec les cibles réelles. Pour démontrer ce problème, deux approches ont été comparées. La première affiche les résultats sans tenir compte des balises EXIF : les boîtes sont alors décalées ou mal orientées. La seconde utilise une fonction de chargement des images qui lit les métadonnées EXIF, identifie l'orientation d'origine, et applique la rotation nécessaire avant de soumettre l'image au modèle, garantissant ainsi une détection précise et correctement positionnée.

```
from ultralytics import YOLO
import matplotlib.pyplot as plt
from PIL import Image, ExifTags
import numpy as np
import os

MODEL_PATH =
'/kaggle/input/best_pest_fall_army_streak_diseases/pytorch/default/
1/best.pt'
model = YOLO(MODEL_PATH)
```



```
def load_image_with_orientation(path):
    img = Image.open(path).convert("RGB")
    try:
        exif = img.getexif()
        if exif:
            orientation_tag = next(k for k, v in
ExifTags.TAGS.items() if v == 'Orientation')
            orientation = exif.get(orientation_tag, 1)
            if orientation == 3:
                img = img.rotate(180, expand=True)
            elif orientation == 6:
                img = img.rotate(270, expand=True)
            elif orientation == 8:
                img = img.rotate(90, expand=True)
    except Exception as e:
        print(f"EXIF non traitée pour {path}: {e}")
    return img

CLASS_COLORS = {
    'fall-armyworm-larva': 'red',
    'healthy-maize': 'blue',
    'maize-streak-disease': 'orange'
}

def draw_boxes(image_path, boxes, classes, confidences,
class_names):
    img = load_image_with_orientation(image_path)
    plt.figure(figsize=(10, 10))
    plt.imshow(img)
    ax = plt.gca()
    for box, cls, conf in zip(boxes, classes, confidences):
        x1, y1, x2, y2 = box
        class_name = class_names[int(cls)]
        color = CLASS_COLORS.get(class_name, 'green')
        label = f"{class_name} {conf:.2f}"
        rect = plt.Rectangle((x1, y1), x2 - x1, y2 - y1, fill=False,
color=color, linewidth=2)
        ax.add_patch(rect)
        ax.text(x1, y1 - 5, label, color=color, fontsize=12,
weight='bold')
    plt.axis('off')
    plt.show()
```

```

def show_side_by_side(original_path, boxes, classes, confidences,
class_names):
    original_img = load_image_with_orientation(original_path)
    annotated_img = np.array(original_img).copy()
    fig, axs = plt.subplots(1, 2, figsize=(15, 7))
    axs[0].imshow(original_img)
    axs[0].set_title("Image Originale")
    axs[0].axis('off')
    axs[1].imshow(annotated_img)
    ax = axs[1]
    for box, cls, conf in zip(boxes, classes, confidences):
        x1, y1, x2, y2 = box
        class_name = class_names[int(cls)]
        color = CLASS_COLORS.get(class_name, 'green')
        rect = plt.Rectangle((x1, y1), x2 - x1, y2 - y1, fill=False,
color=color, linewidth=2)
        ax.add_patch(rect)
        label = f"{class_name} {conf:.2f}"
        ax.text(x1, y1 - 5, label, color=color, fontsize=12,
weight='bold')
    axs[1].set_title("Image avec détections (Orientation Exif)")
    axs[1].axis('off')
    plt.show()

# Exemple d'utilisation
image_file = image_files[99]
img_path = os.path.join(test_dir_path, image_file)
results = model(img_path, iou=0.5, augment=True, imsz=640)
boxes = results[0].boxes.xyxy.tolist() if results[0].boxes else []
classes = results[0].boxes.cls.tolist() if results[0].boxes else []
confidences = results[0].boxes.conf.tolist() if results[0].boxes else []
names = results[0].names

show_side_by_side(img_path, boxes, classes, confidences, names)

```

3.8 Intégration de l'explicabilité (XAI) avec EigenCAM

Qu'est-ce que l'IA explicable (XAI) ?

L'Intelligence Artificielle Explicable (XAI) vise à rendre les décisions d'un modèle d'apprentissage automatique interprétables et compréhensibles pour les humains. Dans le contexte de la détection d'objets, cela signifie visualiser les régions de l'image qui ont

influencé la prédiction du modèle, afin de s'assurer qu'il se base sur des éléments réellement pertinents (et non sur des artefacts ou des biais visuels). [28]

L'explicabilité joue un rôle essentiel pour instaurer la confiance, garantir la responsabilité et répondre aux exigences éthiques ou réglementaires en matière d'IA. En rendant les décisions plus lisibles, **XAI** coche également le principe fondamental de transparence, un pilier clé dans le développement d'une intelligence artificielle fiable, compréhensible et auditable par les parties prenantes humaines.

Dans ce contexte, nous avons choisi d'utiliser **EigenCAM** comme méthode d'explicabilité. Ce choix repose sur plusieurs facteurs : **EigenCAM** permet une visualisation fine et claire des zones d'intérêt activées par le modèle.

Qu'est-ce qu'EigenCAM ?

EigenCAM (Eigen Class Activation Mapping) est une méthode de visualisation de type CAM (**Class Activation Mapping**), tout comme Grad-CAM, Score-CAM ou encore LayerCAM. Cependant, sa particularité réside dans le fait qu'elle ne dépend pas des gradients pour générer ses cartes d'activation. À la place, **EigenCAM** utilise les vecteurs propres (**eigenvectors**) issus d'une analyse en composantes principales (**PCA**) sur les cartes d'activation des couches du réseau.

La **PCA (Analyse en Composantes Principales)** est une méthode statistique qui sert à :

- Réduire la dimensionnalité d'un grand ensemble de données.
- Trouver les directions (axes) dans lesquelles les données varient le plus.
- Ces directions sont appelées composantes principales, et sont représentées mathématiquement par des vecteurs propres (**eigenvectors**).

Notre approche se distingue par la création de cartes de chaleur, appelées **CAM (Class Activation Maps)**, à partir de trois couches spécifiques du modèle, situées aux **indices 16, 19 et 22**. Ces couches représentent différents niveaux d'abstraction : **la couche 16 met en lumière des détails fins et locaux (niveau petit), la couche 19 capture des informations plus globales et contextuelles (niveau moyen), et la couche 22 rassemble des représentations encore plus larges et abstraites (niveau grand)**. Grâce à la méthode **EigenCAM**, on calcule ces cartes en évaluant quelles zones des couches sélectionnées sont les plus activées, c'est-à-dire où le modèle concentre son attention pour faire ses prédictions.

Ces heatmaps sont ensuite superposées sur l'image originale, ce qui permet de visualiser précisément l'influence spatiale des différentes régions sur la décision du modèle. Pour chaque image traitée, nous sauvegardons non seulement l'image originale et celle annotée avec les détections, mais aussi toutes les heatmaps générées, organisées dans un dossier dédié. Enfin, nous présentons une visualisation regroupant l'image originale, l'image détectée, puis les différentes heatmaps, afin de faciliter l'analyse qualitative du fonctionnement interne du modèle à plusieurs échelles. Cette approche multi-échelle et visuelle rend les prédictions de notre modèle plus transparentes, offrant ainsi une interprétabilité fine et une meilleure compréhension de son comportement.

Code de visualisation multi-échelle avec YOLO et EigenCAM [29]

```
def doMultiScaleCAM(modelPath, imagePath, image_id,
modelIndexes=[16, 19, 22]):
    model = YOLO(modelPath).cpu()

    img = cv2.imread(imagePath)
    img = cv2.resize(img, (1024, 1024))
    rgb_img = img.copy()
    img_float = np.float32(img) / 255

    layer_names = {16: "small", 19: "medium", 22: "large"}
    output_dir = f"outputs/{image_id}"

    original_rgb = cv2.cvtColor(rgb_img, cv2.COLOR_BGR2RGB)
    original_pil = Image.fromarray(original_rgb)

    original_pil.save(f"{output_dir}/{image_id}_original.png")
    print(f" Image originale sauvegardée")

    result = model(rgb_img)[0]
    detection_img = result.plot()
    detection_rgb = cv2.cvtColor(detection_img,
cv2.COLOR_BGR2RGB)
    detection_pil = Image.fromarray(detection_rgb)

    detection_pil.save(f"{output_dir}/{image_id}_detection.png")
    print(f" Image avec détection sauvegardée")

    cam_images = []
    for i in modelIndexes:
        target_layers = [model.model.model[i]]
        cam = EigenCAM(model, target_layers)
        grayscale_cam = cam(rgb_img)[0]
        cam_image = show_cam_on_image(img_float,
grayscale_cam, use_rgb=True)
        cam_path =
f"{output_dir}/{image_id}_CAM_{layer_names.get(i,
f'layer_{i}')}.png"
        plt.imsave(cam_path, cam_image)
        print(f" CAM {layer_names.get(i)} sauvegardée")
        cam_images.append((layer_names.get(i), cam_image))
```

```

plt.figure(figsize=(10, 20))
ax1 = plt.subplot(1 + len(cam_images), 1, 1)
combined_top = np.hstack((original_rgb, detection_rgb))
ax1.imshow(combined_top)
ax1.axis('off')
ax1.set_title(f"{image_id} - Original + Détection")

for idx, (layer_name, cam_img) in enumerate(cam_images):
    ax = plt.subplot(1 + len(cam_images), 1, idx + 2)
    ax.imshow(cam_img)
    ax.axis('off')
    ax.set_title(f"{image_id} - Heatmap CAM :
{layer_name}")

plt.tight_layout()

image_ids = ["ID_7474873e", "ID_713b62a5", "ID_1972cd66",
"ID_9695cfc8", "ID_20672ec7"]
base_path = "/kaggle/input/computer-vision-corn-
pest/COMPUTER_VISION_CORN_PEST/images/test"
model_path =
"/kaggle/input/best_pest_fall_army_streak_diseases/pytorch/de
fault/1/best.pt"

for image_id in image_ids:
    image_path = f"{base_path}/{image_id}.jpg"
    print(f"\n🔍 Traitement de l'image : {image_id}")
    doMultiScaleCAM(model_path, image_path, image_id)

```

3.9 Exportation du modèle au format TensorFlow Lite pour une intégration mobile optimisée

Le déploiement de modèles de vision par ordinateur sur des appareils périphériques ou embarqués nécessite un format qui garantisse des performances sans faille. Le format d'exportation **TensorFlow Lite** ou **TFLite** nous permet d'optimiser nos modèles pour des tâches telles que la détection d'objets dans des applications basées sur des périphériques.

Pourquoi exporter vers TFLite ?

Présenté par Google en mai 2017 dans le cadre de son framework **TensorFlow Lite**, ou **TFLite** en abrégé, est un framework d'apprentissage profond open-source conçu pour l'inférence sur appareil, également connu sous le nom d'edge computing. Il donne aux développeurs les outils nécessaires pour exécuter leurs modèles formés sur des appareils mobiles, embarqués et IoT, ainsi que sur des ordinateurs traditionnels.

TensorFlow Lite est compatible avec une large gamme de plates-formes, y compris **Linux embarqué, Android, iOS, et MCU (Microcontroller Unit)**. L'exportation de notre modèle vers TFLite rend nos applications plus rapides, plus fiables et capables de fonctionner hors ligne. [23]

Pipeline de conversion d'un modèle PyTorch YOLO vers TensorFlow Lite

Notre modèle entraîné, nommé `best.pt`, est un modèle **PyTorch**. Pour pouvoir l'exporter au format **TensorFlow Lite**, il faut suivre un pipeline de conversion en plusieurs étapes, car **TFLite** ne lit pas directement les modèles **PyTorch**.

Lorsque l'on exécute la commande `model.export(format="tflite")` dans **Ultralytics**, c'est **TensorFlow Lite** qui prend en charge automatiquement les étapes suivantes pour convertir le modèle **PyTorch** entraîné (`.pt`) vers le format **TFLite** (`.tflite`) :

Étape 1 : PyTorch (`.pt`) → ONNX (`.onnx`)

Le modèle **YOLO**, initialement au format **PyTorch** (`.pt`), est converti au format **ONNX** (Open Neural Network Exchange). **ONNX** sert de format intermédiaire universel, facilitant l'interopérabilité entre différents frameworks d'apprentissage profond.

Étape 2 : ONNX → TensorFlow SavedModel

Une fois le modèle en **ONNX**, il est converti en **TensorFlow SavedModel**, le format standard de **TensorFlow**. Ce format se présente sous la forme d'un dossier contenant :

`saved_model.pb` : la structure du modèle,

un dossier `variables/` contenant les poids et paramètres du modèle.

Étape 3 : TensorFlow SavedModel → TensorFlow Lite (`.tflite`)

Enfin, le modèle **TensorFlow** est transformé en fichier `.tflite`, un format léger et optimisé conçu pour exécuter efficacement les modèles sur des appareils mobiles, embarqués,

ou de faible puissance.

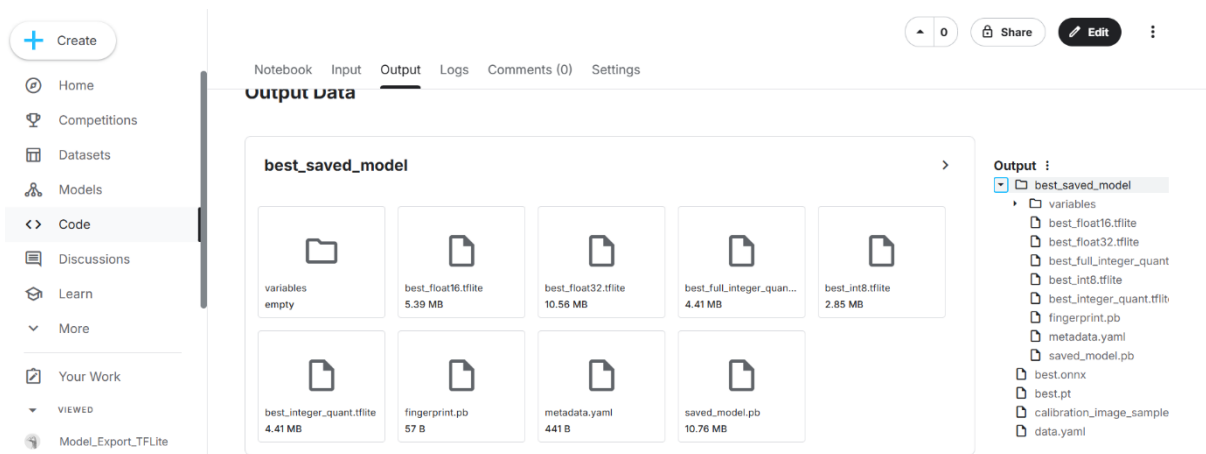


Figure 35: Visualisation des modèles exportés au format TensorFlow Lite : Float32, Float16 et INT8

3.10 Déploiement du modèle

Dans un premier temps, le modèle a été déployé à l'aide de **Streamlit**, une interface web légère, afin de **simuler son efficacité** sur des images de test. Cette solution a permis de visualiser les prédictions, d'évaluer la robustesse du modèle et de proposer une démonstration accessible aux utilisateurs non techniques.

Une **deuxième approche**, envisagée comme une **perspective d'évolution**, consiste à **intégrer le modèle dans une application mobile ou dans des dispositifs embarqués intelligents** tels que des **drones agricoles** ou **robots de terrain**. L'objectif est de permettre une **surveillance autonome et en temps réel des cultures**, directement sur le terrain, sans dépendance à une connexion Internet. Grâce à l'exportation du modèle en formats optimisés comme **TensorFlow Lite (Float32, Float16, INT8)**, cette intégration devient techniquement possible sur des appareils à faibles ressources. De plus, cette intégration pourrait être couplée à des **pulvérisateurs intelligents**, capables d'intervenir uniquement sur les zones affectées, rendant les traitements **plus ciblés, économiques et respectueux de l'environnement**. Ce scénario ouvre la voie à une **agriculture de précision**, où la **détection, le diagnostic et l'intervention sont automatisés et coordonnés**, renforçant ainsi la résilience des systèmes agricoles face aux maladies.

Chapitre 3 : Résultats et discussions

Chapitre 3 : Résultats et discussions

Les chapitres précédents nous ont permis de réaliser un état de l’art complet ainsi que de présenter les outils et les méthodes utilisés pour concevoir et développer notre solution. Dans ce chapitre, nous exposons les résultats obtenus, en nous appuyant sur les fondations posées dans les chapitres précédents. Dans le chapitre précédent, nous avons présenté en détail l’entraînement de notre modèle, notamment la préparation du jeu de données, les techniques d’augmentation, ainsi que les réglages spécifiques utilisés pour optimiser les performances. Nous avons également expliqué le recours à l’apprentissage par transfert et au fine-tuning pour adapter un modèle pré-entraîné à notre problématique spécifique : la détection des maladies et des ravageurs affectant le maïs, en particulier la maladie de la striure du maïs (**maize-streak-disease**) et la chenille légionnaire d’automne (**fall-armyworm-larva**), ainsi que la classe de référence représentant les plants sains (**healthy-maize**).

Dans ce chapitre, nous exposons les résultats obtenus à partir de ce cadre expérimental. Nous présenterons les performances du modèle sur les différentes classes cibles, en analysant les métriques clés et en discutant les points forts ainsi que les limites observées.

Section 1 : Résultats sur le pipeline d’entraînement avec évaluation continue sur le jeu de validation

1.1 Métriques de performances : Précision, Rappel, mAP@0.5 et mAP@0.5-0.95

Pour suivre l’évolution de l’apprentissage de notre modèle, un enregistrement automatique des performances à chaque époque a été réalisé dans un fichier `results.csv`. Ce fichier inclut les principales métriques relatives aux pertes (**loss**) telles que la perte de localisation (**box_loss**), de classification (**cls_loss**) et de distribution (**dfl_loss**) ainsi que les indicateurs de performance comme la **précision**, le **rappel**, et les **mAP** (**mean Average Precision**) calculés sur le jeu de validation (mAP50, mAP50-95).

Au fil des époques, on observe une diminution progressive et significative des pertes d’apprentissage, accompagnée d’une nette amélioration des performances de détection :

epoch	time	train/box_loss	train/cls_loss	train/dfl_loss	metrics/precision(B)	metrics/recall(B)	metrics/mAP50(B)	metrics/mAP50-95(B)	val/box_loss	val/cls_loss	val/dfl_loss
1	80.8373	2.02989	5.199	2.13909	0.16608	0.32435	0.12959	0.04303	2.69124	inf	3.75204
2	169.746	2.06959	4.66791	2.20285	0.03092	0.03773	0.01165	0.00372	3.18168	24.9819	4.92303
3	259.066	2.06327	4.77966	2.18895	0.45971	0.20223	0.09147	0.02914	2.5332	27.5708	3.00533
4	347.497	2.02387	4.6805	2.15432	0.21592	0.19184	0.10212	0.03409	2.64023	14.8013	2.88753
5	437.154	2.00812	4.4524	2.15298	0.33683	0.35157	0.24982	0.09073	2.23032	16.0655	2.38023
6	525.938	1.96326	4.29576	2.11081	0.44764	0.40066	0.35259	0.13167	2.15687	7.2289	2.40593
7	615.045	1.94163	4.18368	2.1045	0.41108	0.49493	0.39162	0.14665	2.10555	5.13626	2.29399
8	703.612	1.90654	4.16742	2.04867	0.57687	0.5424	0.55365	0.20826	2.02996	4.12631	2.20399
9	793.005	1.88845	3.98095	2.04313	0.48585	0.54304	0.45768	0.17479	2.00858	5.36055	2.23693
10	881.577	1.87387	3.91422	2.03541	0.5507	0.44481	0.48265	0.18054	2.04092	4.15111	2.18842
289	25594.8	0.80625	0.85425	1.21903	0.98597	0.97094	0.99103	0.8271	0.71903	0.72276	1.13455
290	25683.7	0.81581	0.86056	1.22668	0.98537	0.97118	0.99105	0.82587	0.71973	0.7214	1.13462
291	25773.0	0.80604	0.8528	1.21827	0.98534	0.97106	0.99101	0.82612	0.72116	0.72115	1.13539
292	25861.8	0.80269	0.86959	1.22799	0.98581	0.9702	0.99107	0.82559	0.72238	0.72039	1.1363
293	25951.0	0.80737	0.87718	1.22361	0.98684	0.97004	0.99106	0.82584	0.72089	0.72079	1.1354
294	26039.5	0.79796	0.85477	1.20955	0.98611	0.97004	0.9911	0.82442	0.72218	0.72124	1.13567
295	26129.0	0.78893	0.84863	1.20596	0.98592	0.97004	0.99115	0.82547	0.72304	0.72026	1.13628
296	26218.2	0.80946	0.86127	1.21715	0.9859	0.97031	0.99115	0.82632	0.72387	0.72261	1.13716
297	26307.4	0.80037	0.85212	1.20619	0.98617	0.97032	0.99116	0.82528	0.72253	0.72298	1.1361
298	26395.8	0.79576	0.84746	1.20595	0.9858	0.97059	0.99113	0.82613	0.72171	0.72255	1.13542
299	26484.9	0.81184	0.87149	1.22523	0.98569	0.97089	0.99114	0.82753	0.71991	0.72179	1.13404
300	26573.6	0.81679	0.86828	1.2342	0.98571	0.97082	0.99115	0.82805	0.72024	0.72041	1.13425

Figure 36:— Évolution des performances lors des 10 premières et 10 dernières époques de l'entraînement YOLO

Ces résultats traduisent une bonne convergence du modèle, avec une capacité robuste à détecter et classifier les objets ciblés dans les images.

1.2 Analyse des erreurs via les matrices de confusion

Dans le domaine de la détection d'objets, la **matrice de confusion** est un outil analytique fondamental qui permet d'évaluer la performance d'un modèle en visualisant la correspondance entre les classes réelles (vérité terrain) et les classes prédites par le modèle. Contrairement à la classification d'images où une seule étiquette est attribuée à l'image entière, en détection d'objets, le modèle doit non seulement identifier la classe de l'objet, mais aussi localiser précisément chaque instance de cet objet au sein de l'image.

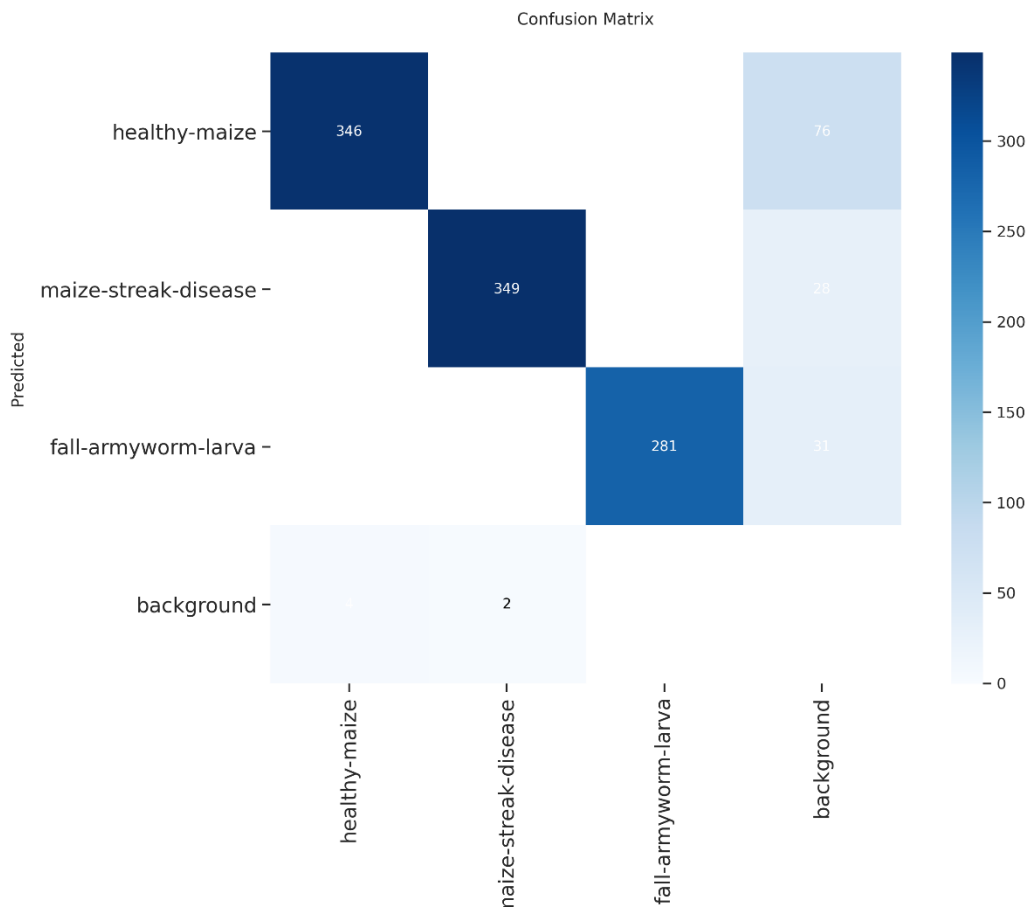


Figure 37: Matrice de confusion des prédictions par classe

Le modèle a correctement détecté la majorité des instances pour chaque classe : **346 pour healthy-maize**, **349 pour maize-streak-disease**, et **281 pour fall-armyworm-larva**. Toutefois, des confusions avec l'arrière-plan subsistent : 76 pour le maïs sain, 28 pour la striure du maïs, et 31 pour la chenille légionnaire. Ces erreurs suggèrent des difficultés à distinguer les éléments visuellement discrets ou mal annotés. À l'inverse, l'arrière-plan a rarement été confondu avec les classes d'intérêt (seulement 6 cas), ce qui indique une bonne capacité du modèle à ignorer les zones non pertinentes.

En somme cette matrice de confusion montre que le modèle détecte efficacement les classes d'intérêt tout en maintenant une bonne sélectivité face à l'arrière-plan, malgré quelques confusions liées à la discrétion visuelle de certaines instances.

1.3 Visualisation des courbes d'évolution des métriques de performance sur le jeu de validation pendant l'entraînement

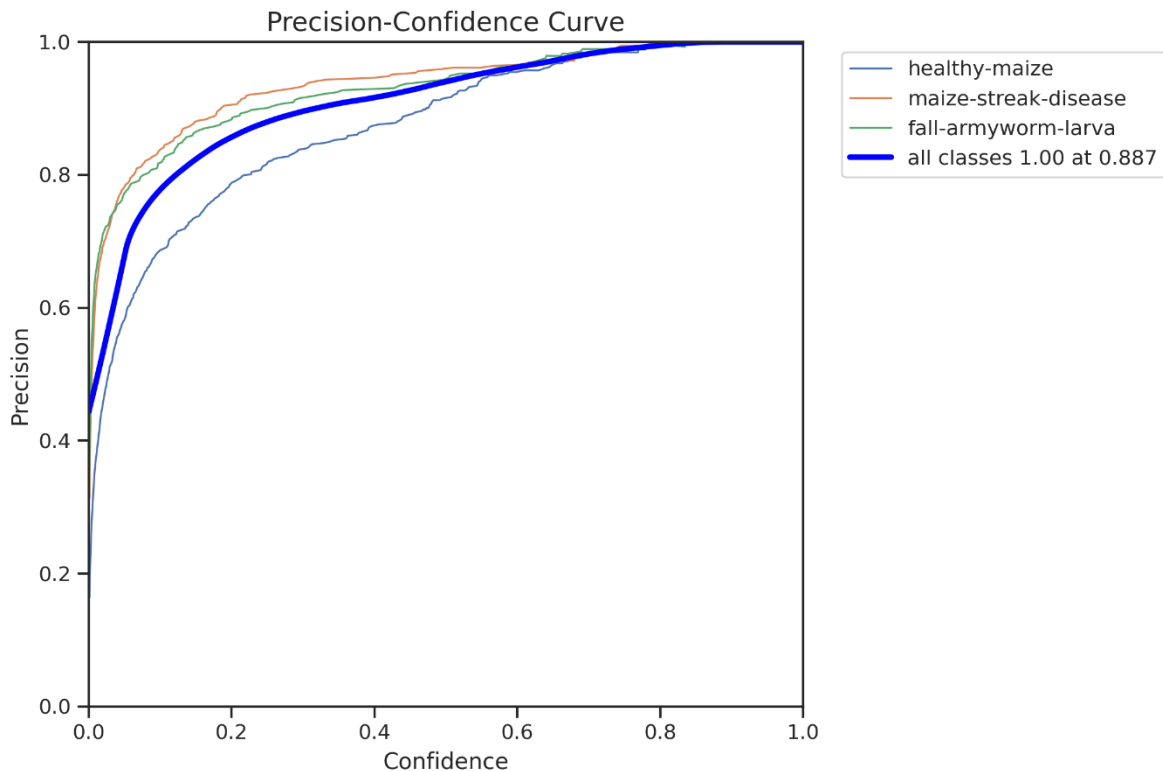


Figure 38: Courbes de précision en fonction du seuil de confiance pour chaque classe

L'image représente la **Courbe de Précision-Confiance** de notre modèle de détection pour les classes de maïs sain, de maladie de la striure du maïs et de chenille légionnaire d'automne. Cette courbe montre comment la précision des détections (c'est-à-dire le pourcentage de détections correctes parmi toutes les détections effectuées) évolue en fonction du seuil de confiance appliqué aux prédictions du modèle

Pour les maladies et ravageurs (**maize-streak-disease et fall-armyworm-larva**) : Les courbes orange et verte sont très élevées, se maintenant proches de **1.0 (100% de précision)** même pour des seuils de confiance relativement bas. Cela indique que le modèle est extrêmement performant pour ces deux classes, générant très peu de faux positifs (détections incorrectes) et identifiant avec une grande fiabilité les maladies et les ravageurs lorsqu'il les

détecte.

Pour le **maïs sain (healthy-maize)** : La courbe bleu clair est légèrement en dessous des autres, particulièrement à des seuils de confiance plus faibles. Cela suggère que le modèle peut avoir un peu plus de faux positifs pour cette classe à ces niveaux de confiance, mais elle atteint également une précision parfaite (1.0) à des seuils de confiance plus élevés (autour de 0.8 et plus).

Performance globale (**all classes**) : La ligne bleue épaisse, représentant la moyenne de toutes les classes, atteint **une précision de 1.0 (100%)** à un seuil de **confiance de 0.887**. Ce point est particulièrement significatif : il signifie que si l'on ne retient que les détections dont le modèle est confiant à **88.7%** ou plus, alors toutes nos détections sont correctes, sans aucun faux positif.

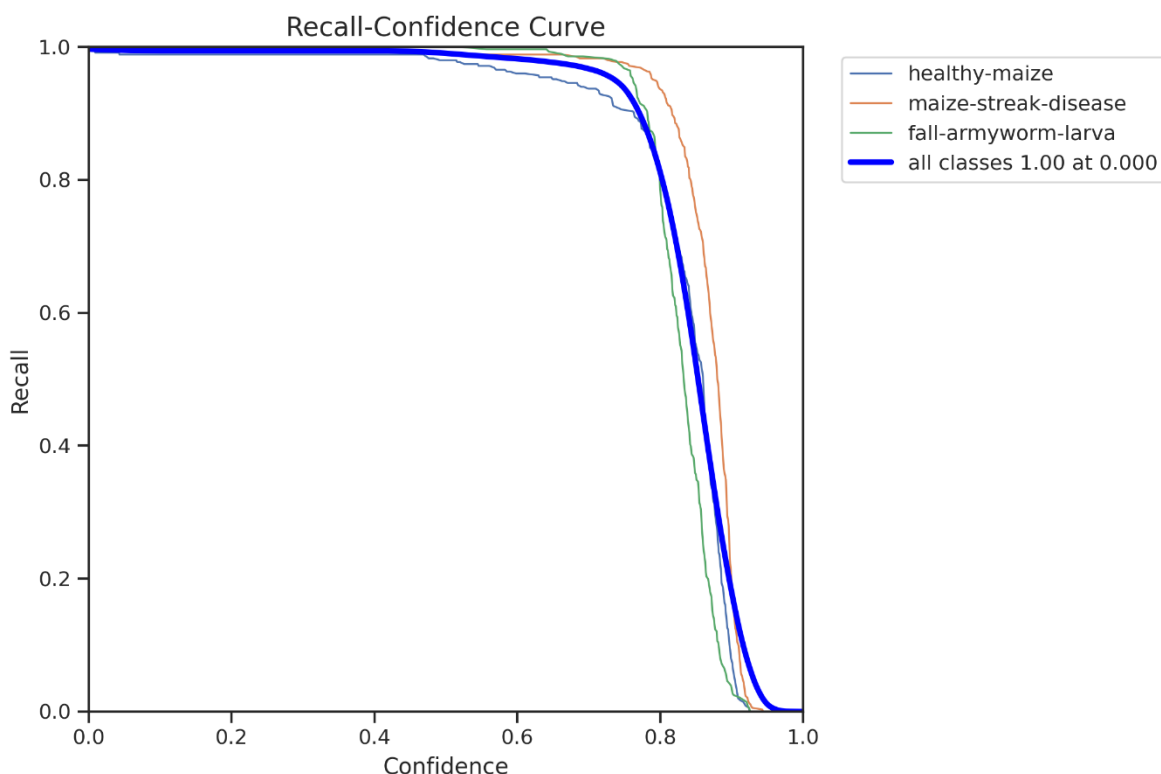


Figure 39: Courbes de rappel en fonction du seuil de confiance pour chaque classe

L'image représente la **Courbe de Rappel-Confiance** de notre modèle de détection pour les classes **de maïs sain, de maladie de la striure du maïs et de chenille légionnaire**. Cette courbe montre comment le rappel des détections (**c'est-à-dire la proportion de toutes les instances réelles présentes dans les images qui ont été correctement détectées par le modèle**) évolue en fonction du seuil de confiance appliqué aux prédictions. Un rappel élevé est fondamental pour s'assurer qu'aucune maladie ou ravageur ne soit manqué sur le terrain.

Pour les maladies et ravageurs (**maize-streak-disease et fall-armyworm-larva**) : Les courbes orange et verte se maintiennent à un niveau de rappel très élevé (**proche de 1.0 ou 100%**) sur une large plage de seuils de confiance, jusqu'à environ **0.8-0.85**. Cela indique que le modèle est très efficace pour détecter la quasi-totalité des instances réelles de ces maladies et ravageurs présentes sur le terrain, générant très peu **de faux négatifs (cas où un objet réel est présent mais non détecté par le modèle)**.

Pour le **maïs sain (healthy-maize)** : La courbe bleu clair est légèrement en dessous des autres et commence à chuter un peu plus tôt à mesure que le seuil de confiance augmente (**autour de 0.6-0.7**). Cela suggère que, pour cette classe, le modèle peut manquer un peu plus d'instances réelles de maïs sain (générer plus de faux négatifs) à des seuils de confiance plus élevés, comparé aux classes de maladies/ravageurs. Cette observation est cohérente avec les faux négatifs observés dans la matrice de confusion normalisée, où une part significative de "**healthy-maize**" était classée comme "**background**" (**non détectée**).

Performance globale (all classes) : La ligne bleue épaisse, représentant la moyenne de toutes les classes, démarre à **un rappel de 1.0 à un seuil de confiance de 0** (ce qui est attendu, car toutes les détections, même incertaines, sont acceptées à un seuil nul). La courbe montre une diminution progressive du rappel à mesure que le seuil de confiance est augmenté. Une baisse plus marquée est observée au-delà **d'environ 0.85**, signifiant que pour garantir un très haut niveau de confiance dans les détections (haute précision), le modèle est amené à ignorer certaines instances réelles qui auraient été détectées avec une confiance plus faible, augmentant ainsi le nombre de faux négatifs.

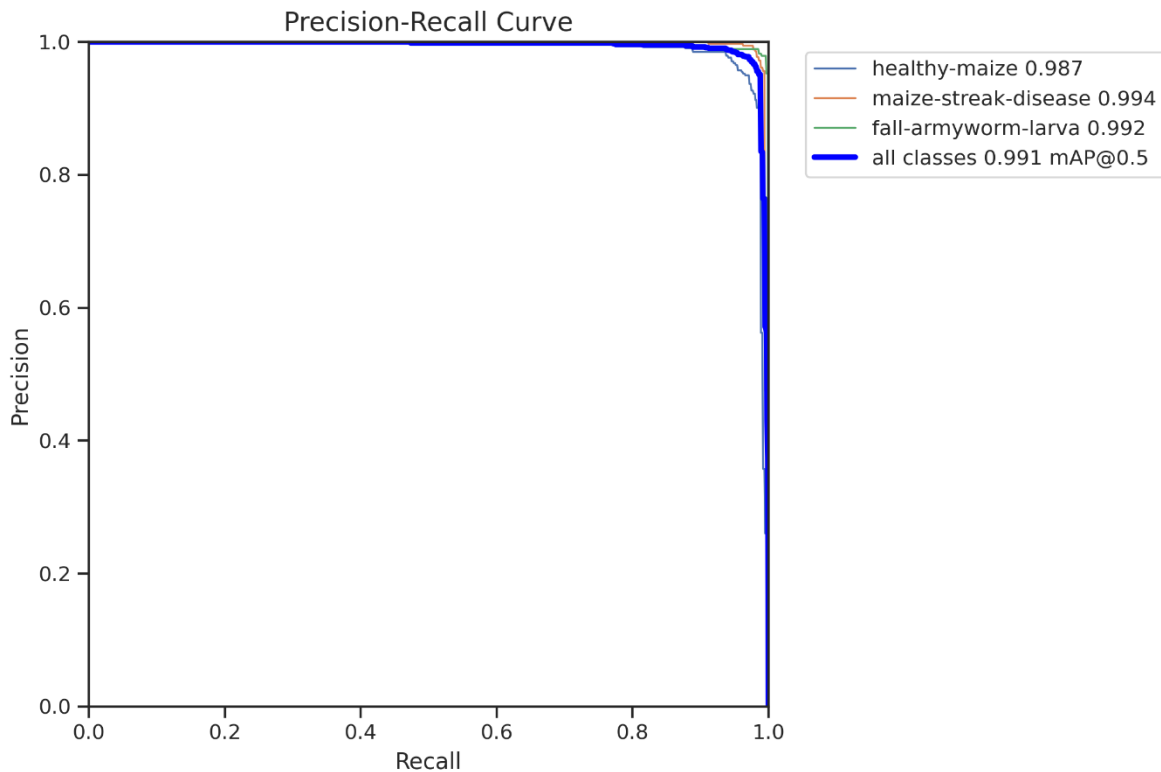


Figure 40: Courbes de Précision Rappel

L'image représente la **Courbe de Précision-Rappel** de notre modèle de détection pour les **classes de maïs sain, de maladie de la striure du maïs et de chenille légionnaire d'automne**. Cette courbe est une métrique de performance fondamentale qui illustre le compromis entre la précision et le rappel du modèle à différents seuils de confiance. Chaque point sur la courbe représente un couple (rappel, précision) obtenu en faisant varier le seuil de confiance des détections. **Plus la courbe est proche du coin supérieur droit du graphique (c'est-à-dire une aire sous la courbe élevée), meilleure est la performance globale du modèle.**

Pour les maladies et ravageurs (maize-streak-disease et fall-armyworm-larva) : Les courbes orange et verte sont très élevées et restent très proches de **1.0** en précision même lorsque le rappel est élevé. L'aire sous ces courbes est presque maximale (**mAP@0.5 de 0.994**

et 0.992 respectivement), ce qui indique une performance exceptionnelle. Cela signifie que le modèle est capable de détecter presque toutes les instances réelles de ces classes (haut rappel) tout en minimisant les faux positifs (haute précision), ce qui est crucial pour identifier efficacement les menaces.

***Pour le maïs sain (healthy-maize) :** La courbe bleue est légèrement inférieure aux autres, mais elle reste également à un niveau de performance très **élevé (mAP@0.5 de 0.987)**. Bien qu'il y ait une légère baisse de précision pour un rappel très élevé (ce qui suggère un peu plus de faux positifs ou une difficulté à maintenir la précision lorsqu'il essaie de tout détecter), la performance reste excellente pour cette classe.*

Performance globale (all classes) : La ligne bleue épaisse représente la performance moyenne sur toutes les classes, avec un **mAP@0.5 global de 0.991**. Cette valeur très élevée est un excellent indicateur de la robustesse et de l'efficacité globale de notre modèle pour la détection des maladies, des ravageurs et des plants sains. **Elle confirme la capacité du modèle à atteindre simultanément un haut niveau de précision (peu d'erreurs) et de rappel (peu d'oublis) sur l'ensemble des classes ciblées.**

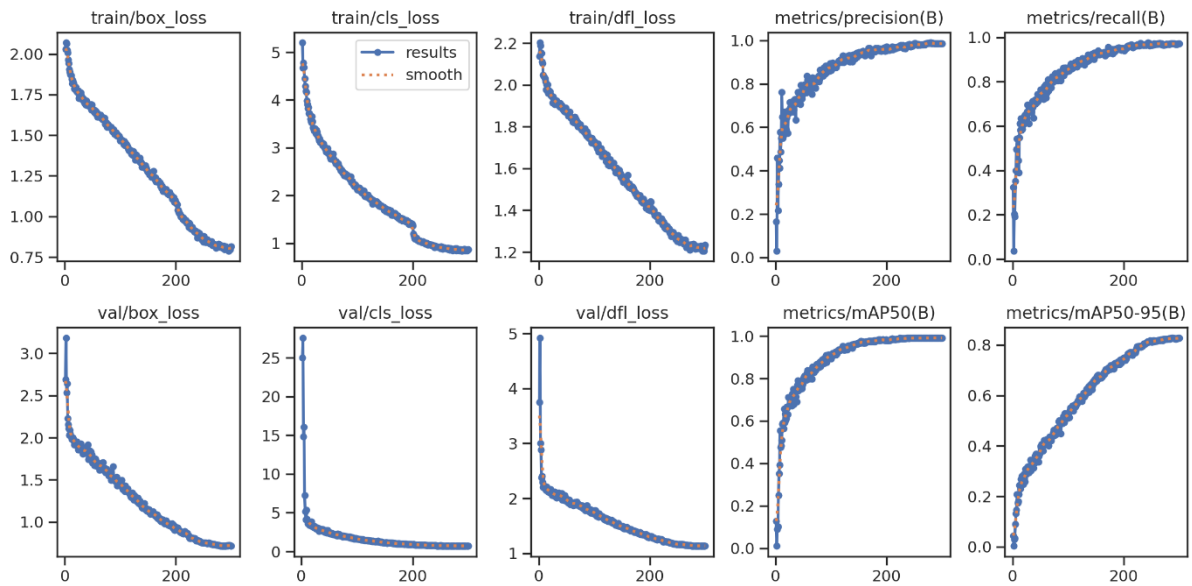


Figure 41: Courbes d'évolution des pertes et des métriques de performance sur les jeux d'entraînement et de validation au cours de l'entraînement de notre modèle.

Les courbes d'évolution des pertes et des métriques de performance présentent un aperçu complet de l'évolution de l'entraînement et de la validation de notre modèle sur **300 époques**. Chaque sous-graphique illustre le comportement d'une métrique clé au fil du temps, permettant d'évaluer la **convergence**, l'**apprentissage** et la **généralisation du modèle**.

La première ligne de l'image présente les métriques de performance calculées sur le jeu d'entraînement (train).

train/box_loss (Perte de la boîte englobante) :

Interprétation : Cette courbe représente la perte liée à la précision de la localisation des objets (position et taille des boîtes englobantes).

Observation : La perte diminue progressivement et significativement de **2.0 à 0.75 sur les 300 époques**.

Conclusion : Le modèle apprend efficacement à localiser de plus en plus précisément les objets dans les images d'entraînement.

train/cls_loss (Perte de classification) :

Interprétation : Cette courbe représente la perte liée à la capacité du modèle à classer correctement les objets détectés.

Observation : La perte de classification diminue également de manière nette de 5.0 à moins de 0.25. On observe une chute plus rapide **après 200 époques**.

Conclusion : Le modèle devient de plus en plus apte à attribuer la bonne classe aux objets identifiés dans le jeu d'entraînement.

train/dfloss (Perte de distribution de la régression des boîtes) :

Interprétation : La perte DFL (Distribution Focal Loss) est une perte plus avancée utilisée dans certains détecteurs d'objets pour optimiser la régression des boîtes en se concentrant sur les distributions des prédictions.

Observation : Elle suit une tendance similaire aux autres pertes, diminuant de 2.0 à 1.2.

Conclusion : Cette perte contribue également à la robustesse et à la précision des prédictions de boîtes.

metrics/precision (Précision sur le jeu d'entraînement) :

Interprétation : Mesure la proportion de détections positives correctes parmi toutes les détections positives effectuées sur le jeu d'entraînement.

Observation : La précision augmente rapidement au début de l'entraînement, atteignant près de 1.0 (100%) et se maintenant à ce niveau élevé après environ 100 époques.

Conclusion : Le modèle est capable de générer des détections très fiables (peu de faux positifs) sur les données avec lesquelles il a été entraîné.

metrics/recall(B) (Rappel sur le jeu d'entraînement) :

Interprétation : *Mesure la proportion de toutes les instances réelles correctement détectées par le modèle sur le jeu d'entraînement.*

Observation : *Le rappel augmente également rapidement, atteignant et maintenant un niveau très élevé (**proche de 1.0**) après environ 100 époques.*

Conclusion : Le modèle est très efficace pour détecter la quasi-totalité des objets d'intérêt dans les données d'entraînement (peu de faux négatifs).

La dernière ligne de l'image présente les métriques de performance calculées sur le jeu de validation(val). Ces courbes sont cruciales car elles évaluent la capacité de généralisation du modèle à des données qu'il n'a jamais vues auparavant.

val/box_loss (Perte de la boîte englobante sur la validation) :

Interprétation : *Indique la précision de la localisation des objets sur les données de validation.*

Observation : *Cette perte diminue de manière stable de **3.0 à moins de 0.75**, reflétant une bonne généralisation.*

Conclusion : Le modèle apprend à localiser précisément les objets non seulement sur les données d'entraînement, mais aussi sur de nouvelles données, ce qui est un signe de bonne généralisation.

val/cls_loss (Perte de classification sur la validation) :

Interprétation : *Indique la capacité du modèle à classer correctement les objets sur les données de validation.*

Observation : *La perte de **classification diminue de 25.0** (valeur initiale très élevée, indiquant un manque de connaissances au début) à **moins de 0.5**. On observe une baisse très nette au début et une stabilisation.*

Conclusion : Le modèle généralise bien sa capacité de classification à des données inconnues.

val/dfl_loss (Perte de distribution de la régression des boîtes sur la validation) :

***Interprétation :** Évalue l'optimisation de la régression des boîtes sur les données de validation.*

***Observation :** Diminue de 5.0 à moins de 0.5.*

***Conclusion :** La stratégie d'optimisation de la localisation est efficace et généralisable.*

metrics/mAP50(B) (mAP@0.5 sur le jeu de validation) :

***Interprétation :** Représente la performance globale du modèle pour la détection d'objets sur le jeu de validation, mesurée par la moyenne des **Average Precisions (AP)** pour toutes les classes, avec un seuil IoU de 0.5.*

***Observation :** Cette métrique augmente de manière impressionnante, atteignant et se stabilisant à près de 1.0 (environ 0.99) après 200 époques.*

***Conclusion :** C'est un indicateur très fort de la performance globale du modèle. Un mAP@0.5 de près de 1.0 signifie que le modèle est exceptionnellement bon pour détecter et classer correctement les objets avec une superposition d'au moins 50% sur des données qu'il n'a pas vues pendant l'entraînement.*

metrics/mAP50-95(B) (mAP@0.5-0.95 sur le jeu de validation) :

***Interprétation :** Représente la moyenne des AP calculées sur plusieurs seuils IoU allant de 0.5 à 0.95 (par pas de 0.05). C'est une métrique plus stricte et plus robuste qui évalue la performance du modèle sur une gamme plus large de critères de précision de la localisation.*

***Observation :** Cette métrique augmente de manière constante de 0 à environ 0.85 sur les 300 époques. Bien qu'elle soit inférieure au mAP@0.5 (ce qui est normal car elle est plus exigeante), sa croissance continue et sa valeur finale élevée (0.85) sont très positives.*

Conclusion : Le modèle ne se contente pas de bien localiser les objets avec un IoU de 0.5 ; il est également très précis sur la localisation des boîtes englobantes à des seuils d'IoU plus stricts.

L'ensemble de ces courbes démontre **une convergence réussie et une excellente capacité de généralisation de notre modèle de détection, d'identification et de localisation des ravageurs (comme la chenille légionnaire) et des maladies (comme la maize streak disease) sur les plants de maïs**. Les pertes diminuent régulièrement sur les ensembles d'entraînement et de validation, tandis que les métriques de performance (**précision, rappel, mAP**) augmentent de manière significative. Le fait que les performances sur le jeu de validation soient très proches de celles du jeu d'entraînement, et que le **mAP@0.5** atteigne un niveau quasi-parfait, indique que le modèle n'est pas en surapprentissage et qu'il est robuste pour la détection des maladies et ravageurs du maïs sur de nouvelles données.

Section 2 : Évaluation des performances sur le jeu de test

Cette section évalue la robustesse finale du modèle en analysant ses performances sur le jeu de test. L'objectif est de mesurer sa capacité à généraliser sur des données jamais vues, à travers les mêmes métriques clés utilisées pendant la phase de validation.

2.1 Évaluation finale des performances : Précision, Rappel, mAP@0.5 et mAP@0.5-0.95 sur le jeu de test

Pour évaluer la performance réelle du modèle une fois entraîné, nous avons mesuré ses métriques finales sur le jeu de test, composé d'images totalement inédites (**185 images uniques totalisant 562 instances annotées**). Cette étape permet de juger la capacité du modèle à détecter avec précision et fiabilité les maladies et ravageurs dans des conditions réelles.

Les métriques obtenues incluant **la précision, le rappel, le mAP@0.5 et le mAP@0.5-0.95** offrent un aperçu clair de la généralisation du modèle au-delà des données d'entraînement et de validation.

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):
100%	12/12	[00:08<00:00,	1.37it/s]				
	all	178	555	0.803	0.808	0.822	0.46
	healthy-maize	12	204	0.646	0.652	0.634	0.326
	maize-streak-disease	26	207	0.875	0.899	0.929	0.621
	fall-armyworm-larva	141	144	0.887	0.874	0.903	0.432

Speed: 1.5ms preprocess, 29.7ms inference, 0.0ms loss, 1.2ms postprocess per image
 Results saved to runs/detect/val2

Évaluation terminée avec les paramètres par défaut :
 metrics/precision(B): 0.8029
 metrics/recall(B): 0.8082
 metrics/mAP50(B): 0.8218
 metrics/mAP50-95(B): 0.4598
 fitness: 0.4960

Figure 42: Résultats d'évaluation du modèle sur le jeu de test : métriques globales et par classe (Précision, Rappel, mAP@0.5, mAP@0.5:0.95)

Le modèle détecte efficacement les maladies et ravageurs du maïs, avec un mAP@0.5 de 0.929 pour maize-streak-disease et de 0.903 pour fall-armyworm-larva. Le mAP@0.5–0.95 atteint 0.621 pour maize-streak-disease et 0.432 pour fall-armyworm-larva, témoignant d'une bonne capacité de généralisation sur des seuils d'IoU variés.

2.2 Détection par classe : interprétation de la matrice de confusion normalisée

Dans le domaine de la détection d'objets, la **matrice de confusion normalisée** est un outil analytique fondamental qui permet d'évaluer la performance d'un modèle en visualisant la correspondance entre les classes réelles (vérité terrain) et les classes prédites par le modèle. Contrairement à la classification d'images où une seule étiquette est attribuée à l'image entière, en détection d'objets, le modèle doit non seulement identifier la classe de l'objet, mais aussi localiser précisément chaque instance de cet objet au sein de l'image.

Cette version normalisée présente les proportions (en pourcentage ou en fractions) de chaque catégorie d'instances réelles qui ont été classées dans chaque classe prédite. Cela permet une lecture directe des taux de rappel pour chaque classe (les valeurs sur la diagonale) ainsi que des proportions d'erreurs spécifiques (faux positifs et faux négatifs) par rapport à la distribution réelle des classes.

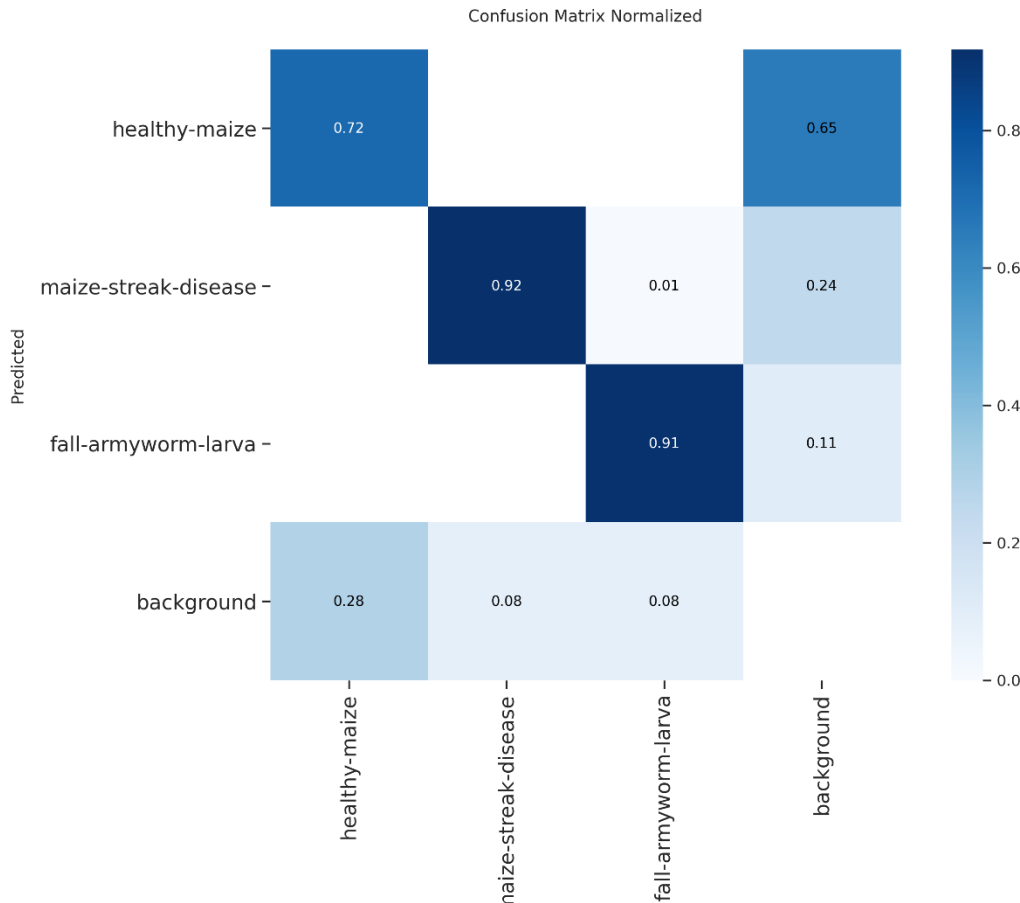


Figure 43: Matrice de confusion normalisée sur le jeu de test

Notre modèle a correctement détecté une forte proportion des instances réelles pour chaque classe : **92% pour maize-streak-disease et 91% pour fall-armyworm-larva**. Toutefois, des confusions significatives avec l'arrière-plan subsistent : **24% de la striure du maïs et 11% de la chenille légionnaire réels ont été manqués par le modèle (classés comme background)**. Ces erreurs suggèrent des difficultés pour le modèle à détecter toutes les instances de ces classes. Inversement, l'arrière-plan a été occasionnellement confondu avec les classes d'intérêt (**28% avec healthy-maize, 8% avec maize-streak-disease, 8% avec fall-armyworm-larva**).

En somme, cette matrice de confusion normalisée confirme l'efficacité du modèle à détecter les classes d'intérêt critiques (**maize-streak-disease** et **fall-armyworm-larva**) sur des données inédites .

2.3 Visualisation des performances finales sur le jeu de test

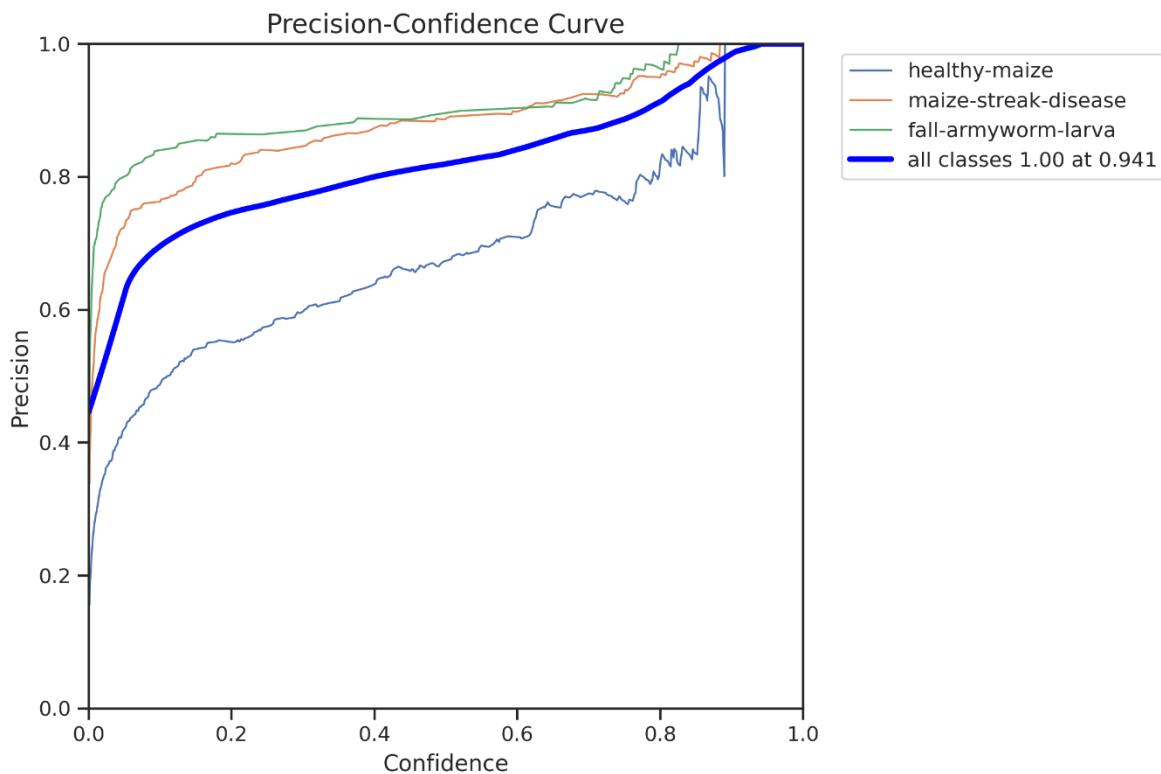


Figure 44:— Courbes de précision en fonction du seuil de confiance pour chaque classe sur le jeu de Test

L'image représente la **Courbe de Précision-Confiance** de notre modèle de détection pour les classes de **maïs sain, de maladie de la striure du maïs et de chenille légionnaire** , sur le jeu de test. Cette courbe montre comment la précision des détections (c'est-à-dire le pourcentage de détections correctes parmi toutes les détections effectuées) évolue en fonction du seuil de confiance appliqué aux prédictions du modèle.

*Pour les maladies et ravageurs (maize-streak-disease et fall-armyworm-larva) : Les courbes orange et verte sont très élevées et se maintiennent à des niveaux de précision supérieurs à **0.8 (80%)** même pour des seuils de confiance relativement bas, et atteignent **1.0** pour des seuils de confiance élevés. Cela indique que le modèle est très performant pour ces deux classes sur des données inédites, générant peu de faux positifs (détections incorrectes) et identifiant avec une grande fiabilité les maladies et les ravageurs lorsqu'il les détecte.*

*Pour le maïs sain (healthy-maize) : La courbe bleu clair est nettement en dessous des autres, particulièrement à des seuils de confiance faibles et modérés, où la précision chute rapidement pour atteindre des niveaux inférieurs à **0.6 (60%)**. Cela suggère que le modèle génère significativement plus de faux positifs pour cette classe sur le jeu de test, ce qui correspond à des détections incorrectes fréquentes de ce qui n'est pas réellement du maïs sain ou à des confusions avec d'autres classes. Elle atteint **1.0 de précision seulement pour des seuils de confiance très élevés (proche de 0.95)**.*

Performance globale (all classes) : La ligne bleue épaisse, représentant la moyenne de toutes les classes, atteint une précision de **1.0 (100%)** à un seuil de confiance de **0.941**. Ce point est significatif : il signifie que si l'on ne retient que les détections dont le modèle est confiant à **94.1%** ou plus, alors toutes nos détections sont parfaitement correctes, sans aucun faux positif. Cependant, ce seuil est plus élevé que celui observé sur le jeu d'entraînement (**0.887**), suggérant une plus grande variabilité des confiances sur le jeu de test.

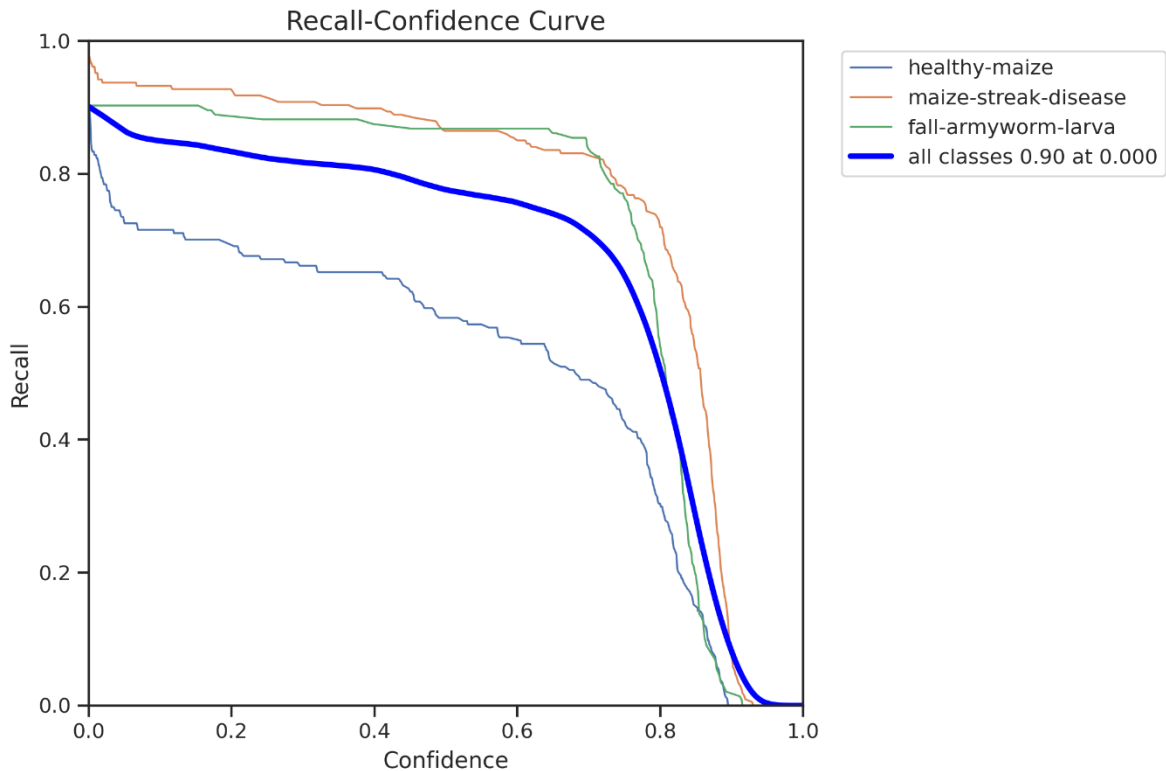


Figure 45: Courbes de rappel en fonction du seuil de confiance pour chaque classe sur le jeu de Test

L'image représente la **Courbe de Rappel-Confiance** de notre modèle de détection pour les classes **de maïs sain, de maladie de la striure du maïs et de chenille légionnaire**, sur le jeu de test. Cette courbe montre comment le rappel des détections (c'est-à-dire le pourcentage de toutes les instances réelles présentes dans les images qui ont été correctement détectées par le modèle) évolue en fonction du seuil de confiance appliqué aux prédictions. Un rappel élevé est fondamental pour s'assurer qu'aucune maladie ou ravageur ne soit manqué sur le terrain.

Pour les maladies et ravageurs (*maize-streak-disease* et *fall-armyworm-larva*) : Les courbes orange et verte se maintiennent à un niveau de rappel très élevé (**proche de 0.9 ou 90%**) pour des seuils de confiance faibles et modérés, et commencent à chuter de manière plus prononcée après **0.8 de confiance**. Cela indique que le modèle est très efficace pour détecter la grande majorité des instances réelles de ces maladies et ravageurs sur des

données inédites, générant peu de faux négatifs (cas où un objet réel est présent mais non détecté par le modèle).

*Pour le maïs sain (healthy-maize) : La courbe bleu clair est nettement en dessous des autres. Son rappel diminue rapidement, atteignant environ **0.65 (65%)** à un seuil de confiance de **0.1** et continuant de chuter ensuite. Cela suggère que, pour cette classe, le modèle manque un pourcentage significatif d'instances réelles de maïs sain sur le jeu de test, générant un nombre plus élevé de faux négatifs comparé aux classes de maladies/ravageurs. Cette observation est cohérente avec les faux négatifs observés dans la matrice de confusion normalisée pour le maïs sain, où une grande partie était classée comme background.*

Performance globale (all classes) : La ligne bleue épaisse, représentant le rappel moyen sur toutes les classes, démarre à **un rappel de 0.90 à un seuil de confiance de 0.000**. La courbe montre une diminution constante du rappel à mesure que le seuil de confiance est augmenté, avec une baisse plus marquée au-delà d'environ **0.75**. Cela signifie que pour garantir un haut niveau de confiance dans les détections (haute précision), le modèle est amené à ignorer davantage d'instances réelles qui auraient été détectées avec une confiance plus faible, augmentant ainsi le nombre de faux négatifs.

En résumé, cette courbe nous montre que notre modèle est très efficace pour détecter la majorité des instances réelles des maladies et ravageurs sur des données inédites. Cependant, le rappel pour le maïs sain est moins performant, indiquant une difficulté à détecter toutes les instances de plants sains, ce qui peut être un point d'amélioration futur.

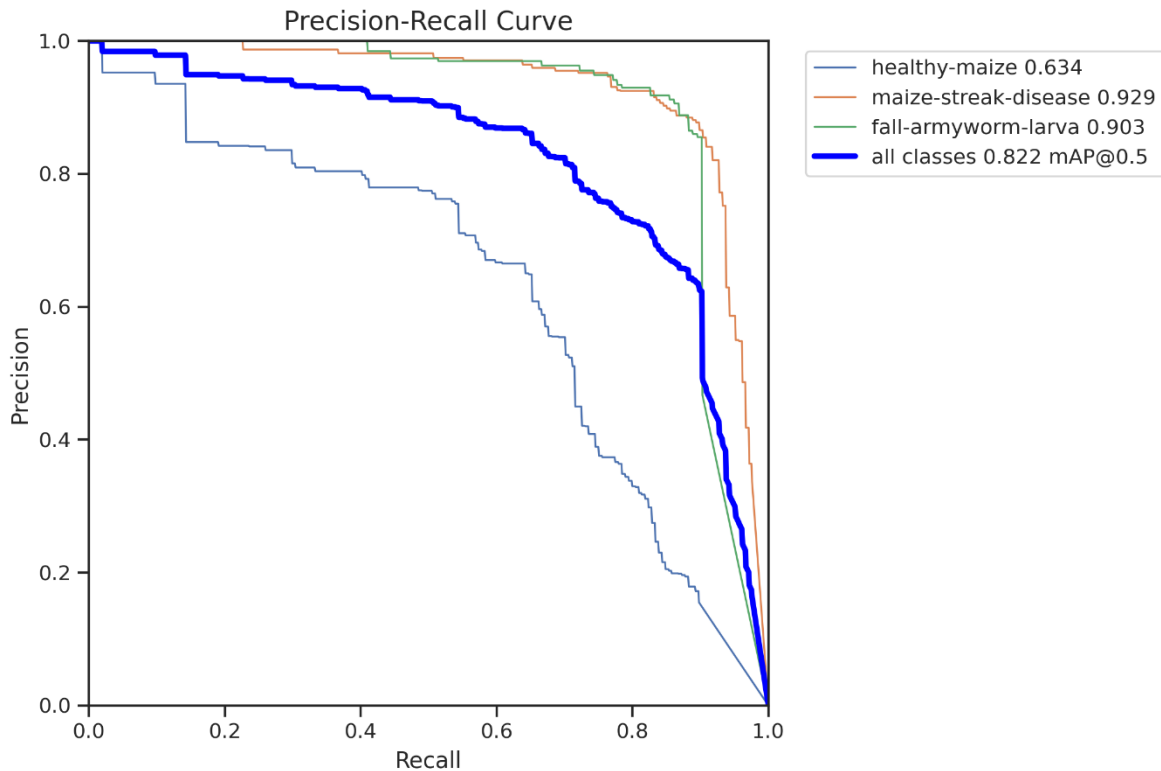


Figure 46: Courbes de Précision/Rappel sur le jeu de test (Precision-Recall Curve)

L'image représente **la Courbe de Précision-Rappel** de notre modèle de détection pour les classes **de maïs sain, de maladie de la striure du maïs et de chenille légionnaire**, sur le jeu de test. Cette courbe est une métrique de performance fondamentale qui illustre le compromis entre la précision et le rappel du modèle à différents seuils de confiance. Chaque point sur la courbe représente un couple (**rappel, précision**) obtenu en faisant varier le seuil de confiance des détections.

Pour les maladies et ravageurs (**maize-streak-disease et fall-armyworm-larva**) : Les courbes orange et verte sont très élevées, avec des valeurs **d'Average Precision (AP) de 0.929 et 0.903 respectivement**. Elles se maintiennent à une précision très haute même lorsque le rappel augmente, indiquant que le modèle est exceptionnellement performant pour détecter presque toutes les instances réelles de ces classes (haut rappel) tout en générant très peu de faux positifs (haute précision) sur des données inédites.

*Pour le maïs sain (**healthy-maize**) : La courbe bleue est significativement inférieure aux autres, avec une **Average Precision de 0.634**. La précision chute plus rapidement à mesure que le rappel augmente, ce qui indique un compromis plus difficile entre ces deux métriques pour cette classe sur le jeu de test. Cela suggère que le modèle rencontre plus de difficultés à maintenir une haute précision lorsqu'il essaie de détecter toutes les instances de maïs sain.*

Performance globale (all classes) : La ligne bleue épaisse, représentant la performance moyenne sur toutes les classes, montre **un mAP@0.5 global de 0.822**. Bien que cette valeur soit très bonne, elle est inférieure à celle observée sur le jeu d'entraînement, principalement en raison de la performance plus faible sur la classe healthy-maize. Néanmoins, un **mAP@0.5 de 0.822** sur un jeu de test inédit reste un indicateur solide de la capacité de généralisation du modèle.

En résumé, les **courbes de Précision-Rappel** sur le jeu de test confirment **l'excellente capacité du modèle à détecter les maladies et ravageurs du maïs avec une précision et un rappel élevé**. Cependant, elles mettent en évidence que la détection du **healthy-maize reste un défi plus important**, le modèle ayant plus de difficulté à maintenir un équilibre optimal entre précision et rappel pour cette classe sur de nouvelles données.

Section 3 Résultats d'inférence sur des images de test

L'inférence correspond à la phase où un modèle entraîné est utilisé pour faire des prédictions sur de nouvelles données, ici des images jamais vues auparavant. Concrètement, il s'agit de fournir une image en entrée du modèle pour que celui-ci identifie et localise les objets (maladies ou ravageurs) détectés, en retour, sous forme de bounding boxes annotées avec les noms des classes et les scores de confiance.

Visualisation des prédictions

Les figures suivantes illustrent le fonctionnement du modèle en conditions réelles à

travers des exemples d'inférence.

Cette première image montre une erreur d'orientation liée au fait que l'image n'a pas été correctement réorientée selon ses métadonnées EXIF, ce qui fait que les boîtes englobantes sont mal alignées avec les objets à détecter. Ce cas souligne l'importance de normaliser l'orientation des images avant inférence, surtout lorsqu'elles proviennent de smartphones ou d'appareils photo.



Figure 47: Effet d'une mauvaise orientation EXIF sur les prédictions du modèle

Chaque exemple présente à gauche l'image d'origine, et à droite la sortie du modèle avec les objets détectés (maladies ou ravageurs), leurs classes et scores de confiance. Ces résultats permettent de visualiser l'efficacité de la détection sur le terrain.

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_0febfcfc.jpg: 640x480 2 fall-armyworm-larvas, 92.5ms
Speed: 3.0ms preprocess, 92.5ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)



Figure 48: Exemple d'inférence correcte sur la classe *fall_armyworm*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_bf0b624c.jpg: 640x480 10 maize-streak-diseases, 92.3ms
Speed: 3.3ms preprocess, 92.3ms inference, 1.5ms postprocess per image at shape (1, 3, 640, 480)



Figure 49: Exemple d'inférence correcte sur la classe *maize_streak*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_c6e92e79.jpg: 640x640 18 healthy-maizes, 115.0ms
Speed: 3.0ms preprocess, 115.0ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 640)



Figure 50: Exemple d'inférence correcte sur la classe healthy

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_3d091ab5.jpg: 640x480 9 healthy-maizes, 93.4ms
Speed: 3.1ms preprocess, 93.4ms inference, 2.0ms postprocess per image at shape (1, 3, 640, 480)



Figure 51: Exemple d'inférence correcte sur la classe healthy

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_d8f0af0a.jpg: 640x480 8 maize-streak-diseases, 92.5ms
Speed: 2.6ms preprocess, 92.5ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)



Figure 52: Exemple d'inférence correcte sur la classe *maize_streak*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_b4780286.jpg: 480x640 1 fall-armyworm-larva, 96.6ms
Speed: 3.8ms preprocess, 96.6ms inference, 1.4ms postprocess per image at shape (1, 3, 480, 640)



Figure 53: Exemple d'inférence correcte sur la classe *fall_armyworm*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_8cd02806.jpg: 640x480 1 fall-armyworm-larva, 85.2ms
Speed: 2.8ms preprocess, 85.2ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)



Figure 54: Exemple d'inférence correcte sur la classe *fall_armyworm*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_71e5bc11.jpg: 480x640 7 maize-streak-diseases, 96.7ms
Speed: 2.5ms preprocess, 96.7ms inference, 1.4ms postprocess per image at shape (1, 3, 480, 640)



Figure 55: Exemple d'inférence correcte d'un cas vraiment précoce sur la classe *maize_streak*

image 1/1 /kaggle/input/computer-vision-corn-pest/COMPUTER_VISION_CORN_PEST/images/test/ID_be01f9f2.jpg: 640x480 1 fall-armyworm-larva, 82.5ms
Speed: 2.6ms preprocess, 82.5ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)

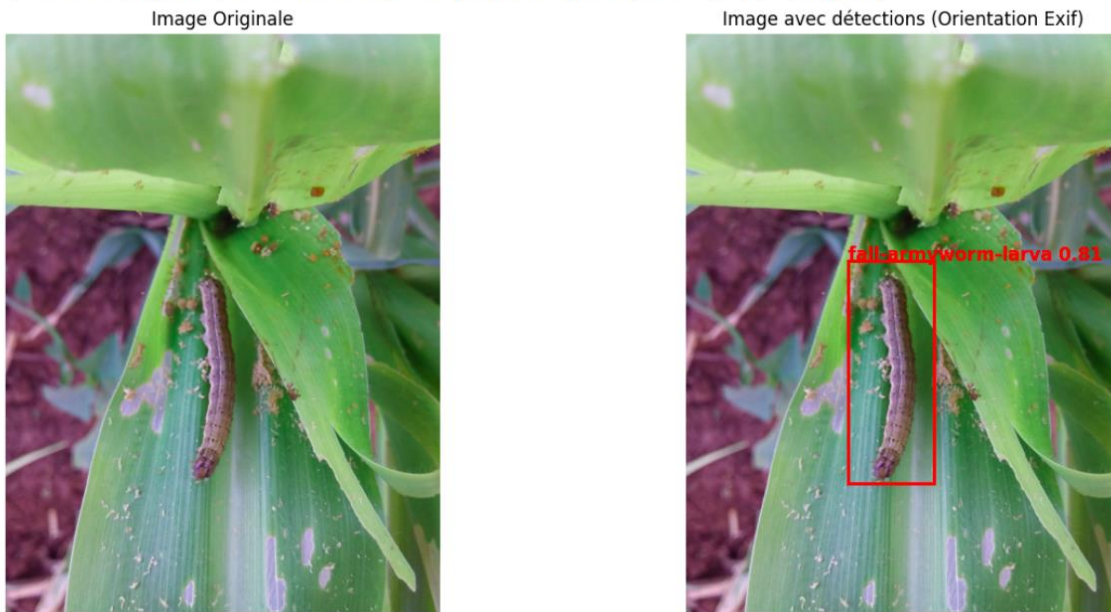


Figure 56: Exemple d'inférence correcte sur la classe *fall_armyworm*

Section 4. Interprétabilité du modèle : Visualisation des facteurs d'explicabilité

L'intelligence artificielle explicable (XAI) permet de comprendre les décisions prises par un modèle, en identifiant les zones de l'image qui ont influencé ses prédictions. Dans le contexte de la détection d'objets, cela se traduit par des cartes d'attention (**heatmaps**), qui montrent où le modèle « regarde » pour détecter une classe.

Nous avons utilisé la méthode **EigenCAM**, qui met en évidence ces régions à partir des activations internes du réseau, sans calcul de gradient. Cela permet d'analyser visuellement les mécanismes de décision du modèle, et de renforcer la confiance dans ses prédictions.

Pour chaque image analysée, nous présentons plusieurs vues complémentaires : l'image originale (entrée brute), l'image avec les prédictions du modèle (bounding boxes, classes, scores), trois heatmaps extraites à différents niveaux du backbone du modèle (ex. couches 16, 19 et 22), correspondant à des niveaux de détail différents

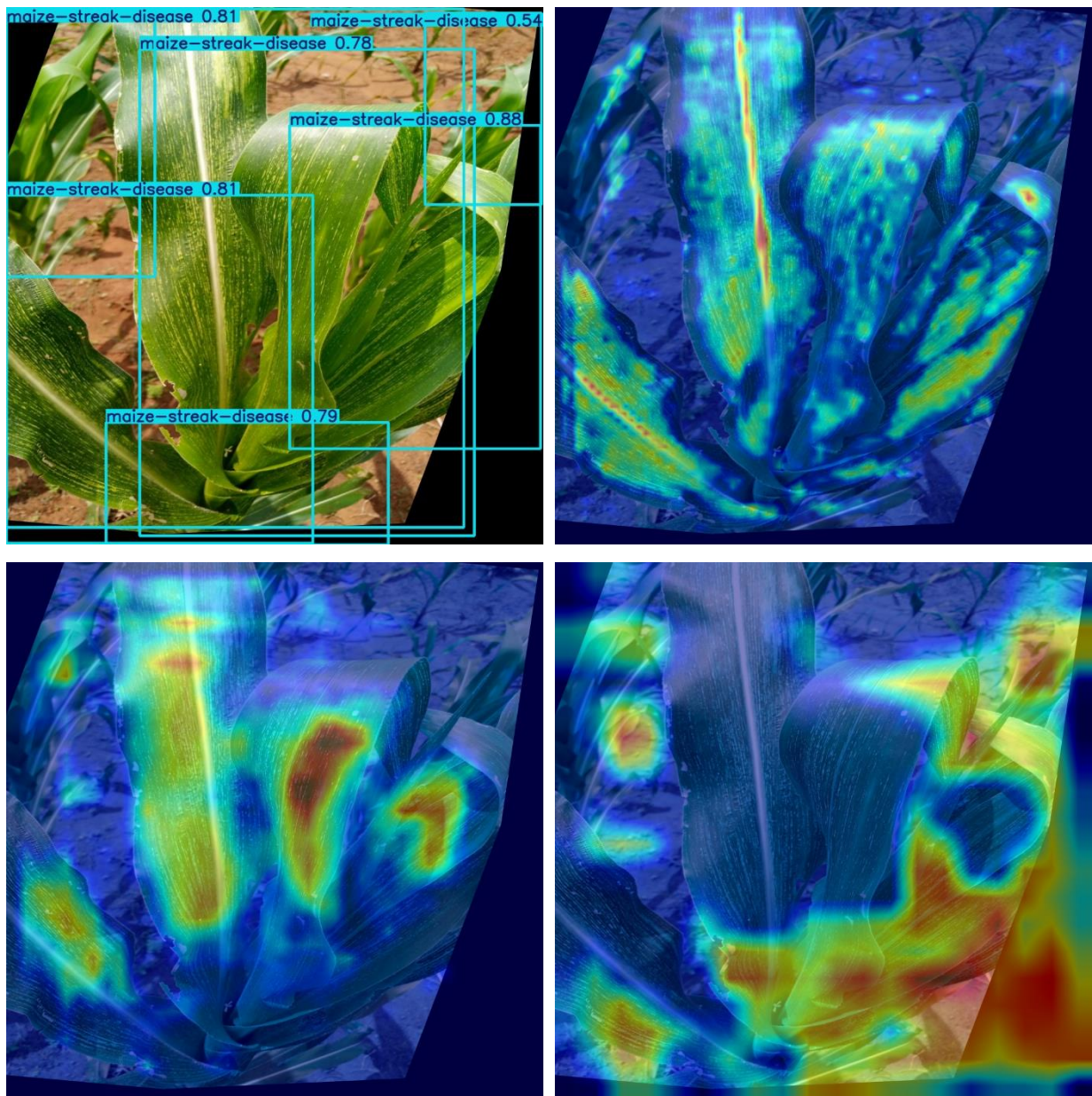


Tableau 3: Visualisation multi-niveaux des cartes CAM (large, medium, small) et détection finale associées à l'image ID_1972cd66(maize_streak).

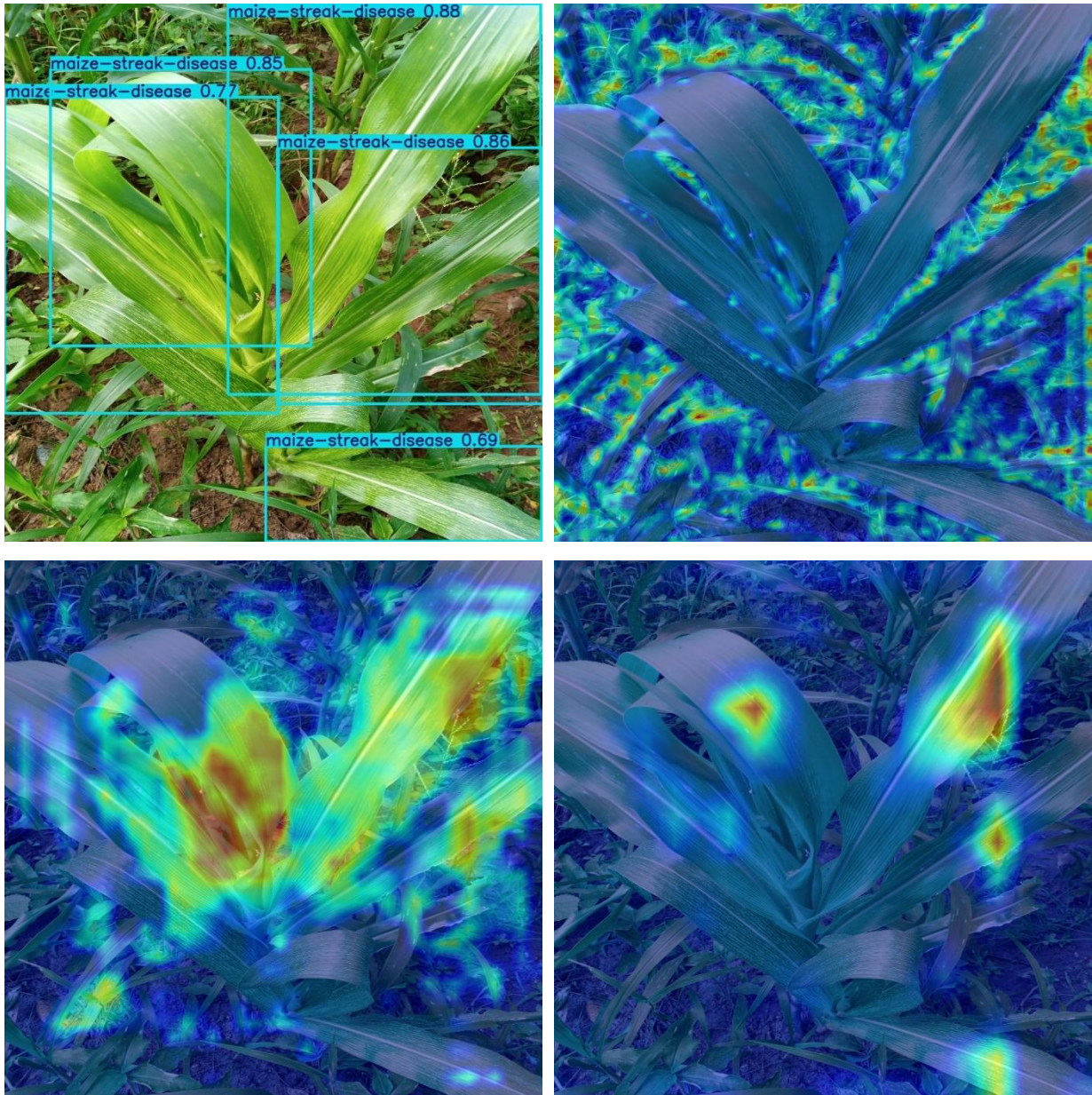


Tableau 4: Visualisation multi-niveaux des cartes CAM (large, medium, small) et détection finale associées à l'image ID_9695cfc8(maize_streak).

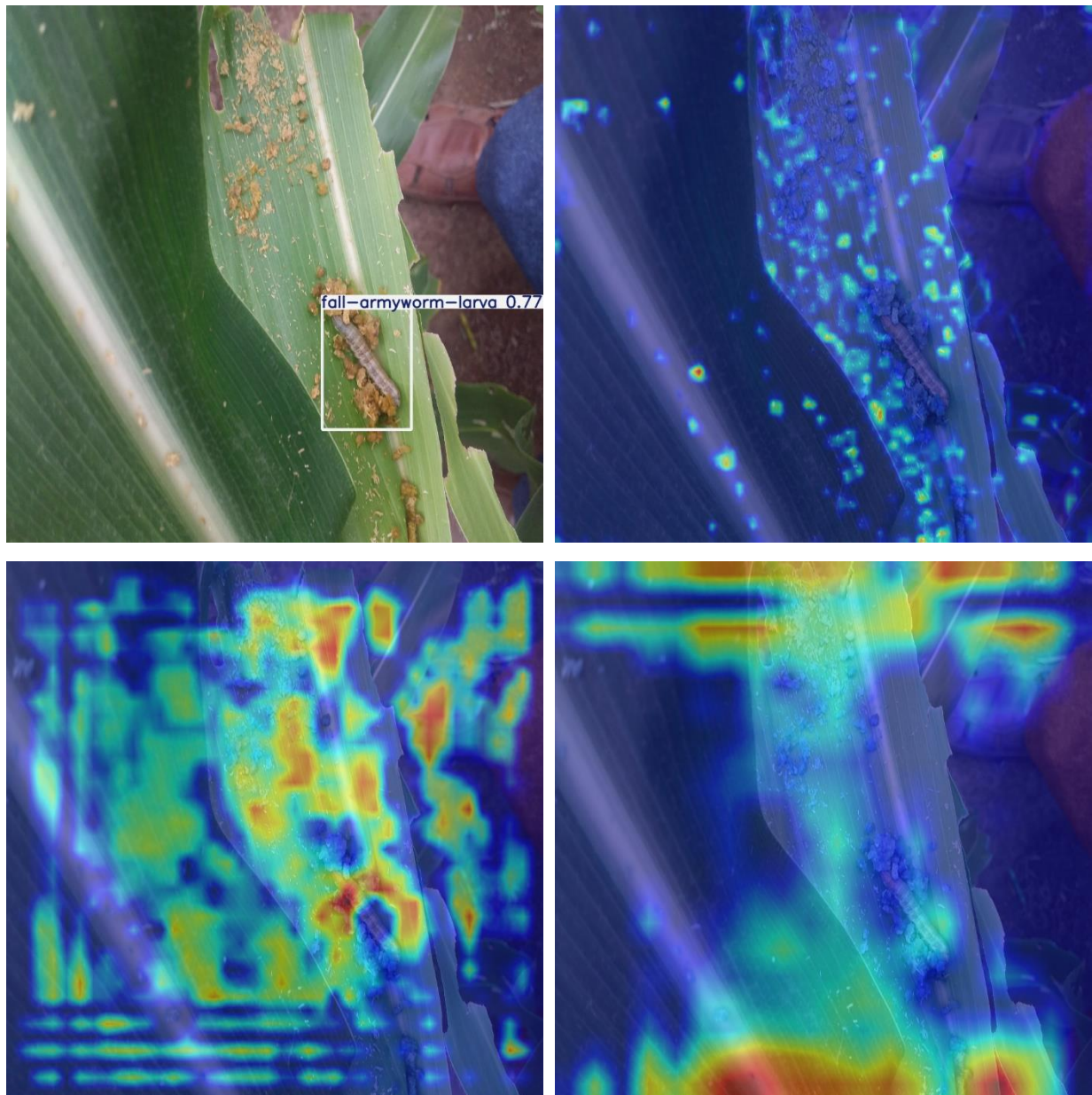


Tableau 5: Visualisation multi-niveaux des cartes CAM (large, medium, small) et détection finale associées à l'image ID_20672ec7(fall_armyworm)

Section 5 : Résultats de l'exportation du modèle pour périphériques embarqués

L'image ci-dessous illustre les différents formats de modèles exportés avec **TensorFlow Lite**, prêts à être intégrés sur des périphériques embarqués pour des cas d'usage en agriculture de terrain :

Modèle float32 : précision complète, utilisé pour des tests en laboratoire ou sur des ordinateurs portables ou stations de travail agricoles.

Modèle float16 : précision réduite pour plus de rapidité et moins de mémoire, adapté à des smartphones Android utilisés sur le terrain par les agriculteurs ou techniciens agricoles.

Modèle int8 : quantifié pour fonctionner sur des appareils très légers et peu énergivores comme : des drones agricoles pour l'inspection aérienne des plantations ,des kits embarqués de vision (Jetson Nano, Raspberry Pi + caméra) intégrés sur des robots agricoles, des terminaux portables utilisés dans des zones rurales à faible puissance de calcul.

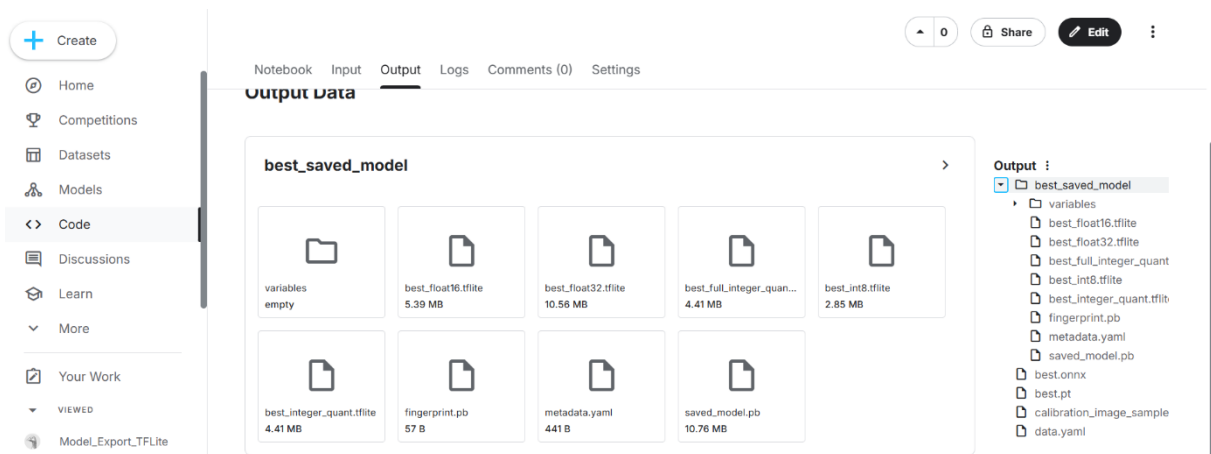


Figure 57: Modèles TensorFlow Lite exportés pour une utilisation sur différents types de dispositifs agricoles embarqués

Section 6 : Déploiement du Modèle via une Application Interactive et Autonome

Dans cette phase, nous avons intégré notre modèle entraîné dans une application développée avec **Streamlit**, afin de simuler son comportement en conditions réelles. Cette application permet de détecter automatiquement les principales affections du maïs, notamment la maladie des stries du maïs, causée par le virus de la striure du maïs, ainsi

que le ravageur **Fall Armyworm** (chenille légionnaire d'automne). Elle prend en charge plusieurs types de sources d'entrée, telles qu'une image importée, une vidéo locale ou un flux en temps réel via une webcam.

L'outil embarque également un module d'explicabilité basé sur la méthode **EigenCAM**, générant des cartes de chaleur qui mettent en évidence les zones de l'image ayant influencé la prédiction du modèle. Cette fonctionnalité renforce la transparence et la confiance dans les décisions de l'IA. L'ensemble du système fonctionne parfaitement, sans besoin de connexion Internet, ce qui le rend parfaitement adapté à une utilisation sur le terrain, y compris dans des zones rurales à faible connectivité.

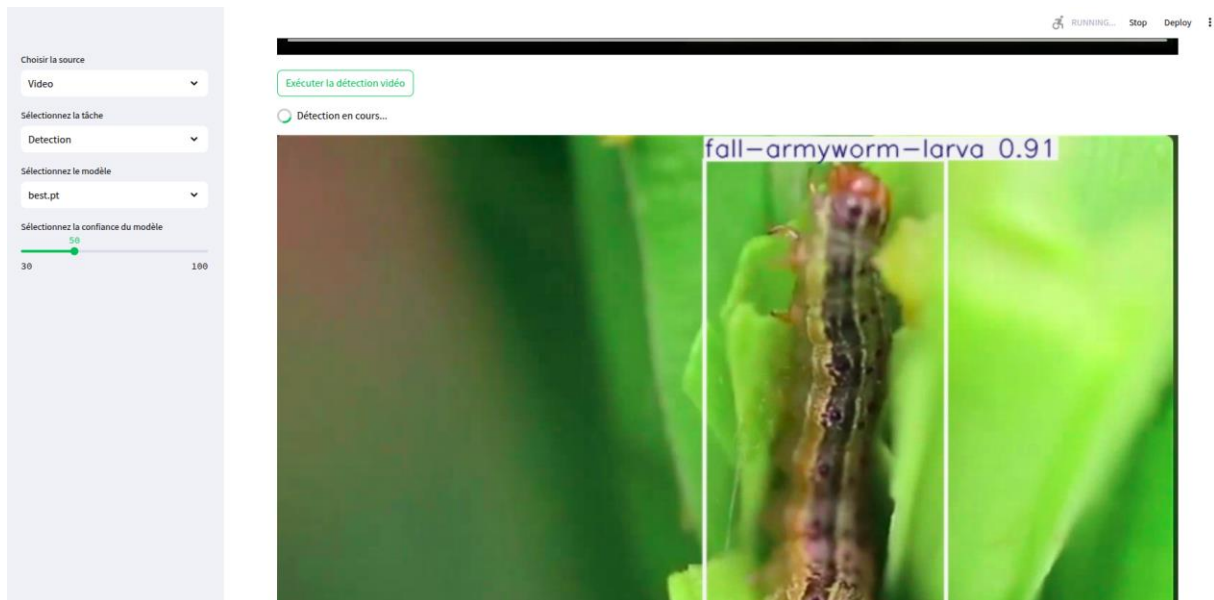


Figure 58: Interface Streamlit affichant une simulation de détection de maladies ou de ravageurs en temps réel au sein d'un flux vidéo.

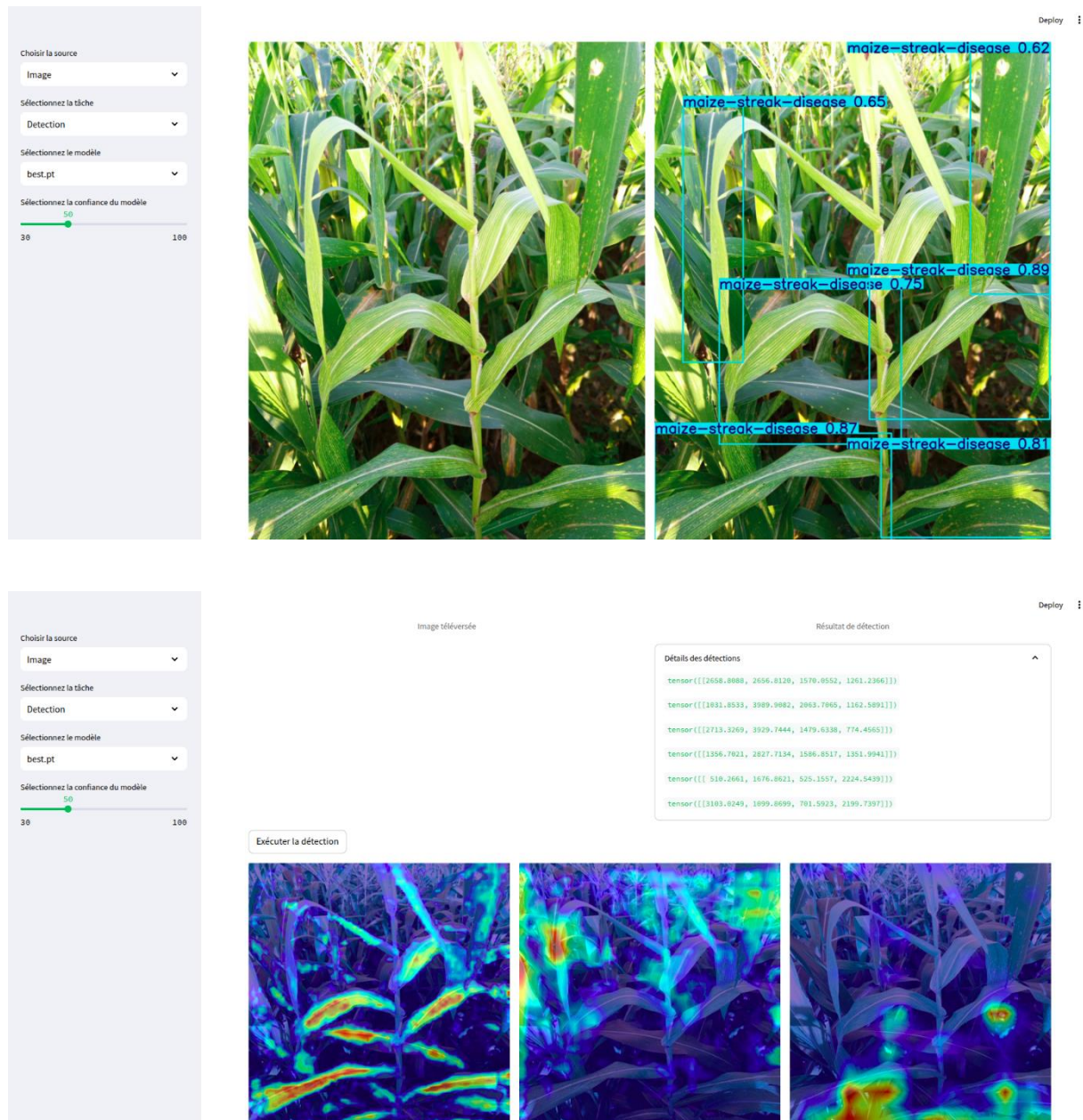


Figure 59: Interface Streamlit affichant une simulation de détection de maladies ou de ravageurs au sein d'une image de plantation de maïs atteint du virus de la striure

Section 7 : Discussions et perspectives

Bien que les performances globales de notre système de détection soient très performantes pour la détection de maladies concernées et de ravageurs, certaines limites subsistent,

notamment concernant la reconnaissance parfaite des plants sains. Cette difficulté représente un enjeu important pour une intégration future du système dans des robots ou drones agricoles autonomes chargés de la surveillance et de l'intervention ciblée dans les champs. En effet, il est essentiel que le modèle puisse non seulement détecter avec précision les maladies et ravageurs, mais aussi identifier clairement ce qui est sain . Pour cela, des efforts seront engagés afin d'améliorer la représentativité des images saines dans les données d'entraînement et d'optimiser la capacité du modèle à les reconnaître.

Par ailleurs, plusieurs pistes d'amélioration et d'extension sont envisagées pour renforcer la robustesse et l'utilité de la solution. Enrichir la base de données avec des images plus variées, couvrant différents stades de manifestations des infestations, permettrait d'améliorer la généralisation du modèle face à la diversité des conditions réelles sur le terrain. Le système pourrait aussi être adapté pour couvrir d'autres pathologies et ravageurs affectant des cultures critiques, ce qui élargirait considérablement son impact pratique. En complément, le développement et l'intégration d'un assistant virtuel ou bot agricole capable d'interagir de manière dynamique avec l'utilisateur constituent une piste prometteuse. Ce bot, orienté vers la compréhension des questions des agriculteurs et techniciens, pourrait fonctionner dans diverses langues locales et nationales afin de garantir une accessibilité maximale. Il offrirait des recommandations personnalisées précises sur les actions à mener, les produits phytosanitaires à utiliser ou les interventions adaptées à chaque situation détectée. En facilitant l'accès à des conseils pratiques et ciblés en temps réel, cet outil renforcerait la prise de décision sur le terrain, contribuerait à une gestion proactive et optimisée des cultures, et favoriserait ainsi une agriculture plus autonome, inclusive, durable et respectueuse de l'environnement.

Conclusion

Le développement et l'intégration des technologies d'intelligence artificielle et de vision par ordinateur dans l'agriculture ouvrent de nouvelles perspectives prometteuses pour relever les défis majeurs liés à la sécurité alimentaire, en particulier dans les régions où l'accès à l'expertise agricole reste limité. À travers cette étude, nous avons conçu un système intelligent dédié à la détection, localisation et diagnostic en temps réel des maladies et ravageurs affectant le maïs, culture essentielle pour l'Afrique subsaharienne. Le recours à une architecture performante comme YOLO, adaptée par fine-tuning à un jeu de données riche et spécifique, permet de garantir rapidité et précision, tout en assurant un fonctionnement efficace même sur des dispositifs mobiles à ressources limitées et hors connexion Internet. Ce système facilite ainsi un diagnostic autonome et instantané, contribuant à la réduction des pertes agricoles et à une meilleure gestion phytosanitaire. L'ajout de mécanismes d'explicabilité assure la transparence du système et renforce la confiance des agriculteurs dans ces outils numériques innovants. Au-delà de la détection spécifique des maladies et ravageurs du maïs, ce travail ouvre la voie à une extension vers d'autres cultures et pathologies, ainsi qu'au déploiement sur des plateformes automatisées telles que les drones agricoles. Ces perspectives permettront d'amplifier l'impact pratique de la solution, notamment en automatisant la surveillance à grande échelle et en optimisant l'usage des traitements phytosanitaires. En somme, ce projet illustre comment l'intelligence artificielle peut devenir un véritable levier pour la transformation numérique de l'agriculture, particulièrement en zones rurales, en offrant des outils accessibles, efficaces et adaptés aux réalités du terrain. Il contribue ainsi à poser les bases d'une agriculture de précision plus résiliente, économiquement viable et respectueuse de l'environnement, en phase avec les stratégies nationales et internationales pour le développement durable.

Références Webographiques & Bibliographiques

- [1] D. Ninja, *KaraAgroAI Maize Dataset*. [En ligne]. Disponible sur : <https://datasetninja.com/kara-agro-ai-maize>
- [2] République du Bénin, *Stratégie Nationale d'Intelligence Artificielle et des Mégadonnées 2023–2027*. [En ligne]. Disponible sur : <https://numerique.gouv.bj/assets/documents/strategie-nationale-d'intelligence-artificielle-et-des-megadonnees-2023-2027.pdf>
- [3] IBM, *Artificial Intelligence - IBM Think*. [En ligne]. Disponible sur : <https://www.ibm.com/fr-fr/think/topics/artificial-intelligence>
- [4] Machine Learnia, *Apprendre le Machine Learning en une semaine*. [En ligne]. Disponible sur : <https://www.machinelearnia.com/apprendre-le-machine-learning-en-une-semaine>
- [5] IBM, *Réseaux de Neurones - IBM Think*. [En ligne]. Disponible sur : <https://www.ibm.com/fr-fr/think/topics/neural-networks>
- [6] IBM, *AI vs ML vs DL vs NN*. [En ligne]. Disponible sur : <https://www.ibm.com/think/topics/ai-vs-machine-learning-vs-deep-learning-vs-neural-networks>
- [7] Ultralytics, *Définition de la Vision par Ordinateur (CV)*. [En ligne]. Disponible sur : <https://www.ultralytics.com/glossary/computer-vision-cv>
- [8] Ultralytics Docs, *YOLOv8 Detection Task*. [En ligne]. Disponible sur : <https://docs.ultralytics.com/tasks/detect/>
- [9] Ultralytics, *Glossaire CNN - Réseaux Neuronaux Convolutifs*. [En ligne]. Disponible sur : <https://www.ultralytics.com/glossary/convolutional-neural-network-cnn>
- [10] Ultralytics, *Composants Clés de la Convolution*. [En ligne]. Disponible sur :

<https://www.ultralytics.com/fr/glossary/convolution#composants-cles-de-la-convolution>

[11] GIMP, *Glossaire GIMP* 2.6. [En ligne]. Disponible sur : <https://docs.gimp.org/2.6/fr/glossary.html>

[12] MDPI, *Agronomy Journal*. [En ligne]. Disponible sur : <https://www.mdpi.com/2073-4395/13/2/521>

[13] arXiv, *Preprint* 2308.03766. [En ligne]. Disponible sur : <https://arxiv.org/abs/2308.03766>

[14] ScienceDirect, *Article scientifique* S2405844024042956. [En ligne]. Disponible sur : <https://www.sciencedirect.com/science/article/pii/S2405844024042956>

[15] Ultralytics, *CPU vs GPU vs TPU*. [En ligne]. Disponible sur : <https://www.ultralytics.com/glossary/cpu#cpu-vs-gpu-and-tpu>

[16] PyTorch, *Site officiel*. [En ligne]. Disponible sur : <https://pytorch.org/>

[17] NVIDIA, *Site officiel*. [En ligne]. Disponible sur : <https://www.nvidia.com/en-us/>

[18] OpenCV, *Site officiel*. [En ligne]. Disponible sur : <https://opencv.org/>

[19] Anaconda, *Site officiel*. [En ligne]. Disponible sur : <https://www.anaconda.com>

[20] Jupyter, *Site officiel*. [En ligne]. Disponible sur : <https://jupyter.org>

[21] Python, *Site officiel*. [En ligne]. Disponible sur : <https://www.python.org>

[22] TensorFlow, *Site officiel*. [En ligne]. Disponible sur : <https://www.tensorflow.org>

[23] Ultralytics Docs, *Intégration TFLite*. [En ligne]. Disponible sur : <https://docs.ultralytics.com/integrations/tflite/>

[24] Streamlit, *Site officiel*. [En ligne]. Disponible sur : <https://streamlit.io>

[25] Ultralytics Docs, *Documentation YOLOv11*. [En ligne]. Disponible sur : <https://docs.ultralytics.com/fr/models/yolo11/>

[26] YouTube, *Présentation Ultralytics YOLOv11*. [En ligne]. Disponible sur : <https://youtu.be/L9Va7Y9UT8E?si=Qt4CBxw63W2B4qDR>

[27] Ultralytics, *Mean Average Precision (mAP)*. [En ligne]. Disponible sur :

<https://www.ultralytics.com/glossary/mean-average-precision-map>

[28] Ultralytics, *Explainable AI (XAI)*. [En ligne]. Disponible sur :

<https://www.ultralytics.com/glossary/explainable-ai-xai>

[29] GitHub – rigvedrs, *YOLO-V11-CAM*. [En ligne]. Disponible sur :

<https://github.com/rigvedrs/YOLO-V11-CAM>

TABLE DES MATIÈRES

ENGAGEMENT	I
DEDICACES	II
REMERCIEMENTS	III
SIGLES ACRONYMES ET ABREVIATIONS	IV
LISTE DES FIGURES	VI
LISTE DES TABLEAUX	X
RESUME	XI
ABSTRACT	XIII
SOMMAIRE	XIV
INTRODUCTION	2
CHAPITRE 1 : REVUE DE LITTERATURE	6
SECTION 1 : DEFINITIONS DES THEMES ET CONCEPTS	6
1.1 Intelligence Artificielle	6
1.2 L'agriculture de précision	10
1.3 Détection d'objets avec les Réseaux de Neurones Convolutifs (CNN)	10
1.4 Les Réseaux de neurones convolutifs :	12
1.5 Définition théorique d'un filtre (ou noyau) et de la convolution en vision par ordinateur	15
1.6 Image numérique	16
1.7 Pixel	16
1.8 Apprentissage par transfert (Transfert Learning) avec les réseaux de neurones convolutifs	17
SECTION 2 : VUE GLOBALE SUR LES PLANTES DE MAIS	19
2.1 Définition	19
2.2 Maladie des Ravageurs des Plantes de Mais : causes, symptômes, mesures	

<i>préventives</i>	19
SECTION 3 : ÉTUDE ET CRITIQUE DE L'EXISTANT	22
3.1 <i>Publications scientifiques</i>	22
CHAPITRE 2 : ARCHITECTURE MATERIELLE, LOGICIELLE ET METHODOLOGIE DE MISE EN ŒUVRE	26
SECTION 1 : ARCHITECTURE MATERIELLE.....	26
SECTION 2 : ARCHITECTURE LOGICIELLE	28
SECTION 3 : ARCHITECTURE LOGIQUE DE LA SOLUTION	41
CHAPITRE 3 : RESULTATS ET DISCUSSIONS.....	70
SECTION 1 : RESULTATS SUR LE PIPELINE D'ENTRAINEMENT AVEC EVALUATION CONTINUE SUR LE JEU DE VALIDATION.....	70
SECTION 2 : ÉVALUATION DES PERFORMANCES SUR LE JEU DE TEST	82
SECTION 3 RESULTATS D'INFERENCE SUR DES IMAGES DE TEST	90
SECTION 4. INTERPRETABILITE DU MODELE : VISUALISATION DES FACTEURS D'EXPLICABILITE.....	96
SECTION 5 : RESULTATS DE L'EXPORTATION DU MODELE POUR PERIPHERIQUES EMBARQUES.....	99
SECTION 6 : DEPLOIEMENT DU MODELE VIA UNE APPLICATION INTERACTIVE ET AUTONOME.....	100
SECTION 7 : DISCUSSIONS ET PERSPECTIVES	102
CONCLUSION	104
REFERENCES WEBOGRAPHIQUES & BIBLIOGRAPHIQUES	XII
TABLE DES MATIÈRES	15

